Limber: Low Overhead SNARKs for Integers from Any PCS

Jessica Chen*
qc585@nyu.edu
New York University

Lucas Xia*
lucasxia@nyu.edu
New York University

Wilson Nguyen
wilsonnguyen@microsoft.com
New York University, Microsoft Research

Benedikt Bünz
bb@nyu.edu
New York University

October 19, 2026

Abstract

In real-world applications of SNARKs, non-native arithmetic, the emulation of computation outside of the specified SNARK field, is a key bottleneck. It introduces large overheads, and proof system designers often resort to non-standard SNARK-friendly hash functions or other means like elliptic curve cycles to mitigate its costs. Besides performance concerns, non-native circuit arithmetization is also a major cause of implementation errors. In a collection of 27 critical bugs in real world ZK systems (0xPARC/zkbugtracker [Zkb]), 9 were related to non-native arithmetization. We tackle these challenges by constructing a *minimal overhead* SNARK for integer computation that generically handles non-native arithmetic. We follow the recipe of Zaratan (PKC 26) [CHA24], which proves an integer relation such as $a \cdot b = c + u \cdot m$ by fingerprinting—reducing it to the same relation but over a randomly sampled prime field. Realizing this recipe requires an integer mod-PCS that commits to integer polynomials and opens their evaluations modulo a random prime, which is crucially chosen after the underlying PCS's setup and commitment phases. Our central contribution is *Limber*, the first practical integer mod-PCS construction that asymptotically has $o(1)$ multiplicative commitment overhead and can be instantiated with any standard field polynomial commitment scheme, including ones over small fields. Combining Limber with a PIOP for integer R1CS over the random prime yields our SNARK. We demonstrate its practicality by implementing our scheme and showing that we can prove an RSA accumulator benchmark 15–30$\times$ faster than circuit-based approaches, and non-native Poseidon2 hashing $24\times$ faster.

Keywords: SNARKs, Polynomial commitment schemes, Non-native arithmetic, Integer arithmetic, Fingerprinting

*Jessica Chen and Lucas Xia contributed equally to this work.

Contents

1 Introduction **1**
- 1.1 Related Work . . . 2

2 Variable Definitions and Notation **5**

3 Technical Overview **6**
- 3.1 Low-Overhead Arithmetization for Non-Native Arithmetic . . . 6
- 3.2 Fingerprinting through A Random Prime . . . 7
- 3.3 Mod-PIOP for Integer Moduli-R1CS over $\mathbb{Z}_p$ . . . 7
- 3.4 Goals of Integer Mod-PCS . . . 8
- 3.5 Building the IntEval Protocol . . . 9
- 3.6 All Together: Limber, an Integer Mod-PCS Compiler §5 . . . 13

4 Preliminaries **14**
- 4.1 Relation . . . 14
- 4.2 Limb-Splitting . . . 15
- 4.3 Polynomial Commitment Scheme (PCS) . . . 15
- 4.4 Interactive Proof and Arguments of Knowledge . . . 17
- 4.5 Polynomial Interactive Oracle Reduction and Polynomial Interactive Oracle Proof . . . 18
- 4.6 PIOP Compilation . . . 20
- 4.7 Mod-PIORs and Their Compilation . . . 20
- 4.8 Useful Relations . . . 21
- 4.9 Number of Primes and Fingerprinting of Polynomial Evaluation . . . 22

5 Limber: An Integer Mod-PCS Compiler **23**
- 5.1 Integer Mod-PCS Setup and Commitment Protocols . . . 23
- 5.2 Integer Mod-PCS Evaluation Protocol . . . 24
- 5.3 Integer Evaluation Protocol . . . 25
- 5.4 Setting the Parameters . . . 30
- 5.5 Overall Integer Mod-PCS Evaluation Protocol . . . 33

6 Spartan-Style PIOP for Integer Mod-R1CS **35**
- 6.1 Fingerprinting PIOR . . . 35
- 6.2 Spartan-Style PIOR for Integer Mod-R1CS Mod p . . . 37
- 6.3 Evaluating Sparse Matrices . . . 40
- 6.4 The Overall Mod-PIOP and Its Compilation into An Argument . . . 41
- 6.5 Coprocessor Model . . . 42

7 Implementation **42**
- 7.1 RSA Accumulator Application . . . 43
- 7.2 Mixed-Field Poseidon2 Hashing . . . 43
- 7.3 Overhead over Native Constraints . . . 43

8 Discussion and Future Work **45**

Acknowledgments **46**

References **46**

A Script for Computing Parameters of Integer Evaluation Protocol **51**

B Additional Proofs **51**

C Use of Generative AI **52**

1 Introduction

A SNARK (succinct non-interactive argument of knowledge) lets a prover convince a verifier that a computation was performed correctly, using a proof that is far smaller and faster to check than rerunning the computation.

Most modern SNARKs natively prove statements expressed as arithmetic circuits over a finite field $\mathbb{F}$. This creates a disconnect with many instances of real-world computation, including cryptographic operations (e.g., RSA-style arithmetic, elliptic curve additions, hash functions, and recursive verifier logic), CPU-style execution of integer arithmetic, LLM evaluations, and floating-point style computations [BCTV14a; KPS18a; LXZ21; Sav+21; GJJZ22; AST24]. To be proven, any such computation must first be re-expressed as a circuit over $\mathbb{F}$; this emulation is known as *non-native arithmetic*, and it is as costly as it is ubiquitous. In practice, non-native arithmetic is often a dominant prover bottleneck that substantially bloats circuit size and prover time by $100\times$ or more over native computation. Circuit optimization techniques such as those in Section 1.1.1 reduce the constraint count, but a substantial overhead remains in practice. This seems fundamental as circuit-based techniques must emulate a multiplication algorithm. Schoolbook multiplication has a quadratic overhead, and faster multiplication algorithms, such as Karatsuba or FFTs, have limited benefits for small instance sizes.

Motivated by this overhead, researchers as well as practitioners go to great lengths to avoid it whenever possible. This includes the use of cycles of elliptic curves [Ben+14; BCTV14b; KST22; KS23] and arithmetization-friendly hash functions [GKKRRS19; AGRRT16; GKS23; AABDS20], among other imperfect solutions that introduce additional overhead, complexity, and new non-standard assumptions. On the other hand, optimized circuits for non-native arithmetic are notoriously tricky to implement. A bug-tracker [Zkb] for real-world SNARK deployments lists 27 bugs, 9 of which stem from non-native arithmetic, and all of which could have led to significant loss of funds. Recently, a non-native arithmetic soundness bug triggered a $\$450k$ USD bug bounty payout; had it gone undiscovered, it could have been catastrophic [Edi23].

In this work, we develop efficient and robust SNARKs for *integer* computations, making non-native arithmetic practical and obviating the need for error-prone and costly circuit-based arithmetization. Our approach allows circuits to express arithmetic over arbitrary moduli, including multiple distinct moduli in a single computation and moduli that may be larger than the SNARK field. Furthermore, unlike prior approaches [CHA24; GWHD25] which are based on specific polynomial commitment schemes each with their own downsides and assumptions, our approach is compatible with arbitrary field polynomial commitment schemes, which can be instantiated from a variety of assumptions. Unlike another mod-PCS compiler work (Qaren) [SSPP26], Limber is highly practical and additionally works over small characteristic fields ($q = \Omega(\lambda^2 n^2)$ where n is the number of variables of the polynomial vs. $\Omega(2^\lambda)$ for Qaren) which can be desirable for prover performance.

SNARKs for Integer Relations. Modern SNARKs [Set20; CBBZ23; GWC19] are designed by combining a polynomial interactive oracle proof (PIOP) with a polynomial commitment scheme (PCS). The PIOP is an information-theoretic protocol that proves the NP relation of interest, and the PCS is used to instantiate the oracles in the PIOP. This separation is not entirely modular. Most PCS[a] require defining the commitment field $\mathbb{F}$ during the setup or the commitment phase, where $\mathbb{F}$ is often chosen for having nice properties like having a Solinas (generalized Mersenne) prime characteristic or being the scalar field of a secure elliptic curve. A PIOP for an arithmetic circuit needs to match the field $\mathbb{F}$ of the PCS scheme in order to compile to a SNARK. Therefore, if we would like to prove computation over a domain or multiple domains like $\mathbb{Z}_{2^n}$ or $\mathbb{Z}_{pq}$ that are not $\mathbb{F}$, we could attempt to instead build a circuit over the integers. Integer arithmetic can enable

[a] Group of unknown order schemes such as DARK [BFS20] and Dew [KKSKKO23] being the notable exceptions.

arbitrary modular arithmetic by proving that $a \cdot b = c + u \cdot m$ for some quotient u and modulus m, but this requires a PIOP and PCS scheme that works over the integers. Zaratan [CHA24] first showed that such a PIOP can be built by reducing the integer constraints modulo a random prime p (fingerprinting). For their PCS construction, they rely on a group of unknown order construction [BFS20; BHRRS21]. Unfortunately, groups of unknown order are practically expensive (on the order of $20\times$ the cost of elliptic curves), and are not post-quantum secure. Zinc(+) [GWHD25; Abd+26] constructs an alternative hash-based integer PCS, using a generalization of the Blaze/Reed-Solomon code [BCFRRZ24] and the Brakedown polynomial commitment [GLSTW23]. However, this construction leads to asymptotically $\sqrt{N}$-sized proofs, higher prover cost due to the use of integer codes, and again limits the choice of PCS to a specific construction. Restricting the choice of PCS sacrifices modularity, which makes it hard to adopt future advances in PCS design and impedes integration with the many existing libraries built on other schemes.

Contributions. In this work, we contribute the following:

- We construct *Limber* (§5), the first (almost[b]) fully PCS-agnostic integer mod-PCS compiler.
  - It has minimal commitment overhead—it can be parameterized such that committing and opening a N-sized polynomial only requires committing to $N + o(N)$ data.
  - It leverages a limb-splitting technique for handling larger integers. In particular, the technique helps us avoid an increase in the size of the R1CS matrices and only adds a small sumcheck to the evaluation protocol.
- We compile a SNARK for integers from Limber and a Spartan-style PIOP (§6), enabling proving arbitrary modular arithmetic with low overhead.
- We implement our SNARK construction and evaluate it (§7) against optimized circuit-based approaches for an RSA accumulator application, achieving 15–$30\times$ prover speedups over the circuit-based approaches and a $4.8\times$ speedup over the concurrent integer SNARK Zinc+, and on a mixed-field Poseidon2 hashing benchmark, where our prover is $24\times$ faster than the circuit-based approach.

1.1 Related Work

1.1.1 Circuit Techniques for Non-Native Arithmetic

A naive approach to a non-native modular multiplication $a \cdot b = c + u \cdot m$ is to limb-split all values, perform schoolbook multiplication, sum the partial products at each limb position, and then resolve the carries before checking that each output limb is correct. This schoolbook multiplication method uses a number of constraints quadratic in the number of limbs. For instance, in the popular Arkworks [ark22] SNARK library, emulating a single non-native 381-bit multiplication in a 255-bit field requires 1032 constraints and 12447 non-zero entries.

To address this blowup, many prior works propose circuit optimization techniques to reduce the number of constraints. For example, xJsnark [KPS18b] uses a multiplication technique to reduce the number of constraints for multiplying two m-bit integers from $O(m^2)$ to $O(m)$. However, the density of the R1CS constraint matrices A, B, C (which encode the circuit) is still quadratic in m, which results in a quadratic cost during setup and polynomial evaluation. MultiSwap [OWWB20], which uses xJsnark, requires 10.7 million R1CS constraints to evaluate a hash to a prime and verify two RSA accumulator updates via Wesolowski proofs of exponentiation [Wes19].

[b] Defined over a field $\mathbb{F}$ with characteristic $\mathrm{char}(\mathbb{F}) = \Omega(\lambda^2 \cdot n^2)$ for $n = \log_2(N)$

In Casting Out Primes [LM22] (used in libraries such as Plonky2 [Pol22]) and in Efficient Foreign Field Arithmetic [AFQ25], the prover shows that the result of a non-native operation is correct modulo sufficiently many small coprime moduli, which implies the validity of the original operation. This Chinese Remainder Theorem (CRT) trick allows them to reduce the sizes of the witness values relative to the small moduli. In turn, this helps them avoid overflowing the field, which would otherwise nullify any integer equivalence (see Section 3 for more details on how this is similarly used in our construction). It also allows for an optimization of using the native field modulus as one of the CRT moduli.

In Aztec's library for foreign field arithmetic, BigField [Azt24], the prover combines the CRT trick with limb-splitting, achieving ≈40 constraints (excluding range checks) for a foreign-field multiplication.

The main issue with these techniques is that the quadratic limb-by-limb product work does not disappear, which factors directly into the prover's commitment costs, often the biggest bottleneck in performance. Perhaps more crucially, optimizing constraints in this fashion can be particularly bug-prone: it notably led to a critical soundness bug for which a $450k bounty was paid [Edi23], and is the culprit behind numerous other bugs in real-world SNARK deployments [Zkb].

To compare against the above methods, we provide a concrete evaluation of our construction on a benchmark involving two RSA accumulator updates and a Poseidon-based hash-to-prime evaluation from [OWWB20]. We compare against two circuit-based approaches for this statement—a naive one built from the Arkworks [ark22] non-native arithmetic gadgets, and the hand-optimized MultiSwap circuit using xJsnark techniques [KPS18b], both proven with Spartan [Set20]. We also compare against Zinc+ [Abd+26], a concurrent integer mod-PCS approach, in Table 1. This benchmark was chosen because it serves as a representative example of a real-world SNARK application, with multiple prior works specifically optimizing this computation [OWWB20; CFHKKO21], but we also provide another benchmark of non-native Poseidon2 hashing in Section 7.2.

In addition to our verifiers being faster and our (Hyrax) proofs smaller than those of the circuit-based approaches, our provers are significantly faster, achieving a $15\times$ speedup over the MultiSwap prover and a $30\times$ speedup over the Arkworks circuit prover. For the concurrent Zinc+ construction that uses a code-based integer PCS (Zip+), we wrote the circuit in their framework (see the footnotes of Table 1). Our Hyrax prover is $4.8\times$ faster, our verifier is $5.8\times$ faster, and our proofs are $3.4\times$ smaller and our Brakedown prover achieves similar prover and verifier times with a larger proof. We note that we do not compare against other integer SNARK works such as Zaratan [CHA24] or Qaren [SSPP26] because they do not provide an implementation.

	# Constraints	Prove	Verify	Proof size
Arkworks [ark22] circuit (Hyrax + Spartan)	26.4 million	63.7s	5.74s	857KB
MultiSwap [OWWB20] circuit (Hyrax + Spartan)	10.7 million	31.5s	4.30s	463KB
Zinc+ [Abd+26] (concurrent)	$2^{11} \times 248$ trace[c]	10.2s	363ms	0.94MB
Limber-Spartan (with Hyrax)	12,796	2.11s	63ms	273KB
Limber-Spartan (with Brakedown)	12,796	2.07s	59ms	8.2MB

Table 1: We evaluate our implementation with an execution involving two RSA accumulator updates and a hash-to-prime evaluation. The constraint counts for this work are integer constraints; the Zinc+ count is the size of its execution trace; the others are normal R1CS constraints. The Spartan rows use the same Spartan prover as this work, instantiated with Hyrax over BLS12-381 (the field of both circuits). Proof sizes are zstd-compressed. All numbers are **single-threaded** benchmark results on a MacBook (Apple M4 Pro chip, 24 GB RAM, 14 cores).

[c] Zinc+ does not benchmark this statement, so we ported the circuit into their AIR-style framework. Zinc+ constraints are the

1.1.2 Integer Mod-PCS

In Table 2 we provide a detailed comparison to the most relevant works that also construct integer mod-PCS to achieve SNARKs for integers: Zaratan [CHA24], Zinc [GWHD25] and the concurrent Zinc+ [Abd+26] (whose mod-PCSs are called Zip and Zip+, respectively), and Qaren [SSPP26]. We compare the integer mod-PCS constructions in these works in terms of commitment overhead, prover cost, and verifier cost (in bit operations). Note that Zip, Zip+, and Qaren technically all construct rational mod-PCSs, i.e. they extract bounded rational polynomials instead of integer ones, which can work for specific applications like fractional currency but not many common ones. Rational PCSs can be combined with additional range proofs to convert them to integer PCSs. Limber, in contrast, builds an integer PCS directly, avoiding a conversion step.

Zaratan [CHA24], the first integer mod-PCS construction, is based on DARK, a PCS over groups of unknown order [BFS20]. This PCS has large concrete overheads and is not quantum secure. Unlike the rational constructions, Zaratan extracts integer polynomials directly but does so through binary challenges and repetition; these challenges are sound in the interactive setting but we note that they are vulnerable to grinding attacks in the non-interactive setting [AFK21].

Zip(+) [GWHD25; Abd+26] are code-based integer PCSs in the style of Ligero/Brakedown [AHIV17; GLSTW23], with $\sqrt{N}$ verifier time. While they do not need to commit to extra elements, they incur performance overheads from the use of a custom integer code. In Zinc, this is a variant of the Blaze RAA code, and in Zinc+, it is a new pseudo-integer Reed-Solomon code, which ties Zinc(+) to this single style of PCS construction. Zinc+, which is concurrent, also supports a new arithmetization called universal constraint system (UCS), that includes constraints over polynomial rings. We leave it for future work whether the techniques of Limber can be adapted to UCS-style constraints.

The most immediate comparison is Qaren [SSPP26], the first integer PCS compiler, that is compatible with a wide class of underlying PCSs. There are, however, two main performance downsides of Qaren compared to Limber:

1. The Qaren compiler introduces at least $14 \log N$ commitment overhead from having to do an integer commitment on the product of two polynomials evaluated at a massive point $\alpha = q^{7 \log N}$. In practice, this translates to a significant overhead on the order of $100\times$ or more. Limber has $o(1)$ multiplicative commitment overhead.
2. The Qaren integer commitment scheme only supports committing to the bit decomposition of the integer. For group-based systems, this results in another $\log N$-factor overhead over Pippenger's algorithm, and for hash-based systems, this adds a λ-factor overhead as each bit is represented as an $O(\lambda)$-sized field element. Limber can support a more general limb-splitting of the original witness that avoids such large overheads.

rows and columns of an execution trace rather than R1CS constraints

Integer mod-PCS	PCS Compatibility	Commitment Overhead	Prover cost	Verifier cost
Zaratan PCS [CHA24]	DARK	$\mathsf{poly}(\lambda)$	$N \log N \mathsf{poly}(\lambda)$	$\mathsf{poly}(\lambda, \log N)$
Zip [GWHD25]	Int-Brakedown	None	$N\lambda$	$\sqrt{N\lambda}$
Zip+ [Abd+26] (concurrent)	Int-Brakedown	None	$N\lambda \log N$	$\sqrt{N\lambda}$
Qaren [SSPP26]	Some	$\log N$	$N\lambda \log(N\lambda)$	$\log(N + \lambda)$
Limber (this work)	Any	$o(1)$	$N\lambda \log \lambda$	$\lambda \log N \log \lambda$

Table 2: We consider a n-variable multilinear polynomial f whose hypercube evaluations have magnitude at most $2^{O(\lambda)}$, i.e., they are $O(\lambda)$-bit values (in the technical sections we write this magnitude bound as T_f). $N := 2^n$. All the costs in the table are asymptotic and the big-O notation is omitted. For simplicity of comparison, we assume that multiplications with fewer than λ bits only require a linear amount of operations. PCS compatibility refers to the ability to use various kinds of underlying PCSs for constructing the integer mod-PCS. The commitment overhead refers to the multiplicative amount of extra elements committed beyond just the input polynomial. The prover cost includes the commitment work, integer and field work, and opening cost.

2 Variable Definitions and Notation

Variable Definitions: First, we provide a list of definitions for variables that we will use consistently throughout this work, along with a set of practical values.

	Definition	Examples
$\mathbb{F}$	Underlying field of the PCS	—
q	Characteristic of $\mathbb{F}$	64, 128, 256-bit prime
p	Random prime that mod-R1CS PIOP is run over	128-bit prime
λ	Security parameter	128
N	# of R1CS constraints, variables, length of polys, 2^n	2^{10} to 2^{30}
n	$\log N$	10 to 30
T_f	Norm bound on integers	2^{32} to 2^{2048}
T	Norm bound on a limb of a value; a T_f-bounded value splits into limbs of bound T (Section 3.5.5)	2^{32} or 2^{64}
l	Log number of limbs for integer	0 to 6
m	# of variables of limb-split polynomial, $n + l$	10 to 36
k	# of var. partially evaluated per iteration in IntEval	1 to 10
P	Size of IntEval randomly sampled primes	2^{20} to 2^{40}
s	# of IntEval randomly sampled primes for λ-bit security	5 to 10

Notation: We use $\mathsf{Primes}(a, b)$ to denote the set of primes in the range $[a, b)$. In particular, $|\mathsf{Primes}(a, b)| = \pi(b) - \pi(a)$, where $\pi(x)$ denotes the number of primes less than x.
We use the hat to signify an element in $\mathbb{F}$, e.g. $\hat{x} \in \mathbb{F}$. For integer elements x, we do not dress them with any

extra notation.
We use $\mathbb{F}^{\leq d}[n]$ to denote the set of all polynomials in $\mathbb{F}$ with n variables and individual degree d. $\mathbb{F}^{\leq 1}[n]$ represents a multilinear polynomial.
For any n-variate multilinear polynomial $f \in \mathbb{F}^{\leq 1}[n]$, we define $||f||$ as the maximum absolute value of its evaluations over the Boolean hypercube $\{0,1\}^n$, and we use $[[f]]$ to denote the polynomial oracle of f.
For any n_1-variate multlinear polynomial f_1 and n_2-variate multilinear polynomial f_2, we define $f := (f_1, f_2)$ to be the polynomial with $n := \lceil \log(2^{n_1} + 2^{n_2}) \rceil$ variables, whose evaluations over $\{0,1\}^n$ are the evaluations of f_1 over $\{0,1\}^{n_1}$ concatenated with the ones of f_2 over $\{0,1\}^{n_2}$, padded with zeros up to 2^n.
For any prime $q \in \mathbb{N}$, field $\mathbb{F}$ with characteristic q, and any vector $\boldsymbol{v} \in \mathbb{F}^N$, we define the `toInt` function as the canonical lifting from a field vector to an integer vector of norm less than q, namely $\boldsymbol{v} \in \mathbb{Z}^N_{<q} \leftarrow \texttt{toInt}(\hat{\boldsymbol{v}} \in \mathbb{F}^N)$. If $\hat{\boldsymbol{v}}$ is the vector of $\mathbb{F}$ evaluations of some polynomial $\hat{f}$ over the Boolean hypercube, then the integer polynomial $f \in \mathbb{Z}^{\leq d}[n] \leftarrow \texttt{toInt}(\hat{f} \in \mathbb{F}^{\leq d}[n])$, where f is the polynomial with evaluations $\boldsymbol{v} \leftarrow \texttt{toInt}(\hat{\boldsymbol{v}})$. Similarly, we define the `toField` function as the canonical casting from an integer vector with magnitude less than q to a field vector, namely $\hat{\boldsymbol{v}} \in \mathbb{F}^N \leftarrow \texttt{toField}(\boldsymbol{v} \in \mathbb{Z}^N_q)$.

3 Technical Overview

In this section, we outline the intuition and high-level details of our constructions.

3.1 Low-Overhead Arithmetization for Non-Native Arithmetic

Motivated by the problem of non-native arithmetic, we can transform the standard R1CS relation into one that supports modular arithmetic, as proposed in Zinc [GWHD25]. The following is an informal definition for the moduli-R1CS relation (see formal definition in Definition 6.1).

Definition 3.1 (Informal)**.** *The indexed relation* $\mathsf{R}_{\mathsf{imodR1CS}}$ *is the set of all tuples*

$$(\mathbb{i}; \mathbb{x}; \mathbb{w}) = ((A, B, C, \mathsf{mods}); \boldsymbol{x}; (\boldsymbol{w}, \mathsf{quos}))$$

where A, B, C *are integer matrices and* mods *(moduli),* $\boldsymbol{x}$ *(public input),* $\boldsymbol{w}$ *(witness), and* quos *(quotients) are integer vectors, such that for* $\boldsymbol{z} := (\boldsymbol{x}, \boldsymbol{w})$*, the following equation over integer vectors and matrices holds,*

$$A\boldsymbol{z} \circ B\boldsymbol{z} = C\boldsymbol{z} + \mathsf{mods} \circ \mathsf{quos} \tag{1}$$

This arithmetization naturally supports modular operations by adding an extra modulus and quotient value for each constraint. As a result, there are no extra intermediate values that need to be computed, like there are in naive limb-by-limb multiplication, and it directly can support both large and distinct moduli.

Another key distinction between $\mathsf{R}_{\mathsf{imodR1CS}}$ and the standard R1CS relation is that $\mathsf{R}_{\mathsf{imodR1CS}}$ is over integers, whereas the equality in standard R1CS holds over a finite field. A finite field is not sufficient for the moduli-R1CS relation because an adversary can always solve for quos in the field as follows: $\mathsf{quos}_i = (A_i\boldsymbol{z} \cdot B_i\boldsymbol{z} - C_i\boldsymbol{z})\mathsf{mods}_i^{-1}$ for each constraint i such that $\mathsf{mods}_i \neq 0$. Therefore, the challenge here is proving this relation over the integers rather than over a finite field.

Integers are prohibitively expensive for the prover, particularly because their size can grow exponentially in the depth of the prover algorithm. As a result, a naive SNARK over the integers will suffer large overheads in prover performance.

3.2 Fingerprinting through A Random Prime

To avoid working directly with integers, we adopt the classical *fingerprinting* technique (cf. Reed–Solomon fingerprinting [Tha20]), as used in [CHA24], to reduce the integer relation above to one over a finite field $\mathbb{Z}_p$. Note that the random fingerprinting prime p is distinct from, and sampled independently of, the characteristic q of the PCS field $\mathbb{F}$.

Consider the following informal definition for the moduli-R1CS relation over a random prime p (see formal definition in Definition 6.2).

Definition 3.2 (Informal)**.** *The indexed relation* $\mathsf{R}_{\mathsf{imodR1CS}}(p)$ *is the set of all tuples*

$$(\mathbb{i}; \mathbb{x}; \mathbb{w}) = ((A, B, C, \mathsf{mods}); \boldsymbol{x}; (\boldsymbol{w}, \mathsf{quos}))$$

such that $\boldsymbol{z} := (\boldsymbol{x}, \boldsymbol{w})$ *and the following holds,*

$$A\boldsymbol{z} \circ B\boldsymbol{z} \equiv_p C\boldsymbol{z} + \mathsf{mods} \circ \mathsf{quos} \tag{2}$$

Instead of checking the moduli-R1CS relation over the integers, what if the verifier instead checks the relation over a random prime p? More precisely, the question is: is it sufficient to prove $\mathsf{R}_{\mathsf{imodR1CS}}(p)$ instead of $\mathsf{R}_{\mathsf{imodR1CS}}$?

If the relation holds over the integers, then the relation holds over the random prime p unconditionally. Moreover, the negation holds - if the relation does not hold over the integers then it won't hold over a random prime p, with overwhelming probability. The fingerprinting lemma can be simply stated as follows (see Lemma 4.29 for the formal lemma and proof):

Lemma 3.3 (Informal)**.** *For a uniformly random λ-bit prime p, integer $y \neq 0$, the probability that $y = 0 \bmod p$ is upper-bounded by approximately $\frac{\log |y|}{2^\lambda}$.*

With bounds on $A, B, C, \mathsf{mods}, \boldsymbol{z}, \mathsf{quos}$, we can get a reasonable upper bound (in the number of bits) on the value of each row. Then, with a large enough prime p (say λ-bit), if there exists a row i that is not satisfied in the integer relation, then there is a negligible probability in λ that the same row is satisfied in the mod p relation.

Constructing a SNARK for $\mathsf{R}_{\mathsf{imodR1CS}}(p)$. A common recipe for obtaining a SNARK for such NP relations is to construct an information-theoretic Polynomial Interactive Oracle Proof (PIOP), and then instantiate the oracles in the PIOP with an appropriate Polynomial Commitment Scheme (PCS) over a field [BFS20; CHMMVW20; Set20].

Let us start by designing the PIOP, which proves these mod p equations. We call this a *mod-PIOP*.

3.3 Mod-PIOP for Integer Moduli-R1CS over $\mathbb{Z}_p$

Fortunately, many existing PIOP constructions for R1CS work over any prime field $\mathbb{Z}_p$. Since the multilinear sumcheck protocol [LFKN90] can be run over an arbitrary field, a sumcheck-based PIOP can be used to prove the integer moduli-R1CS relation over $\mathbb{Z}_p$ out of the box. In our full protocol in Section 6, we adapt Spartan [Set20] to our setting, but we can similarly construct protocols based on HyperPlonk [CBBZ23] or other PIOPs like GKR [GKR08].

Spartan-Style Mod-PIOP construction §6. At the start of the protocol, the prover sends oracles of the witness integer polynomials w, quos (or commitments over $\mathbb{F}$ after compiling to a SNARK), and the verifier samples the random large λ-bit fingerprinting prime p. Our goal is now to prove Equation (2):

$$A\boldsymbol{z} \circ B\boldsymbol{z} \equiv_p C\boldsymbol{z} + \mathsf{mods} \circ \mathsf{quos} \tag{2}$$

If we rephrase this equation as a polynomial equation, we get that we want to prove that for each constraint $\boldsymbol{i} \in \{0,1\}^n$,

$$\left(\sum_{\boldsymbol{j}\in\{0,1\}^n} A(\boldsymbol{i},\boldsymbol{j})z(\boldsymbol{j})\right)\left(\sum_{\boldsymbol{j}\in\{0,1\}^n} B(\boldsymbol{i},\boldsymbol{j})z(\boldsymbol{j})\right) \equiv_p \sum_{\boldsymbol{j}\in\{0,1\}^n} C(\boldsymbol{i},\boldsymbol{j})z(\boldsymbol{j}) + \mathsf{mods}(\boldsymbol{i})\mathsf{quos}(\boldsymbol{i}) \tag{3}$$

The prover and verifier engage in a Spartan-style sumcheck-based protocol to show this. The primary difference with Spartan is that the shape of the sumcheck changes from standard R1CS to the moduli-R1CS shape that includes mods and quos. At the end of the sumchecks, the protocol reduces to checking evaluations of the polynomials. Here, we run into two issues. One, we need to commit to integer polynomials, potentially with large norm. Secondly, and perhaps the more challenging issue, is that the polynomial evaluations must be proven over $\mathbb{Z}_p$ for a random prime p, and not over the field $\mathbb{F}$ which the polynomial was committed in.

A standard PCS is *insufficient* to prove these evaluation queries, as the evaluation claim $v = f(\boldsymbol{r}) \in \mathbb{F}$ is defined over the same field that the polynomial was committed to. What we require is an integer PCS that can open the evaluation of a committed polynomial f at a point $\boldsymbol{r}$ over a different field $\mathbb{Z}_p$, $(v \equiv_p f(\boldsymbol{r}))$. We call such a PCS an *integer mod-PCS*.

3.4 Goals of Integer Mod-PCS

Limber, our integer mod-PCS construction, targets two goals: PCS-agnosticity — the ability to compile any standard PCS over a field into an integer mod-PCS — and low compilation overhead. This combination is powerful: it lets us reuse existing and future PCS constructions, inherit the security assumptions and performance of the underlying PCS, and avoid hand-designing a specialized integer mod-PCS for each use case.

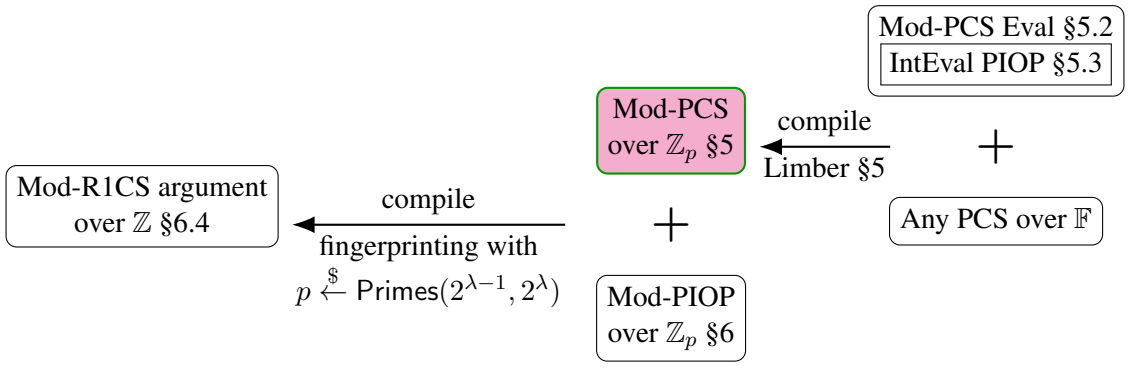


Figure 1: Construction of integer Mod-R1CS argument. $\mathbb{F}$ is any field with characteristic $q = \Omega(\lambda^2 n^2)$, and p is a λ-bit random prime.

Figure 1 illustrates how Limber fits into the overall construction and how it is compiled from a standard PCS. At its core is the IntEval protocol, which proves integer evaluations: Limber uses IntEval to turn any standard PCS into an integer mod-PCS. Combining Limber with a mod-PIOP for $\mathsf{R}_{\mathsf{imodR1CS}}(p)$ and the fingerprinting reduction (Protocol 6.1) then yields a SNARK for integer computation.

Setting up the problem. Consider the multilinear integer polynomial $f \in \mathbb{Z}^{\leq 1}[n]$, whose hypercube evaluations (evaluations over $\{0,1\}^n$) that define it are bounded by T_f, and an underlying PCS that commits over some field $\mathbb{F}$ with characteristic q.

Let us first make a restrictive assumption that f has hypercube evaluations that are small or in other words $\mathsf{T}_f < q$. Then, the prover can just commit to the integer polynomial f using the same commit algorithm as the underlying PCS. Let us therefore focus on designing the evaluation protocol, and leave the integer commitment for larger integers for later (Section 3.5.5).

3.5 Building the IntEval Protocol

The Integer Evaluation Protocol (Section 5.3) is the main contribution in this work. The evaluation protocol needs to show that for an integer polynomial $f(\boldsymbol{X})$ and integer opening point $\boldsymbol{r}$, $f(\boldsymbol{r}) = y$ (see formal definition in Definition 5.4). We start from a basic construction and discuss optimizations step by step.

3.5.1 Strawman evaluation protocol - very large q

Let's say q, the characteristic of the PCS field $\mathbb{F}$, is large enough such that the evaluation of the polynomial at the point $\boldsymbol{r}$ does not overflow q; that is, $|f(\boldsymbol{r})| < q$. Then, since there is no overflow, the mod q evaluation we get by default from the underlying PCS evaluation protocol *is* the integer evaluation $f(\boldsymbol{r})$. The verifier can just take the integer evaluation $f(\boldsymbol{r}) \bmod p$ to get the evaluation over $\mathbb{Z}_p$. The verifier also needs to enforce that f is sufficiently bounded (all hypercube evaluations at most T_f in magnitude), which can be achieved through a range check protocol. There exist efficient range check and (relatedly) lookup protocols which don't require additional commitment work during proving [ZBKMNS22; EFG22; STW24; PH23].

However, this approach requires q to be prohibitively large. With $|\boldsymbol{r}| < p$, we only have the upper bound $|f(\boldsymbol{r})| < 2^n p^n \mathsf{T}_f$,[d] so we need q to be *larger* than $2^n p^n \mathsf{T}_f$. This means q needs more than λn bits, as p^n is λn bits. More concretely, consider a running example of $n = 20$ and $\log p \approx \lambda = 128$, in which case q is required to be at least 2560 bits.

This requirement is undesirable since a large field characteristic increases the prover time, verifier time and proof size, and restricts the choice of the PCS. Therefore, a natural goal is to reduce the requirement on q such that we can compile from a much wider range of PCS constructions.

Note that we work with multilinear PIOPs because the bit-length of a multilinear polynomial evaluation grows only logarithmically in the length of the polynomial, whereas a univariate polynomial evaluation grows linearly, which would blow up what q needs to be.

3.5.2 First technique: Chinese Remainder Theorem

To test whether $y = f(\boldsymbol{r})$, the verifier can choose distinct prime moduli $p_1, p_2, \ldots, p_s \in \mathsf{Primes}(P/2, P)$ and test relative to all of these moduli. The Chinese Remainder Theorem (CRT, Theorem 4.3) allows us to reduce a bounded integer opening to several openings relative to these smaller moduli as long as their product is larger than the integer we want to evaluate.

Reducing to smaller moduli is powerful because the claims over these smaller moduli are much smaller, as everything can be taken mod p_i. The prover is now proving evaluations at $\boldsymbol{r} \bmod p_i$ for every $i \in [s]$:

$$f(\boldsymbol{r} \bmod p_i) \equiv_{p_i} y$$

Now, instead of needing $q > f(\boldsymbol{r})$, we only need $q > f(\boldsymbol{r} \bmod p_i)$ for every $i \in [s]$. Since $p_i < P$, our bound becomes

$$q > 2^n P^n \mathsf{T}_f$$

[d] $f(r)$ can be written as $\sum_{b \in \{0,1\}^n} f_b \cdot \mathsf{eq}(b, r)$. Each term in the sum is a hypercube evaluation f_b bounded by T_f and a Lagrange polynomial $\mathsf{eq}(b, r)$ which is bounded by $|r|^n$. Overall, there are 2^n terms in the sum.

Fortunately, we can choose P to be significantly smaller than p, so this is a much more reasonable requirement on q. However, if P and therefore the prime moduli are small, then the number of moduli s needs to be large. By CRT, we need $P^s \geq \prod_{i=1}^{s} p_i > y \approx 2^{\lambda n}$. This implies that $s \approx \frac{n \cdot \lambda}{\log(P)}$. More concretely, consider an example where $n = 20$ and $P = 2^{32}$; then q is required to be more than $n \log P = 640$ bits large, and $s \approx 80$, adding approximately an extra $80 \cdot N$ prover $\mathbb{F}$ operations.

3.5.3 Second technique: choose CRT moduli randomly

We can relax the requirement on the number of CRT moduli s by choosing the $p_1, \ldots, p_s$ randomly instead of deterministically. Say the prover was dishonest and sent a claimed integer y' that is not equal to the true evaluation $y = f(\boldsymbol{r})$, and let $S = \{p'_1, p'_2, \ldots\}$ be the set of all primes in $\mathsf{Primes}(P/2, P)$ that divide $y' - y$. The dishonest prover passes verification with y' if and only if every random prime chosen is in S. Note that the size of S is at most $\log_{P/2} |y' - y| \approx n\lambda / \log P$, since each prime in S is a distinct divisor of $y' - y$ of size at least $P/2$. By sampling the primes from the set $\mathsf{Primes}(P/2, P)$ which is significantly larger than $|S|$, the probability that a sampled p_i is in S is roughly proportional to $\frac{n\lambda / \log P}{P / \log P} = \frac{n\lambda}{P} \approx \frac{1}{P}$. Given that each p_i is sampled independently, we can achieve a negligible soundness error by sampling s primes such that $P^s \approx 2^{\lambda}$. Previously, using fixed primes required $P^s > y \approx 2^{n\lambda}$. Randomly sampling the p_i's thus reduces the number of CRT primes s needed, and the associated prover overhead by a factor of almost n.

More precisely, we can write the probability that a malicious prover passes verification as:

$$\Pr[\forall i \in [s], y' \equiv_{p_i} y] = \Big(\frac{|S|}{|\mathsf{Primes}(P/2, P)|}\Big)^s$$

Given that the product of all primes in S should not exceed the difference between y' and y, we get the bound $|S| \leq \frac{8\lambda n}{\log P}$. The lower bound $|\mathsf{Primes}(P/2, P)| \geq P/4 \log P$ holds assuming $P \geq 2^5$ and applying fundamental bounds on the prime counting function (see Corollary B.1 for details). Thus, we can bound the above probability by at most $\left(\frac{32\lambda n}{P}\right)^s$. In order to achieve at most negligible probability $2^{-\lambda}$, we set $P \gg 32\lambda n$ and $s \approx \frac{\lambda}{\log P}$ for sufficiently large P, which is independent of n. In the concrete example where $n = 20, \lambda = 128$ and $P = 2^{32}$, $s = 9$ suffices for negligible soundness error, compared to the previous $s = 80$ with deterministic CRT moduli. We provide a formal analysis in the proof of Theorem 5.5.

3.5.4 Third technique: partial evaluation and polynomial decomposition

Even given our optimizations so far, the field characteristic of the underlying PCS is still required to be large ($q > 2^n P^n \mathsf{T}_f$) as it depends exponentially on the number of variables n. This exponential dependence is the result of fully evaluating f over the integers and requiring q to be sufficiently large such that evaluations mod q do not overflow and are equivalent to the evaluations over the integers. To resolve this, we instead break the problem of checking full evaluations of f down to instead only checking partial evaluations of a few variables of $\boldsymbol{r} \bmod p_i$. We then recursively apply this technique until we have fully evaluated the polynomial. This limits the norm blowup of the evaluations and allows us to make the requirement on q to be independent of n.

More concretely, let $k < n$ be the number of variables we partially evaluate f on and define $\boldsymbol{r}^{(i)} := \boldsymbol{r} \bmod p_i$ for every $i \in [s]$. If we partially evaluate f on k variables, then the resulting polynomial has $n - k$ variables:

$$f(X_1, \ldots, X_{n-k}, r^{(i)}_{n-k+1}, \ldots, r^{(i)}_n) =: g^{(i)}_1(X_1, \ldots, X_{n-k})$$

and its evaluations over the hypercube are upper-bounded by $2^k P^k \mathsf{T}_f$.[e] Therefore, we only need $q > 2^k P^k \mathsf{T}_f$ in order to do this partial evaluation without overflow.

However, the norm of each partially evaluated polynomial $\hat{g}_1^{(i)}$ is large as a result of the partial evaluation. In order to fully evaluate the polynomial, a reduction step is needed in order to be able to continue evaluation on the rest of the variables.

Modulo Decomposition. To reduce the norm, the prover can decompose the partially evaluated polynomial $g^{(i)}$ into a remainder polynomial $a^{(i)}$ and a quotient polynomial $b^{(i)}$ with regard to p_i:

$$g^{(i)}(\boldsymbol{X}) = a^{(i)}(\boldsymbol{X}) + p_i \cdot b^{(i)}(\boldsymbol{X})$$

where $||a^{(i)}|| < P$. Since we are only interested in the evaluation modulo p_i, the remainder polynomial $a^{(i)}$ can serve as the polynomial for the next partial evaluation. Therefore, we can recursively perform partial evaluations and polynomial decompositions until we have fully evaluated the polynomial.

Let $\mathsf{t} := \lceil (n-k)/k \rceil$ be the number of iterations. If we unroll the partial evaluation and decomposition for t iterations, we get the following steps for $j \in [\mathsf{t}]$, where $a_0^{(i)} = f$ for every $i \in [s]$:

$$\begin{aligned}
&a_{j-1}^{(i)}(X_1, \ldots, X_{n-jk}, r_{n-jk+1}^{(i)}, \ldots, r_{n-(j-1)k}^{(i)}) \\
&=: g_j^{(i)}(X_1, \ldots, X_{n-jk}) \\
&=: a_j^{(i)}(X_1, \ldots, X_{n-jk}) + p_i \cdot b_j^{(i)}(X_1, \ldots, X_{n-jk})
\end{aligned}$$

See Figure 2 for the overall structure and intuition. In each iteration $j \in [\mathsf{t}]$, the prover computes the partial evaluated polynomial $g_j^{(i)}$ with norm at most $\frac{q-P}{2}$[f] and its decomposition, and sends the oracles $[[\hat{a}_j^{(i)}]], [[\hat{b}_j^{(i)}]]$. The verifier checks the polynomial identity $a_{j-1}^{(i)}(X_1, \ldots, X_{n-jk}, r_{n-jk+1}^{(i)}, \ldots, r_{n-(j-1)k}^{(i)}) = a_j^{(i)}(X_1, \ldots, X_{n-jk}) + p_i \cdot b_j^{(i)}(X_1, \ldots, X_{n-jk})$ by sampling a random point. Importantly, it suffices to do this check over the native field $\mathbb{F}$. At the end, the verifier queries the final oracle to get $y_i \leftarrow a_{\mathsf{t}}^{(i)}(r_1^{(i)}, \ldots, r_k^{(i)})$ and checks whether $y_i \equiv_{p_i} y$ for every $i \in [s]$.

The verifier needs to additionally range check each $a_j^{(i)}, b_j^{(i)}$ to ensure their norms are correctly upper-bounded by $P, \lfloor \frac{q}{P} \rfloor$, respectively. For the final opening to not overflow, we need $q > 2^k P^{k+1}$, since $a_{\mathsf{t}}^{(i)}(\boldsymbol{X})$ has a norm of P over the hypercube and has at most k variables. Since partially evaluated polynomials $g_j^{(i)}$ are required to be upper-bounded by $\frac{q-P}{2}$, it gives us the condition that $\frac{q-P}{2} > 2^k P^k \max(\mathsf{T}_f, P)$. Combining these conditions, we require that $q > 2^{k+1} P^k \max(\mathsf{T}_f, P) + P$, or, more loosely,

$$q > 2^{k+2} P^{k+1} \mathsf{T}_f \tag{4}$$

Crucially, the exponent in this bound depends on k, not n: the exponential dependence is now only on k, which we are free to set much smaller. In the extreme case, we can set $k = 1$ and get that $q = \Omega(P^2)$. Since we have set $P = \Omega(\lambda n)$, this gives $q = \Omega(\lambda^2 n^2)$, i.e. polynomial in the security parameter.

$o(1)$ multiplicative commitment overhead. With partial evaluations and polynomial decompositions, the prover needs to additionally send $2 \cdot s \cdot \mathsf{t}$ oracles of the remainder and quotient polynomials. However, these additional oracles are exponentially smaller than the original witness, since partial evaluation reduces the

[e] Since $g_1^{(i)}$ was partially evaluated at k variables, each evaluation can be written as a sum of 2^k terms. Each term is bounded by a product of k variables and a hypercube evaluation that is at most T_f.

[f] This norm bound is needed for completeness. See details in the completeness proof of Theorem 5.5.

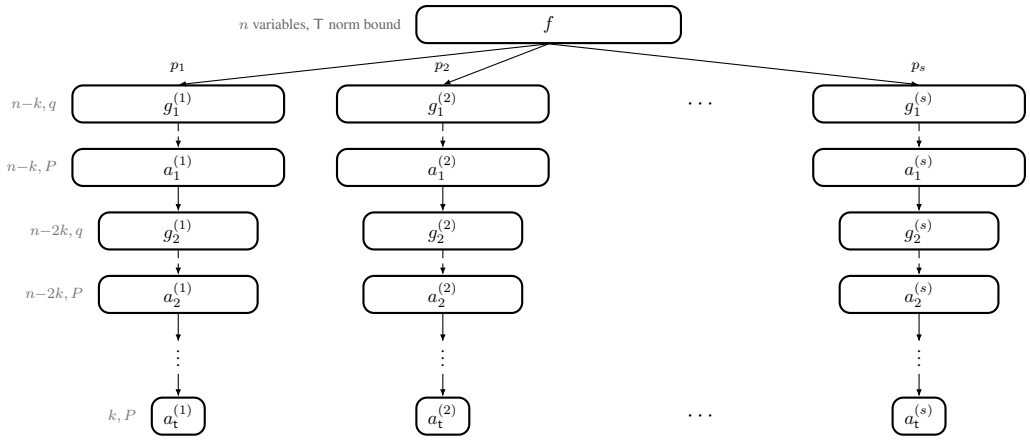


Figure 2: Partial evaluations with polynomial decompositions. Each node represents a polynomial, solid arrows represent partial evaluation, and dashed arrows represent modulo decomposition. The grey label to the left of each node gives that polynomial's number of variables and norm bound. The quotient polynomials $b_j^{(i)}$ from modulo decompositions are omitted in the figure. The committed polynomial f has n variables. For each small prime $p_i \in \mathsf{Primes}(P/2, P)$, the prover runs an independent protocol with $\mathsf{t} = \lceil (n-k)/k \rceil$ iterations. In iteration j for prime i, the prover fixes k variables of the carried polynomial at $\boldsymbol{r}^{(i)} = \boldsymbol{r} \bmod p_i$ to form the partial evaluation $g_j^{(i)}$, then decomposes it with respect to p_i as $g_j^{(i)} = a_j^{(i)} + p_i\, b_j^{(i)}$. Both oracles $[[a_j^{(i)}]], [[b_j^{(i)}]]$ are committed and range-checked, and the small-norm remainder $a_j^{(i)}$ ($|\cdot| < P$) is carried into the next iteration. After t iterations, the verifier queries the final oracles to get $y^{(i)} = a_{\mathsf{t}}^{(i)}(\boldsymbol{r}_1^{(i)}, \ldots, \boldsymbol{r}_k^{(i)})$, and checks whether $y^{(i)} = y \pmod{p_i}$ for every i.

length of the polynomial by 2^k. The total size of these oracles is approximately $2^{n-k} s \log q$ bits. Compared to the original witness polynomial size, $2^n \cdot \log \mathsf{T}_f$ bits, the extra commitment overhead is $\frac{2^{n-k} s \log q}{2^n \log \mathsf{T}_f} = \frac{s \log q}{2^k \log \mathsf{T}_f}$. By setting k appropriately, e.g. $k = \Omega(\log(\lambda))$, we achieve $o(1)$ multiplicative commitment overhead, which means the cost of committing to additional polynomials in the compiled argument is sublinear in the cost of committing to the witness. With larger settings of k (> 10), we can essentially pay for only a tiny extra fraction of the original commitment cost. As discussed in the previous paragraph, note that a larger setting of k forces a larger field characteristic q. We formally derive the parameters in Section 5.4 and give concrete parameter tradeoffs in Table 3.

3.5.5 Fourth technique: limb-splitting

So far, the bound on the field characteristic, $q > 2^{k+2} P^{k+1} \mathsf{T}_f$, has remained linear in the norm bound T_f, which means we are making a restrictive assumption that the polynomial we want to commit to has small norm, which in practice, is often not the case, for example for cryptographic operations involving large fields like RSA or elliptic curves.

To handle polynomials with large evaluations over the hypercube, we can try limb-splitting the values into T-sized limbs, where $\mathsf{T} < q$. For a value $v < \mathsf{T}_f$, we can obtain the limb-split v_ℓ by writing v in base T. To be more precise, let the number of limbs needed for an evaluation be $L := \lceil \log_{\mathsf{T}} \mathsf{T}_f \rceil$ and let $l = \log L$. Define the gadget vector as $\boldsymbol{\ell} := [1, \mathsf{T}, \ldots, \mathsf{T}^{L-1}]$. Let $\boldsymbol{f} \in \mathbb{Z}^N$ be the evaluations of f over the Boolean hypercube. For every $i \in [N]$, we split the T_f-bounded entry $\boldsymbol{f}[i]$ into L limb entries $\boldsymbol{f}_\ell^{(i)}$, such that $\langle \boldsymbol{\ell}, \boldsymbol{f}_\ell^{(i)} \rangle = \boldsymbol{f}[i]$. The final limb-split polynomial $f_\ell \in \mathbb{Z}^{\leq 1}[n+l]$ has NL evaluations over the hypercube

$\boldsymbol{f}_\ell := \boldsymbol{f}_\ell^{(1)} || \ldots || \boldsymbol{f}_\ell^{(N)}$ and norm bound T. Correct limb-splitting should satisfy the following

$$\sum_{\boldsymbol{k} \in \{0,1\}^l} \ell(\boldsymbol{k}) \cdot f_\ell(\boldsymbol{x}, \boldsymbol{k}) = f(\boldsymbol{x})$$

Now, to commit to an integer polynomial f, we can instead commit to the limb-split f_ℓ instead.

To prove an evaluation claim $f(\boldsymbol{r}) = v \bmod p$, we can first prove the sumcheck equation $f(\boldsymbol{r}) = \sum_{\boldsymbol{k} \in \{0,1\}^l} \ell(\boldsymbol{k}) \cdot f_\ell(\boldsymbol{r}, \boldsymbol{k}) = v \bmod p$, which reduces to evaluation claims $\ell(\boldsymbol{r'}) \bmod p$ and $f_\ell(\boldsymbol{r}, \boldsymbol{r'}) \bmod p$. Note that this sumcheck only has l rounds as opposed to $n + l$, so it is very lightweight. Since $\ell(\boldsymbol{X})$ is highly structured, the verifier can directly check the first evaluation claim on its own in $O(l)$ $\mathbb{Z}_p$ operations. For $f_\ell(\boldsymbol{X})$, which is norm-bounded by T, we apply the IntEval protocol. Moreover, the requirement on the field characteristic now becomes

$$q \geq 2^{k+1} P^k \max(\mathsf{T}, P) + P$$

where T has replaced T_f. This final lower bound on the field characteristic q is only exponentially dependent on k, the number of partially evaluated variables per iteration, and linearly dependent on the limit P for the random primes in IntEval and the norm bound T of the limb-split polynomial $\hat{f}_\ell$. Importantly, it is *not* dependent on the random λ-bit prime p, the total number of variables n, nor the norm bound of the original polynomial $\hat{f}$.

Remark 3.4. While the oracle to the limb-split polynomial $[[\hat{f}_\ell]]$ has increased length by a factor of L, the total size of the oracle remains the same at $2^n \log \mathsf{T}_f$ bits, since limb-splitting preserves the information of the original hypercube evaluations but represents them in a different form.

3.6 All Together: Limber, an Integer Mod-PCS Compiler §5

Given a standard PCS = $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Eval})$ over some field $\mathbb{F}$ with characteristic q, we compile it into an integer mod-PCS as follows, using the techniques from Section 3.5.

- $\mathsf{modPCS.Setup}(1^\lambda, \mathsf{T}, \mathsf{T}_f)$: T is the base norm bound for limb-splitting, and T_f is the norm bound of the polynomial that will be committed. Run the $(\mathsf{vk'}, \mathsf{ck'}) \leftarrow \mathsf{PCS.Setup}(1^\lambda)$, and output $\mathsf{vk} := (\mathsf{vk'}, \mathsf{T}, \mathsf{T}_f)$, $\mathsf{ck} := (\mathsf{ck'}, \mathsf{T}, \mathsf{T}_f)$.

- $\mathsf{modPCS.Commit}(\mathsf{ck}, f \in \mathbb{Z}^{\leq 1}[n])$: The prover limb-splits f into f_ℓ according to the base bound specified by ck, where $L = \lceil \log_{\mathsf{T}} \mathsf{T}_f \rceil$. The prover casts f_ℓ to $\mathbb{F}$ and gets $\hat{f}_\ell \leftarrow \texttt{toField}(f_\ell)$. The prover sends the verifier the commitment $\mathsf{com}_f \leftarrow \mathsf{PCS.Commit}(\mathsf{ck'}, \hat{f}_\ell)$. Then, the prover and verifier run a range check argument to prove the base norm bound holds, i.e. $\|\hat{f}_\ell\| < \mathsf{T}$. If the range check passes, the verifier outputs com_f; otherwise, the verifier outputs $\perp$.

- $\mathsf{modPCS.Eval}((\mathsf{ck}, \mathsf{vk}); \mathsf{com}_f, p, \boldsymbol{r}_f \in \mathbb{Z}_p^n, v \in \mathbb{Z}_p; f)$: This is a public-coin interactive protocol between a prover who has the integer polynomial f and a verifier who has the polynomial commitment com_f. The prover first limb-splits f into f_ℓ according to the base bound specified by ck, and then the prover and the verifier run a sumcheck argument to check the correctness of the limb-split polynomial f_ℓ. At the end of the sumcheck, the verifier needs to check whether the prover-sent evaluation $y' \in \mathbb{Z}_p$ is equal to $f_\ell(\boldsymbol{r}_f, \boldsymbol{r}_\ell) \bmod p$, where $\boldsymbol{r}_\ell$ are the random sumcheck challenges.
  To prove this, the prover sends the integer evaluation $y \in \mathbb{Z}$. The verifier checks whether $y' = y \bmod p$, and they run the IntEval argument to prove $y = f(\boldsymbol{r}_f, \boldsymbol{r}_\ell)$.

We show in Section 5 that the compiled modPCS is correct with modPCS.Eval having low soundness error, if the parameters are set appropriately and the underlying PCS and range checks are correct/complete with low soundness error.

4 Preliminaries

Assumptions:

- A, B, C matrices have norm less than T. Although this seems very restricting to the computation we support, it still is fully general as we can always build large circuit constants from their T-sized limbs in the witness, which is not restricted to T. Alternatively, we can also treat the matrices as integer polynomials and have the prover evaluate them as such.
- $q = \Omega(\lambda^2 m^2)$: our protocol works for small fields, but not necessarily tiny fields.
- mods can be efficiently evaluated by the verifier. Most computations will only use a small set of distinct moduli, so the mods polynomial can be structured in a way so it is easy to compute. However, if this is not true, we can have the protocol also treat mods as an integer polynomial and have the prover evaluate it as such like the other polynomials.

The following definitions and theorems are used throughout this work.

Definition 4.1 (Multilinear Extension)**.** *Let $N = 2^n$. For any field $\mathbb{F}$ and vector $\boldsymbol{u} \in \mathbb{F}^N$, we denote by $u \in \mathbb{F}^{\leq 1}[n]$ the multilinear extension (MLE) of $\boldsymbol{u}$ defined as follows,*

$$u(X_1, \ldots, X_n) := \sum_{\boldsymbol{x} \in \{0,1\}^n} \boldsymbol{u}_{\sum_{i=1}^n \boldsymbol{x}_i 2^{i-1}} \cdot \mathsf{eq}(\boldsymbol{x}; X_1, \ldots, X_n)$$

Lemma 4.2 (Schwartz-Zippel Lemma [Sch80; Zip79])**.** *Let S be any finite subset of $\mathbb{F}$, and let $r_1, \ldots, r_n$ be n field elements selected independently and uniformly from set S. Then for any polynomial f in $\mathbb{F}$ with n variables and total degree d,*

$$\Pr[f(r_1, \ldots, r_n) = 0] \leq \frac{d}{|S|}$$

Theorem 4.3 (Chinese Remainder Theorem)**.** *Let $m_1, m_2, \ldots, m_s$ be pairwise coprime positive integers greater than 1. For any given sequence of integers $a_1, a_2, \ldots, a_s$ such that $0 \leq a_i < m_i$ for every $i \in [s]$, there exists a unique integer $0 \leq x < \prod_{i=1}^s m_i$ such that*

$$x \equiv a_i \bmod m_i \quad \forall i \in [s].$$

Remark 4.4. The unique integer described in the Chinese Remainder Theorem (Theorem 4.3) can be found in time $O(\log^2 M)$, where $M = \prod_{i=1}^s m_i$. We refer to [Sho09] for the details of this algorithm. While there exist other approaches that are more efficient, this technique suffices for CRT usage in this work.

4.1 Relation

Definition 4.5 (Indexed Oracle Relation)**.** *An indexed oracle relation R is the set of all tuples $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w})$ where $\mathbb{i}$ is the index, $\mathbb{x}$ is the explicit instance, $\mathbb{o}$ is the oracle instance, and $\mathbb{w}$ is the witness.*

Definition 4.6 (Cartesian product of relations)**.** *Given relations* $\mathsf{R}_1 := (\mathbb{i}_1; \mathbb{x}_1; \mathbb{o}_1; \mathbb{w}_1)$, $\mathsf{R}_2 := (\mathbb{i}_2; \mathbb{x}_2; \mathbb{o}_2; \mathbb{w}_2)$, *their Cartesian product is the following,*

$$\mathsf{R}_1 \times \mathsf{R}_2 := \left\{ \Big((\mathbb{i}_1, \mathbb{i}_2); (\mathbb{x}_1, \mathbb{x}_2); (\mathbb{o}_1, \mathbb{o}_2); (\mathbb{w}_1, \mathbb{w}_2) \Big) \;\middle|\; \begin{array}{l} (\mathbb{i}_1; \mathbb{x}_1; \mathbb{o}_1; \mathbb{w}_1) \in \mathsf{R}_1 \\ (\mathbb{i}_2; \mathbb{x}_2; \mathbb{o}_2; \mathbb{w}_2) \in \mathsf{R}_2 \end{array} \right\}$$

R^k *denotes the* k *Cartesian product of the same relation* R.

4.2 Limb-Splitting

Limb-splitting is a technique used to split elements into T-bounded values. Here, we describe an algorithm that limb-splits an integer polynomial f and produces an integer polynomial of its limbs relative to base T.

Protocol 4.1: $\mathsf{LimbSplit}(f, \mathsf{T}, \mathsf{T}_f)$

1. Let $\boldsymbol{f}$ be the evaluations of f over the Boolean hypercube. Define $N := |\boldsymbol{f}|$, $L := \lceil \log_{\mathsf{T}} \mathsf{T}_f \rceil$ and gadget vector $\boldsymbol{\ell} := [1, \mathsf{T}, \ldots, \mathsf{T}^{L-1}]$.
2. For every $i \in [N]$, decompose $\boldsymbol{f}[i]$ into an L-sized vector $\boldsymbol{f}_\ell^{(i)}$ by writing it in base T. We have that $|\boldsymbol{f}_\ell^{(i)}| < \mathsf{T}$ and $\langle \boldsymbol{\ell}, \boldsymbol{f}_\ell^{(i)} \rangle = \boldsymbol{f}[i]$.
3. Define the (NL)-sized limb-split vector $\boldsymbol{f}_\ell := \boldsymbol{f}_\ell^{(1)} || \ldots || \boldsymbol{f}_\ell^{(N)}$, and output the limb-split polynomial f_ℓ whose hypercube evaluations are $\boldsymbol{f}_\ell$.

The *gadget polynomial* ℓ is the multilinear extension of the gadget vector $\boldsymbol{\ell} = [1, \mathsf{T}, \ldots, \mathsf{T}^{L-1}]$ over $l = \log L$ variables. Note that evaluating the gadget polynomial only takes $O(\log L)$ multiplications.

Lemma 4.7. *For any multilinear integer polynomial* f *with* n *variables and norm bound* T_f *and any limb bound* T*, it holds that* $f(\boldsymbol{X}) = \sum_{\boldsymbol{k} \in \{0,1\}^l} \ell(\boldsymbol{k}) f_\ell(\boldsymbol{X}, \boldsymbol{k})$*, where* $f_\ell \leftarrow \mathsf{LimbSplit}(f, \mathsf{T}, \mathsf{T}_f)$ *and* ℓ *is the gadget polynomial.*

Proof. Both sides are multilinear in $\boldsymbol{X}$, so it suffices to verify the identity on the Boolean hypercube. Fix $\boldsymbol{i} \in \{0,1\}^n$ and let i be the index representation. By construction of $\mathsf{LimbSplit}$, $f_\ell(\boldsymbol{i}, \boldsymbol{k}) = \boldsymbol{f}_\ell^{(i)}[\boldsymbol{k}]$ and $\ell(\boldsymbol{k}) = \boldsymbol{\ell}[\boldsymbol{k}]$ for every $\boldsymbol{k} \in \{0,1\}^l$. Hence

$$\sum_{\boldsymbol{k} \in \{0,1\}^l} \ell(\boldsymbol{k})\, f_\ell(\boldsymbol{i}, \boldsymbol{k}) = \langle \boldsymbol{\ell}, \boldsymbol{f}_\ell^{(i)} \rangle = \boldsymbol{f}[i] = f(\boldsymbol{i}),$$

which holds by definition. Since both sides are multilinear and agree on $\{0,1\}^n$, they are equal as polynomials. □

4.3 Polynomial Commitment Scheme (PCS)

Definition 4.8 (Polynomial Commitment Scheme)**.** *A Polynomial Commitment Scheme* PCS *over a field* $\mathbb{F}$ *is a tuple* $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Eval})$ *in which* Setup *and* Eval *are PPT algorithms and* Commit *is a deterministic algorithm:*

- $\mathsf{Setup}(1^\lambda, \mathsf{aux}) \to (\mathsf{vk}, \mathsf{ck})$ *generates the verifier key* vk *and committer key* ck*. These keys include* $\mathbb{F}$ *the field,* n *the number of variables,* d *the maximum degree in each variable, and other necessary information contained in* aux*.*

- $\mathsf{Commit}(\mathsf{ck}, f) \to \mathsf{com}$ *is a deterministic algorithm that takes a multilinear polynomial* $f \in \mathbb{F}^{\leq 1}[n]$ *and outputs a public commitment* com*.*

- $\mathsf{Eval}((\mathsf{ck}, \mathsf{vk}); \mathsf{com}, \boldsymbol{r}, v; f) \to \mathsf{accept}/\mathsf{reject}$ *is an interactive public-coin protocol between two PPT algorithms,* $\mathsf{P}(\mathsf{ck}, f, \boldsymbol{r}, v)$ *and* $\mathsf{V}(\mathsf{vk}, \mathsf{com}, \boldsymbol{r}, v)$*, that proves membership in the evaluation relation* $\mathcal{R}_{\mathsf{eval}}$ *(Definition 4.9), i.e. that the committed polynomial* f *satisfies* $f(\boldsymbol{r}) = v$*.*

The Eval algorithm proves membership in the following polynomial evaluation relation.

Definition 4.9 (Polynomial evaluation relation)**.**

$$\mathcal{R}_{\mathsf{eval}}(\mathsf{ck}) := \left\{ \left((\boldsymbol{r}, v, \mathsf{com});\ f\right) \;\middle|\; \begin{array}{l} f \in \mathbb{F}^{\leq 1}[n],\ \boldsymbol{r} \in \mathbb{F}^n,\ v \in \mathbb{F}, \\ \mathsf{com} = \mathsf{Commit}(\mathsf{ck}, f),\ \textit{and}\ f(\boldsymbol{r}) = v \end{array} \right\}.$$

A PCS has the following properties:

- A PCS is *correct* if an honest committer can convince the verifier of any evaluation. Specifically, if the prover is honest, then for all multilinear polynomials $f \in \mathbb{F}^{\leq 1}[n]$ and all points $\boldsymbol{r} \in \mathbb{F}^n$,

$$\Pr\left[\mathsf{Eval}((\mathsf{vk}, \mathsf{ck}); \mathsf{com}, \boldsymbol{r}, v; f) = \mathsf{accept} \;\middle|\; \begin{array}{l} (\mathsf{vk}, \mathsf{ck}) \leftarrow \mathsf{Setup}(1^\lambda) \\ \mathsf{com} := \mathsf{Commit}(\mathsf{ck}, f) \\ v \leftarrow f(\boldsymbol{r}) \end{array} \right] = 1$$

- Eval is an *argument of knowledge* with knowledge error ϵ_{PCS}, following [CHMMVW20]: for every PPT adversary $\mathcal{A}$ there exists a non-uniform PPT extractor Ext for any $\mathsf{aux} \in \{0,1\}^{poly(\lambda)}$such that the following probability is at most ϵ_{PCS}:

$$\Pr\left[\begin{array}{l} \exists j \in [m] :\ ((\boldsymbol{r}_j, v_j, \mathsf{com}); f) \notin \mathcal{R}_{\mathsf{eval}} \\ \wedge \bigwedge_{j \in [m]} \mathsf{V}(\mathsf{vk}, \mathsf{com}, \boldsymbol{r}_j, v_j; \pi_j) = \mathsf{accept} \end{array} \;\middle|\; \begin{array}{l} (\mathsf{vk}, \mathsf{ck}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (\mathsf{com}, (\boldsymbol{r}_j, v_j, \pi_j)_{j \in [m]}) \leftarrow \mathcal{A}(\mathsf{vk}, \mathsf{ck}, \mathsf{aux}) \\ f \leftarrow \mathsf{Ext}(\mathsf{vk}, \mathsf{ck}, \mathsf{aux}) \end{array} \right]$$

  Here π_j is the transcript of the j-th (public-coin, interactive) Eval execution, $\mathsf{V}(\mathsf{vk}, \mathsf{com}, \boldsymbol{r}_j, v_j; \pi_j) = \mathsf{accept}$ denotes that the Eval verifier accepts it, and Ext has access to $\mathcal{A}$'s random tape.

Remark 4.10. A PCS over $\mathbb{Z}$ is called an integer PCS.

Definition 4.11 (Integer Mod-PCS)**.** *An integer mod-PCS is an integer PCS with the following differences: the auxiliary input to* Setup *contains a norm bound that must be respected by all committed polynomials;* Commit *may be public-coin interactive; and* Eval *additionally takes in a prime* p *that the evaluation is done with respect to.*

- aux *contains a norm bound* T_f*, which is included in both* vk *and* ck *generated by* $\mathsf{Setup}(1^\lambda, \mathsf{aux})$*.*

- *For polynomial* f *that respects the* T_f *bound included in* ck*,* $\mathsf{Commit}((\mathsf{ck}, \mathsf{vk}), f) \to \mathsf{com}/\mathsf{reject}$ *is an interactive public-coin protocol between deterministic* $\mathsf{P}(\mathsf{ck}, f)$ *and PPT* $\mathsf{V}(\mathsf{vk})$*.* P *first generates a public commitment* com*. The prover sends* com *to the verifier; after some interactions,* V *either outputs* com *or* reject*.*

- *For polynomial f that respects the T_f bound included in* ck, $\mathsf{Eval}((\mathsf{ck},\mathsf{vk});\mathsf{com},p,\boldsymbol{r},v;f)\to\mathsf{accept/reject}$ *takes in a prime p, a vector $\boldsymbol{r}\in\mathbb{Z}_p^n$, and a claimed evaluation $v\in\mathbb{Z}_p$. It is an interactive public-coin protocol between two PPT algorithms,* $\mathsf{P}(\mathsf{ck},f,p,\boldsymbol{r},v)$ *and* $\mathsf{V}(\mathsf{vk},\mathsf{com},p,\boldsymbol{r},v)$*, that proves $f(\boldsymbol{r})\equiv_p v$.*

The integer mod-PCS evaluation relation $\mathcal{R}_{\mathsf{modeval}}$ is the modular analogue of the standard polynomial evaluation relation $\mathcal{R}_{\mathsf{eval}}$ (Definition 4.9), stated over the committed integer polynomial.

Definition 4.12 (Integer mod-PCS evaluation relation)**.**

$$\mathcal{R}_{\mathsf{modeval}}(\mathsf{ck}) := \left\{ \big((p,\boldsymbol{r}_f,v,\mathsf{com}_f);\ f\big) \;\middle|\; \begin{array}{l} p\in\mathsf{Primes}(2^{\lambda-1},2^{\lambda}),\ \boldsymbol{r}_f\in\mathbb{Z}_p^n,\ v\in\mathbb{Z}_p,\\ f\in\mathbb{Z}^{\leq 1}[n],\ ||f||<\mathsf{T}_f,\\ \mathsf{com}_f=\mathsf{Commit}(\mathsf{ck},f),\ \textit{and}\ f(\boldsymbol{r}_f)\equiv_p v \end{array} \right\},$$

where T_f is the norm bound contained in ck*.*

The correctness of an integer mod-PCS is identical to that of a standard PCS, except that v does not have to be the full integer evaluation for Eval but only an integer that is congruent to $f(\boldsymbol{r})$ mod p. Similarly, the knowledge soundness of an integer mod-PCS is identical to that of a standard PCS, except that it is relative to $\mathcal{R}_{\mathsf{modeval}}$ instead of $\mathcal{R}_{\mathsf{eval}}$.

4.4 Interactive Proof and Arguments of Knowledge

We adapt the definitions for interactive proofs of knowledge from HyperPlonk [CBBZ23].

Definition 4.13 (Interactive Proof and Argument of Knowledge)**.** *An interactive protocol $\Pi=(\mathsf{Setup},\mathsf{I},\mathsf{P},\mathsf{V})$ is an argument of knowledge for an indexed relation* R *that works as follows,*

- Offline phase. *The indexer* I *is a deterministic algorithm that receives as input an index $\mathbb{i}$ and outputs* $(\mathsf{vp},\mathsf{pp})$.
- Online phase. *The prover* P *and the verifier* V *interact, and the output of the verifier is denoted by the random variable $\langle\mathsf{P}(\mathsf{pp},\mathbb{x},\mathbb{w}),\mathsf{V}(\mathsf{vp},\mathbb{x})\rangle$:*

Π is considered *public-coin* if all of the verifier messages (including the final output) can be computed as a deterministic function given a random public input. We say that Π has (perfect) completeness, (adaptive) soundness error $\epsilon:\mathbb{N}\times\mathbb{N}\mapsto[0,1]$, and knowledge soundness error $\delta:\mathbb{N}\times\mathbb{N}\mapsto[0,1]$ if the following holds.

- **Completeness:** For all $(\mathbb{i};\mathbb{x};\mathbb{w})\in\mathsf{R}$,

$$\Pr\left[\langle\mathsf{P}(\mathsf{pp},\mathbb{x},\mathbb{w}),\mathsf{V}(\mathsf{vp},\mathbb{x})\rangle=1 \;\middle|\; \begin{array}{c}\mathsf{gp}\leftarrow\mathsf{Setup}(1^\lambda)\\ (\mathsf{vp},\mathsf{pp})\leftarrow\mathsf{I}(\mathsf{gp},\mathbb{i})\end{array}\right]=1$$

- **Soundness (adaptive):** Let $\mathsf{L}(\mathsf{R})$ be the language corresponding to the indexed relation R such that $(\mathbb{i},\mathbb{x})\in\mathsf{L}(\mathsf{R})$ if and only if there exists $\mathbb{w}$ such that $(\mathbb{i};\mathbb{x};\mathbb{w})\in\mathsf{R}$. For every pair of probabilistic polynomial time adversarial prover algorithms $(\mathcal{A}_1,\mathcal{A}_2)$,

$$\Pr\left[\langle\mathcal{A}_2(\mathbb{i},\mathbb{x},\mathsf{st}),\mathsf{V}(\mathsf{vp},\mathbb{x})\rangle=1\wedge(\mathbb{i},\mathbb{x})\notin\mathsf{L}(\mathsf{R}) \;\middle|\; \begin{array}{c}\mathsf{gp}\leftarrow\mathsf{Setup}(1^\lambda)\\ (\mathbb{i},\mathbb{x},\mathsf{st})\leftarrow\mathcal{A}_1(\mathsf{gp})\\ (\mathsf{vp},\mathsf{pp})\leftarrow\mathsf{I}(\mathsf{gp},\mathbb{i})\end{array}\right]\leq\epsilon(|\mathbb{i}|,|\mathbb{x}|)$$

  We say a protocol is computationally sound if ϵ is negligible in λ. If $(\mathcal{A}_1,\mathcal{A}_2)$ are unbounded and ϵ is negligible, then the protocol is statistically sound.

- **Knowledge Soundness:** There exists a polynomial $\mathsf{poly}(\cdot)$ and a probabilistic polynomial-time oracle machine Ext called the *extractor* such that given oracle access to any pair of probabilistic polynomial time adversarial prover algorithms $(\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr\left[\begin{array}{l|c} \langle \mathcal{A}_2(\mathbb{i}, \mathbb{x}, \mathsf{st}), \mathsf{V}(\mathsf{vp}, \mathbb{x})\rangle = 1 & \mathsf{gp} \leftarrow \mathsf{Setup}(1^\lambda) \\ \wedge (\mathbb{i}, \mathbb{x}, \mathbb{w}) \notin \mathsf{R} & (\mathbb{i}, \mathbb{x}, \mathsf{st}) \leftarrow \mathcal{A}_1(\mathsf{gp}) \\ & (\mathsf{vp}, \mathsf{pp}) \leftarrow \mathsf{I}(\mathsf{gp}, \mathbb{i}) \\ & \mathbb{w} \leftarrow \mathsf{Ext}^{\mathcal{A}_1, \mathcal{A}_2}(\mathsf{gp}, \mathbb{i}, \mathbb{x}) \end{array}\right] \leq \delta(|\mathbb{i}|, |\mathbb{x}|)$$

  If the knowledge error δ is negligible in λ, then an interactive protocol is "knowledge sound", aka an "argument of knowledge." If the adversary is unbounded, then the argument is called an interactive proof of knowledge.

Remark 4.14 (Non-interactive arguments)**.** Interactive public-coin arguments can be made non-interactive using the Fiat-Shamir transform [FS87]. The Fiat-Shamir transform replaces the verifier challenges with hashes of the transcript up to that point. The works by [AFK21; Wik21] show that this is secure for multi-round special-sound protocols and multi-round oracle proofs. [CCHLRR18] show that the sumcheck protocol and more generally sumcheck-based protocols have round-by-round soundness.

4.5 Polynomial Interactive Oracle Reduction and Polynomial Interactive Oracle Proof

Polynomial Interactive Oracle Reductions (PIORs) are a generalization of Polynomial Interactive Oracle Proofs (PIOPs). Below we give a definition that follows from the definition of Interactive Oracle Reductions (IORs) in [BCFW25]. Our definition is equivalent to the PIOR definition in [SSPP26].

Definition 4.15 (Polynomial Interactive Oracle Reduction)**.** *A public-coin PIOR from a relation* R *to relations* $\mathsf{R}'_1 \times \cdots \times \mathsf{R}'_k$ *is a triple* $\mathsf{PIOR} = (\mathsf{I}, \mathsf{P}, \mathsf{V})$ *that works as follows*

- Offline phase. *The indexer* I *is a deterministic algorithm that receives as input an index* $\mathbb{i}$ *and outputs a tuple* $(\mathbb{i}', \iota, \mathbb{I})$ *consisting of a new index* $\mathbb{i}'$*, a short index* ι *(explicit input for* V*), and a long index* $\mathbb{I}$ *(implicit input for* V*). Concretely, it computes oracles to multilinear polynomials in* $\mathbb{i}$ *and encodes them as an oracle description in* $\mathbb{I}$*.*
- Online phase. *The prover* P *is an interactive algorithm that receives as input* $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$*, and the verifier is an interactive oracle algorithm that receives as (explicit) input* $(\iota, \mathbb{x})$ *and query access to* $\mathbb{o}$*. They interact over* μ *rounds as follows.*
  In each round $i \in [\mu]$*,* P *sends a collection of polynomial oracles* π_i *and* V *sends a uniformly random message* $\rho_i \in \{0,1\}^{r_i}$*. After* μ *rounds of interaction,* V *makes queries to* $\mathbb{o}$ *and the oracles* $\pi_1, \ldots, \pi_\mu$ *sent by* P*. After that:*
  - V *outputs* $\mathsf{out}_1, \ldots, \mathsf{out}_k$*. For every* $i \in [k]$*,* out_i *is either* $\mathsf{accept}/\mathsf{reject}$*, or can be parsed as new instances and oracle instances* $(\mathbb{x}'_i, \mathbb{o}'_i)$*, and*
  - P *outputs new witnesses* $\mathbb{w}'_i$ *for every* $i \in [k]$ *such that* $(\mathbb{i}'; \mathbb{x}'_i; \mathbb{o}'_i; \mathbb{w}'_i) \in \mathsf{R}'_i$*.*

  Without loss of generality, V*'s output is a deterministic function of its input and the transcript.*

We say that PIOR has (perfect) completeness and soundness error $\epsilon : \mathbb{N} \mapsto [0,1]$ if the following holds.

- **Completeness.** For every $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$,

$$\Pr\left[\begin{array}{l|l} & (\mathbb{i}', \iota, \mathbb{I}) \leftarrow \mathsf{I}(\mathbb{i}) \\ & \pi_1 \leftarrow \mathsf{P}(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \\ & \rho_1 \leftarrow \{0,1\}^{r_1} \\ \forall i \in [k],\ \ \mathsf{out}_i = \mathsf{accept} \vee & \vdots \\ \Big(\mathsf{out}_i = (\mathbb{x}'_i; \mathbb{o}'_i) & \pi_\mu \leftarrow \mathsf{P}(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}; \rho_1, \ldots, \rho_{\mu-1}) \\ \quad \wedge (\mathbb{i}'; \mathbb{x}'_i; \mathbb{o}'_i; \mathbb{w}'_i) \in \mathsf{R}'_i\Big) & \rho_\mu \leftarrow \{0,1\}^{r_\mu} \\ & \mathbb{w}'_1, \ldots, \mathbb{w}'_k \leftarrow \mathsf{P}(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}; \\ & \qquad \rho_1, \ldots, \rho_\mu) \\ & \mathsf{out}_1, \ldots, \mathsf{out}_k \leftarrow \\ & \quad \mathsf{V}^{\mathbb{o}, \pi_1, \ldots, \pi_\mu}((\iota, \mathbb{I}); \mathbb{x}; \rho_1, \ldots, \rho_\mu) \end{array}\right] = 1$$

- **Soundness.** Let $\mathsf{L}(\mathsf{R})$ be the language of R. For every $(\mathbb{i}; \mathbb{x}; \mathbb{o}) \notin \mathsf{L}(\mathsf{R})$ and unbounded malicious prover $\tilde{\mathsf{P}}$,

$$\Pr\left[\begin{array}{l|l} & (\mathbb{i}', \iota, \mathbb{I}) \leftarrow \mathsf{I}(\mathbb{i}) \\ & \pi_1 \leftarrow \tilde{\mathsf{P}} \\ & \rho_1 \leftarrow \{0,1\}^{r_1} \\ \forall i \in [k],\ \ \mathsf{out}_i = \mathsf{accept} \vee & \vdots \\ \Big(\mathsf{out}_i = (\mathbb{x}'_i; \mathbb{o}'_i) & \pi_\mu \leftarrow \tilde{\mathsf{P}}(\rho_1, \ldots, \rho_{\mu-1}) \\ \quad \wedge (\mathbb{i}'; \mathbb{x}'_i; \mathbb{o}'_i; \mathbb{w}'_i) \in \mathsf{R}'_i\Big) & \rho_\mu \leftarrow \{0,1\}^{r_\mu} \\ & \mathbb{w}'_1, \ldots, \mathbb{w}'_k \leftarrow \tilde{\mathsf{P}}(\rho_1, \ldots, \rho_\mu) \\ & \mathsf{out}_1, \ldots, \mathsf{out}_k \leftarrow \\ & \quad \mathsf{V}^{\mathbb{o}, \pi_1, \ldots, \pi_\mu}((\iota, \mathbb{I}); \mathbb{x}; \rho_1, \ldots, \rho_\mu) \end{array}\right] \leq \epsilon(\mathbb{i}, \mathbb{x}, \mathbb{o})$$

Efficiency measures.
- # Rounds: Number of rounds of interaction. A round is a prover message and a verifier message.
- Oracles: Number of oracles sent by the prover, the total number of bits in the oracles, and the number of field elements in the oracles.
- Prover Work: Number of bit operations performed by the prover, excluding the work needed to compute oracles, if any.
- Proof Size: Number of bits communicated by the prover to the verifier, excluding the oracles, if any.
- Verifier Work: Number of bit operations performed by the verifier.
- Oracle Queries: Number of queries made by the verifier to the oracles.

Remark 4.16. If a PIOR for relation R is of the form $\mathsf{R} \rightarrow \perp$, or equivalently, the verifier only outputs $\mathsf{accept}/\mathsf{reject}$ at the end of the protocol, then it is a PIOP for R.

Remark 4.17. One can obtain a PIOP for relation R by continuing a PIOR for R with the PIOP(s) for the reduced relation(s). The soundness error of the PIOP is at most the sum of the soundness errors of each of its PIOR components.

Lemma 4.18 is adapted from Lemma 2.3 in [CBBZ23] but generalized to PIORs and cases where oracles of polynomials in $\mathbb{w}$ are to be sent during the protocol. We omit the proof since it is trivially the same.

Lemma 4.18 (Sound PIORs are knowledge sound)**.** *Consider a PIOR with soundness error ϵ for indexed oracle relation R, such that for all $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$, $\mathbb{w}$ consists only of polynomials such that oracles to these*

polynomials are contained in either $\mathbb{o}$ or the transcript of the protocol (i.e. sent by the prover during runtime). The PIOR has knowledge-soundness error ϵ, and the extractor runs in time $O(|\mathbb{w}|)$.

As pointed out by Lemma A.1 in [SSPP26], the same statement can be made for the round-by-round soundness of such PIORs by applying the same extraction idea of Lemma 2.3 in [CBBZ23].

Lemma 4.19 (Round-by-round sound PIORs are round-by-round knowledge sound)**.** *Consider a PIOR with round-by-round soundness error ϵ for indexed oracle relation R, such that for all $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$, $\mathbb{w}$ consists only of polynomials such that oracles to these polynomials are contained in either $\mathbb{o}$ or the transcript of the protocol (i.e. sent by the prover during runtime). The PIOR has round-by-round knowledge-soundness error ϵ, and the extractor runs in time $O(|\mathbb{w}|)$.*

4.6 PIOP Compilation

PIOP compilation transforms a PIOP into an interactive argument of knowledge Π by replacing the oracles with polynomial commitments. Every query by the verifier is replaced with an invocation of the Eval protocol at the query point. The compiled verifier accepts if the PIOP verifier accepts and if the output of all Eval invocations is 1. If Π is public-coin, then it can be further compiled to a Non-interactive ARgument of Knowledge (NARK) through the Fiat-Shamir transform.

Theorem 4.20 (PIOP Compilation)**.** *[BFS20; CHMMVW20] If the polynomial commitment scheme PCS has witness-extended emulation, and if the t-round Polynomial IOP for $\mathcal{R}$ has negligible knowledge error, then Π, the output of the PIOP compilation, is a secure (non-oracle) argument of knowledge for $\mathcal{R}$. The compilation also preserves zero knowledge. If PCS is hiding and Eval is honest-verifier zero-knowledge, then Π is honest-verifier zero-knowledge. The efficiency of the resulting argument of knowledge Π depends on the efficiency of both the PIOP and PCS:*

- *Prover time: The prover time is equal to the sum of (i) prover time of the PIOP, (ii) the oracle length times the commitment time, and (iii) the query complexity times the prover time of PCS.*
- *Verifier time: The verifier time is equal to the sum of (i) the verifier time of the PIOP and (ii) the verifier time for PCS times the query complexity of the PIOP.*
- *Proof size: The proof size is equal to the sum of (i) the message complexity of the PIOP times the commitment size and (ii) the query complexity times the proof size of PCS. If the proof size is $O(\log^c(|\mathbb{w}|))$, then we say the proof is succinct.*

Batching: *The prover time, verifier time, and proof size can be significantly reduced using batch openings of the polynomial commitments. After batching, the proof size only depends on the number of oracles plus a single polynomial commitment opening.*

4.7 Mod-PIORs and Their Compilation

Mod-Oracle. An integer mod-oracle $[[f]]$ of polynomial f is a polynomial oracle that opens the integer evaluations mod some given modulus. Integer mod-oracles can be instantiated using integer mod-PCS.

Mod-PIOR. A mod-PIOR/PIOP is a PIOR/PIOP in which some or all the polynomial oracles (in either its index, oracle instance, or transcript) are mod-oracles. An integer mod-PIOP is one where the oracles are integer mod-oracles and can be compiled using an integer mod-PCS to yield an interactive argument.
In the following, we adapt Lemmas 4.18 and 4.19 for integer mod-PIORs.

Lemma 4.21 (Sound integer mod-PIORs are knowledge sound)**.** *Consider an integer mod-PIOR with soundness error ϵ for indexed oracle relation R, such that for all $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$, $\mathbb{w}$ consists only of integer polynomials such that they are norm bounded by some T that is $O(\lambda)$ bits long, and integer mod-oracles to these polynomials are contained in either $\mathbb{o}$ or the transcript of the protocol (i.e. sent by the prover during runtime). The mod-PIOR has knowledge-soundness ϵ, and the extractor runs in time $O_\lambda(|\mathbb{w}|)$, where O_λ hides $\mathsf{poly}(\lambda)$ factors.*

Proof. Let $\mathsf{L}(\mathsf{R})$ be the language corresponding to R such that $(\mathbb{i}; \mathbb{x}; \mathbb{o}) \in \mathsf{L}(\mathsf{R})$ if and only if $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$. We will show that we can construct an extractor Ext that can produce $\mathbb{w}^*$ with an error the same as the soundness error such that $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}^*) \in \mathsf{R}$ if and only if $(\mathbb{i}; \mathbb{x}; \mathbb{o}) \in \mathsf{L}(\mathsf{R})$.

For each integer mod-oracle of a n-variate integer polynomial f with degree d in each variable, let integer $s' = \lceil \frac{\log \mathsf{T} + 3dn \log(d+1)}{\lambda - 1} \rceil$, which is $O(1)$ since $\log \mathsf{T} = O(\lambda)$ and $dn \log(d+1) = O(\lambda)$ (with $d = O(1)$ and $n = O(\lambda)$. Ext picks s' distinct primes $p_1, \ldots, p_{s'} \leftarrow \mathsf{Primes}(2^{\lambda-1}, 2^\lambda)$ and let $\{\boldsymbol{r}_1, \ldots, \boldsymbol{r}_{(d+1)^n}\}$ be the $(d+1)^n$ distinct integer points in $\{0, \ldots, d\}^n$. For each point $\boldsymbol{r}_i$, Ext queries the integer mod-oracle $[[f]]$ at $\boldsymbol{r}_i$ mod p_j for each $j \in [s']$, namely s' queries, to reconstruct the integer evaluation at $\boldsymbol{r}_i$, which can be done in $O((s')^2\lambda^2) = O(\lambda^2)$ time (see Remark 4.4). In total, the extractor queries $[[f]]$ for $s'(d+1)^n = s'|f| = O(|f|)$ times to extract f, and thus the total number of queries to produce $\mathbb{w}^*$ is $O(|\mathbb{w}|)$.

If $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}^*) \in \mathsf{R}$ then by definition trivially $(\mathbb{i}; \mathbb{x}; \mathbb{o}) \in \mathsf{L}(\mathsf{R})$. Now we prove the other direction; assume that $(\mathbb{i}; \mathbb{x}; \mathbb{o}) \in \mathsf{L}(\mathsf{R})$ but $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}^*) \notin \mathsf{R}$. Then there must exist $\mathbb{w} \neq \mathbb{w}^*$ such that $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$. The product of the primes is at least $(2^{\lambda-1})^{s'} \geq 2^{\log \mathsf{T} + 3dn \log(d+1)}$; the integer evaluation at $\boldsymbol{r}_i$ for any $i \in [(d+1)^n]$ is at most $\mathsf{T} \cdot (d+1)^n \cdot d^{dn} = 2^{\log \mathsf{T} + n \log(d+1) + dn \log d} \leq 2^{\log \mathsf{T} + 2dn \log(d+1) + n}$, because there are at most $(d+1)^n$ nonzero terms in the polynomial, and each term is a product of a coefficient at most T, at most n elements each with norm at most d and exponent at most d. By the Chinese Remainder Theorem (Theorem 4.3), the reconstructed integer must be unique since it is smaller than the product of the primes. Moreover, since the relation only admits polynomials as witnesses and these polynomials are degree d and n-variate, there cannot be two distinct witnesses that agree on $(d+1)^n$ oracle queries. Therefore, $\mathbb{w} = \mathbb{w}^*$, which leads to a contradiction. The $\mathbb{w}^*$ output by the extractor is, thus, the unique and valid witness for $(\mathbb{i}; \mathbb{x}; \mathbb{o})$ in the language $\mathsf{L}(\mathsf{R})$, and the knowledge soundness error is the same as the soundness error. □

Lemma 4.22 (Round-by-round sound integer mod-PIORs are round-by-round knowledge sound)**.** *Consider an integer mod-PIOR with round-by-round soundness error ϵ for indexed oracle relation R, such that for all $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}$, $\mathbb{w}$ consists only of integer polynomials such that they are norm bounded by some T that is $O(\lambda)$ bits long, and integer mod-oracles to these polynomials are contained in either $\mathbb{o}$ or the transcript of the protocol (i.e. sent by the prover during runtime). The mod-PIOR has round-by-round knowledge-soundness ϵ, and the extractor runs in time $O_\lambda(|\mathbb{w}|)$.*

Proof. The extractor construction is the same as in the proof of Lemma 4.21, adapted to the round-by-round notion. □

4.8 Useful Relations

Definition 4.23 (Range oracle relation)**.**

$$\mathsf{R}_{\mathsf{Range}} := \left\{ (\mathsf{T};\ [[f]];\ f) \;\middle|\; \mathsf{T} \in \mathbb{Z}_{\geq 0},\ f \in \mathbb{F}^{\leq 1}[n],\ \|f\| < \mathsf{T} \right\}.$$

Definition 4.24 (Batch range oracle relation)**.**

$$\mathsf{R}_{\mathsf{BatchRange}(k)} := \left\{ \left(\mathsf{T};\ ([[f_i]])_{i\in[k]};\ (f_i)_{i\in[k]}\right) \;\middle|\; \begin{array}{l} \mathsf{T} \in \mathbb{Z}_{\geq 0},\ \forall i \in [k]: \\ f_i \in \mathbb{F}^{\leq 1}[n],\ \|f_i\| < \mathsf{T} \end{array} \right\}.$$

Definition 4.25 (R1CS relation)**.** *The indexed oracle relation* $\mathsf{R}_{\mathsf{R1CS}}(\mathbb{F})$ *is the set of all tuples* $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) = ((\hat{A}, \hat{B}, \hat{C}); \hat{x}; [[\hat{w}]]; \hat{w})$ *such that for some field* $\mathbb{F}$*,* $\hat{A}, \hat{B}, \hat{C} \in \mathbb{F}^{\leq 1}[2n], \hat{x} \in \mathbb{F}^{\leq 1}[n_x], \hat{w} \in \mathbb{F}^{\leq 1}[n_w]$*, let* $n := \log(2^{n_x} + 2^{n_w})$ *and* $\hat{z} \in \mathbb{F}^{\leq 1}[n] := (\hat{x}, \hat{w})$*, and the following holds for every* $\boldsymbol{i} \in \{0,1\}^n$*,*

$$\Big(\sum_{\boldsymbol{j} \in \{0,1\}^n} \hat{A}(\boldsymbol{i}, \boldsymbol{j}) \hat{z}(\boldsymbol{j}) \Big) \Big(\sum_{\boldsymbol{j} \in \{0,1\}^n} \hat{B}(\boldsymbol{i}, \boldsymbol{j}) \hat{z}(\boldsymbol{j}) \Big) = \sum_{\boldsymbol{j} \in \{0,1\}^n} \hat{C}(\boldsymbol{i}, \boldsymbol{j}) \hat{z}(\boldsymbol{j}) \tag{5}$$

Definition 4.26 (Sumcheck relation)**.**

$$\mathsf{R}_{\mathsf{SUM}} := \left\{ (v;\ [[f]];\ f) \;\middle|\; f \in \mathbb{F}^{\leq d}[n],\ \textstyle\sum_{\boldsymbol{x} \in \{0,1\}^n} f(\boldsymbol{x}) = v \right\}.$$

Theorem 4.27. *The PIOP for* $\mathsf{R}_{\mathsf{SUM}}$ *is perfectly complete and has knowledge error* $\delta^{d,n}_{\mathsf{SUM}} := dn/|\mathbb{F}|$*.*

The classic sumcheck protocol [LFKN90] is a PIOP for $\mathsf{R}_{\mathsf{SUM}}$. Given a tuple $(\mathbb{x}; \mathbb{o}; \mathbb{w}) = (v; [[f]]; f)$ for n-variate degree d polynomial f such that $\sum_{\boldsymbol{x} \in \{0,1\}^n} f(\boldsymbol{x}) = v$:

- For $k = 1, \ldots, n$:
  - The prover computes $u_k(X) := \sum_{\boldsymbol{x} \in \{0,1\}^{n-k}} f(\boldsymbol{\alpha}, X, \boldsymbol{x})$, where $\boldsymbol{\alpha} := [\alpha_i]_{i=1}^{k-1}$ are challenges. The prover sends the oracle $[[u_k]]$ to the verifier. u_k is univariate and of individual degree at most d.
  - The verifier checks that $v = u_k(0) + u_k(1)$, samples $\alpha_k \leftarrow \mathbb{F}$, sends α_k to the prover, and sets $v \leftarrow u_k(\alpha_k)$.
- Finally, the verifier accepts if $f(\boldsymbol{\alpha}) = v$.

We refer to [Tha20] for the proof of Theorem 4.27.

4.9 Number of Primes and Fingerprinting of Polynomial Evaluation

Theorem 4.28 (Number of Primes [RS62])**.** *Let* $\pi(x)$ *denote the number of primes less than or equal to* x*. Then for* $x \geq 17$*,*

$$\frac{x}{\ln x} < \pi(x) < 1.25506 \frac{x}{\ln x}$$

Lemma 4.29 (Probabilistic Vanishing of Integer)**.** *Let* z *be a* T_z*-bounded integer. If* $z \neq 0$*, then*

$$\Pr_{p \xleftarrow{\$} \mathsf{Primes}(2^{\lambda-1}, 2^{\lambda})} [z \equiv_p 0] \leq \frac{\log \mathsf{T}_z}{2^{\lambda-2}}$$

Proof. Let S be the distinct, λ-bit prime factors of z. For any λ-bit prime p, if $f(\boldsymbol{i}) \equiv_p 0$, then it must hold that $p \in S$. Therefore,

$$\begin{aligned}
&\Pr_{p \xleftarrow{\$} \mathsf{Primes}(2^{\lambda-1}, 2^{\lambda})} [z \equiv_p 0] = \frac{|S|}{|\mathsf{Primes}(2^{\lambda-1}, 2^{\lambda})|} \\
&\leq \frac{\log_{2^{\lambda-1}}(z)}{\pi(2^{\lambda}) - \pi(2^{\lambda-1})} = \frac{\log z / \log(2^{\lambda-1})}{\pi(2^{\lambda}) - \pi(2^{\lambda-1})} \\
&\leq \frac{\log \mathsf{T}_z}{(\lambda - 1) \cdot \frac{1}{\ln 2} \left(\frac{2^{\lambda}}{\lambda} - 1.25506 \cdot \frac{2^{\lambda-1}}{\lambda - 1} \right)}
\end{aligned}$$

$$= \frac{\ln 2 \cdot \log \mathsf{T}_z}{\frac{2^\lambda(\lambda-1)}{\lambda} - 1.25506 \cdot 2^{\lambda-1}} \leq \frac{\ln 2 \cdot \log \mathsf{T}_z}{0.5 \cdot 2^{\lambda-1}} \leq \frac{\log \mathsf{T}_z}{2^{\lambda-2}}$$

The penultimate inequality holds because $\lambda \geq 9 > \frac{2}{2-1.75506}$, hence $2\lambda - 2 > 1.75506\lambda$, and therefore $\frac{2(\lambda-1)}{\lambda} \geq 1.75506$, giving $\frac{2^\lambda(\lambda-1)}{\lambda} - 1.25506 \cdot 2^{\lambda-1} \geq 0.5 \cdot 2^{\lambda-1}$; the last inequality uses $\ln 2 < 1$. □

5 Limber: An Integer Mod-PCS Compiler

In this section, we describe Limber, a compiler that transforms any $\mathsf{PCS} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Eval})$ over some field $\mathbb{F}$ with characteristic q into an integer mod-PCS. At a high level, the compiler commits to an integer polynomial by *limb-splitting* it into small-norm pieces and committing to those with the underlying PCS. Evaluation is reduced to evaluation of the limb-split polynomial via a limb-reconstructing sumcheck. Evaluating the limb-split polynomial is handled by our IntEval protocol, which reduces to underlying PCS evaluations. We describe the setup and commitment protocols (§5.1), the evaluation protocol (§5.2), and the core IntEval argument (§5.3). We discuss parameter selection (§5.4), and prove the security of Limber (§5.5).

Theorem 5.1. *Given any correct standard polynomial commitment scheme* $\mathsf{PCS} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Eval})$ *where* $\mathsf{PCS.Eval}$ *has knowledge error* ϵ_{PCS}*, and any perfectly complete batch range check argument with soundness error* ϵ_{rc}*, there exists an integer mod-PCS* modPCS *that is correct and whose* $\mathsf{modPCS.Eval}$ *(Protocol 5.3) is a public-coin interactive argument of knowledge for the integer mod-PCS evaluation relation* $\mathcal{R}_{\mathsf{modeval}}$ *(Definition 4.12) with knowledge error* $\epsilon_{\mathsf{PCS}} + \epsilon_{\mathsf{rc}} + \frac{2l}{p} + \epsilon_{\mathsf{IntEvalArg}}$.

Proof. See Section 5.5 for the proof. □

5.1 Integer Mod-PCS Setup and Commitment Protocols

We construct the setup and commitment algorithms of modPCS generically from an arbitrary standard polynomial commitment scheme PCS over a field $\mathbb{F}$. To commit to f, the prover first limb-splits it into a polynomial f_ℓ whose coefficients are bounded by T, casts f_ℓ into $\mathbb{F}$, and commits to it with PCS. A range check certifies that the committed field polynomial encodes small limbs.

Setup. T is the norm bound for limb-splitting, and T_f is the norm bound for the polynomial f that is committed later. Without loss of generality, we take T_f to be a power of T, i.e. $\mathsf{T}_f = \mathsf{T}^L$ where $L = \log_{\mathsf{T}} \mathsf{T}_f$ is the number of limbs (otherwise round T_f up to the next power of T). Setup runs $(\mathsf{vk}', \mathsf{ck}') \leftarrow \mathsf{PCS.Setup}(1^\lambda)$ and outputs $\mathsf{vk} := (\mathsf{vk}', \mathsf{T}, \mathsf{T}_f)$ and $\mathsf{ck} := (\mathsf{ck}', \mathsf{T}, \mathsf{T}_f)$.

Protocol 5.1: $\mathsf{modPCS.Setup}(1^\lambda, \mathsf{T}, \mathsf{T}_f)$

1. $(\mathsf{vk}', \mathsf{ck}') \leftarrow \mathsf{PCS.Setup}(1^\lambda)$.
2. Output $\mathsf{vk} := (\mathsf{vk}', \mathsf{T}, \mathsf{T}_f)$ and $\mathsf{ck} := (\mathsf{ck}', \mathsf{T}, \mathsf{T}_f)$.

Remark 5.2. Note that the polynomial norm bound T_f among the inputs to $\mathsf{modPCS.Setup}$ is specific to the polynomial f that will be committed. In other words, this setup algorithm is not universal. To achieve universal setup, one can use an idea similar to the $\mathsf{PC.Trim}$ algorithm in Marlin [CHMMVW20]. That is, let $\mathsf{modPCS.Setup}$ take as input $(1^\lambda, \mathsf{T}, \mathsf{T}_{\max})$, where $\mathsf{T}_{\max}$ is the maximum supported norm bound $\mathsf{T}_{\max}$, and output some public parameter pp. Later, use a "trimming" algorithm $\mathsf{modPCS.Trim}$ that trims the public parameter for a specific polynomial norm bound T_f and outputs a committer key ck and a verifier key vk.

Commit. The prover first splits the polynomial f into a limb polynomial f_ℓ with hypercube evaluations bounded by T, and then commits to the limb polynomial in $\mathbb{F}$ and sends the commitment to the verifier. The commitment is output by the verifier if it passes the range check.

Protocol 5.2: $\mathsf{modPCS.Commit}((\mathsf{ck}, \mathsf{vk}), f)$

$\underline{\mathsf{P}(\mathsf{ck} := (\mathsf{ck}', \mathsf{T}, \mathsf{T}_f), f) \leftrightarrow \mathsf{V}(\mathsf{vk} := (\mathsf{vk}', \mathsf{T}, \mathsf{T}_f)):}$

1. Prover limb splits f and casts it to a field: $\hat{f}_\ell \leftarrow \mathtt{toField}(\mathsf{LimbSplit}(f, \mathsf{T}, \mathsf{T}_f))$.
2. Prover computes $\mathsf{com}_f := \mathsf{PCS.Commit}(\mathsf{ck}', \hat{f}_\ell)$ and sends com_f to the verifier.
3. Prover and verifier run a Range Check argument (Range Check PIOP compiled with PCS) to prove that $|\hat{f}_\ell| < \mathsf{T}$. If the range check passes, verifier outputs com_f; otherwise, verifier outputs reject.

Remark 5.3. For an honest committer, modPCS.Commit never rejects: a correctly limb-split f_ℓ is T-bounded, so the range check passes.

5.2 Integer Mod-PCS Evaluation Protocol

The modPCS evaluation protocol proves membership in the integer mod-PCS evaluation relation $\mathcal{R}_{\mathsf{modeval}}$ (Definition 4.12), i.e. that $f(\boldsymbol{r}_f) \equiv_p v$ for the committed integer polynomial f, opening point $\boldsymbol{r}_f \in \mathbb{Z}_p^n$, and claimed evaluation $v \in \mathbb{Z}_p$. To do this, the prover and verifier first run a sumcheck to reduce the claim to an evaluation claim in $\mathbb{Z}_p$ about the limb-split polynomial. Then the prover sends over the full integer evaluation of the limb-split polynomial at a random point, the verifier checks the congruence of the integer evaluation with the claimed evaluation in $\mathbb{Z}_p$, and they run an IntEval argument (Section 5.3) to prove the correctness of the integer evaluation. The IntEval argument is run on the limb-split polynomial f_ℓ, which has $m = n + l$ variables, so Theorem 5.5 and Corollary 5.8 apply with this m.

Protocol 5.3: $\mathsf{modPCS.Eval}((\mathsf{ck}, \mathsf{vk}); \mathsf{com}_f, p, \boldsymbol{r}_f, v; f)$

$\underline{\mathsf{P}(\mathsf{ck}, p, \boldsymbol{r}_f, v, f) \leftrightarrow \mathsf{V}(\mathsf{vk}, \mathsf{com}_f, p, \boldsymbol{r}_f, v):}$

1. Let n be the number of variables in f and $n + l$ the number of variables in f_ℓ.
   Prover and Verifier run a sumcheck argument to prove the following,
   $$\sum_{\boldsymbol{k} \in \{0,1\}^l} \ell(\boldsymbol{k}) \cdot f_\ell(\boldsymbol{r}_f, \boldsymbol{k}) \equiv_p v$$
   At the end of this sumcheck, Prover sends a claimed mod p evaluation $y' \in \mathbb{Z}_p$, and Verifier needs to check in the following steps $y' \stackrel{?}{\equiv}_p f_\ell(\boldsymbol{r}_f, \boldsymbol{r}_\ell)$, where $\boldsymbol{r}_\ell \in \mathbb{Z}_p^l$ are the random challenges.
2. Prover sends the integer evaluation $y \in \mathbb{Z}$, and Verifier directly checks $y' \stackrel{?}{=} y \bmod p$.
3. Prover and Verifier run the IntEval argument (the compilation of the IntEval PIOR 5.4 and a Range Check PIOP with PCS) to check $y \stackrel{?}{=} f_\ell(\boldsymbol{r}_f, \boldsymbol{r}_\ell)$.

5.3 Integer Evaluation Protocol

We define the integer evaluation relation $\mathsf{R}_{\mathsf{IntEval}}$ as follows. The main difference from the standard polynomial evaluation relation $\mathcal{R}_{\mathsf{eval}}$ (Definition 4.9) is that the evaluation point and the claimed evaluation are integers as opposed to $\mathbb{F}$ elements. Note that the opening point $\boldsymbol{r}$ needs to be bounded by p as well.

Definition 5.4 (Integer evaluation relation)**.**

$$\mathsf{R}_{\mathsf{IntEval}}(p) := \left\{ \left((\boldsymbol{r}, y);\, [[\hat{g}]];\, \hat{g}\right) \;\middle|\; \begin{array}{l} \hat{g} \in \mathbb{F}^{\leq 1}[m],\ \boldsymbol{r} \in \mathbb{Z}_p^m,\ y \in \mathbb{Z}, \\ g(\boldsymbol{r}) = y \ \textit{ where } \ g := \mathtt{toInt}(\hat{g}) \end{array} \right\},$$

where $\mathbb{F}$ is a field with characteristic q.

In PIOR 5.4, we propose a public-coin interactive protocol that proves the integer evaluation relation. Our protocol hinges on a randomized version of the Chinese Remainder Theorem (Theorem 4.3) - an incorrect evaluation cannot agree with the correct evaluation modulo too many random small primes. Therefore, if we sample enough of these random prime moduli p_i, we can ensure that the probability of accepting an incorrect evaluation is negligible.

For each of the s random small primes p_i, we repeatedly partially evaluate f at k points of the random challenges $\bmod p_i$. The key trick is that avoiding overflow in $\mathbb{F}$ (in particular q, the characteristic) is equivalent to operating over $\mathbb{Z}$. Therefore, our partially evaluated polynomial is still consistent with our original polynomial mod p_i as long as k is not too large to cause overflow. In each iteration, we split the partially evaluated polynomial into remainder and quotient polynomials as a method of norm reduction, and recursively iterate only on the remainder polynomial until it has no more than k variables. This then reduces to standard polynomial evaluation, and we check that the evaluation at the last k points of $\boldsymbol{r} \bmod p_i$ is congruent to y modulo p_i.

PIOR 5.4: $\mathsf{R}_{\mathsf{IntEval}}(p) \to \mathsf{R}^2_{\mathsf{BatchRange}(st)}$

Parse: $\mathbb{x} = (\boldsymbol{r}, y); \mathbb{o} = [[\hat{g}]]; \mathbb{w} = \hat{g}$

1. Verifier checks that the claimed evaluation is bounded, $|y| < M \cdot ||\boldsymbol{r}||^m \cdot \mathsf{T}$, and rejects if it is not. Verifier samples random small prime $p_i \leftarrow \mathsf{Primes}(P/2, P)$ for every $i \in [s]$. Define $\hat{\boldsymbol{r}}^{(i)} := \mathtt{toField}(\boldsymbol{r} \bmod p_i)$ for every $i \in [s]$.

2. Prover initializes polynomials $\hat{a}_0^{(i)} \leftarrow \hat{g}$ for every $i \in [s]$. Define the number of iterations $\mathsf{t} = \lceil (m-k)/k \rceil$.
   For every $i \in [s], j \in [\mathsf{t}]$:

a) Prover partially evaluates $\hat{a}_{j-1}^{(i)}$ at k variables to get the following $(m - jk)$-variate multilinear polynomial,
$$\hat{g}_j^{(i)}(\boldsymbol{X}_1, \ldots, \boldsymbol{X}_{m-jk}) = \hat{a}_{j-1}^{(i)}(\boldsymbol{X}_1, \ldots, \boldsymbol{X}_{m-jk}, \hat{\boldsymbol{r}}_{m-jk+1}^{(i)}, \ldots, \hat{\boldsymbol{r}}_{m-(j-1)k}^{(i)})$$
such that $||\hat{g}_j^{(i)}|| < (q-P)/2$

b) Prover decomposes $\hat{g}_j^{(i)}$ into unique $(m-jk)$-variate multilinear remainder and quotient polynomials $\hat{a}_j^{(i)}, \hat{b}_j^{(i)}$ such that $\forall \boldsymbol{x} \in \{0,1\}^{m-jk}, \hat{a}_j^{(i)}(\boldsymbol{x}) = \hat{g}_j^{(i)}(\boldsymbol{x}) \bmod p_i$ and $\hat{b}_j^{(i)}(\boldsymbol{x}) = \lfloor \hat{g}_j^{(i)}(\boldsymbol{x})/p_i \rfloor$. Based

on these definitions, we have the following integer equation for every $\boldsymbol{x} \in \{0,1\}^{m-jk}$,

$$a_j^{(i)}(\boldsymbol{x}) + p_i \cdot b_j^{(i)}(\boldsymbol{x}) = g_j^{(i)}(\boldsymbol{x}) \tag{6}$$

Prover sends polynomial oracles $[[\hat{a}_j^{(i)}]], [[\hat{b}_j^{(i)}]]$ to the verifier.

3) Verifier batch queries the final remainder polynomial $[[\hat{a}_{\mathsf{t}}^{(i)}]]$ at $(\hat{\boldsymbol{r}}_1^{(i)}, \ldots, \hat{\boldsymbol{r}}_{m-\mathsf{t}k}^{(i)})$ for every $i \in [s]$ and gets back response $\hat{y}_i$. Verifier checks whether $\texttt{toInt}(\hat{y}_i) \equiv_{p_i} y$.

4) Verifier selects random $\hat{\boldsymbol{\gamma}} \in \mathbb{F}^{m-k}$. Verifier batch queries $[[\hat{a}_{j-1}^{(i)}]], [[\hat{a}_j^{(i)}]]$, $[[\hat{b}_j^{(i)}]]$ for every $i \in [s], j \in [\mathsf{t}]$, to check proper decomposition:
$\hat{a}_j^{(i)}(\hat{\boldsymbol{\gamma}}_1, \ldots, \hat{\boldsymbol{\gamma}}_{m-jk}) + p_i \cdot \hat{b}_j^{(i)}(\hat{\boldsymbol{\gamma}}_1, \ldots, \hat{\boldsymbol{\gamma}}_{m-jk}) = \hat{a}_{j-1}^{(i)}(\hat{\boldsymbol{\gamma}}_1, \ldots, \hat{\boldsymbol{\gamma}}_{m-jk}, \hat{\boldsymbol{r}}_{m-jk+1}^{(i)}, \ldots, \hat{\boldsymbol{r}}_{m-(j-1)k}^{(i)})$

5) To verify that $[[\hat{a}_j^{(i)}]], [[\hat{b}_j^{(i)}]]$ for every $i \in [s], j \in [\mathsf{t}]$ all have correctly bounded norms, prover outputs the following two witnesses

$$\mathbb{w}'_a = \{\hat{a}_j^{(i)}\}_{i=1,j=1}^{s,\mathsf{t}}, \quad \mathbb{w}'_b = \{\hat{b}_j^{(i)}\}_{i=1,j=1}^{s,\mathsf{t}}$$

and verifier outputs the associated instances and oracle instances for each $\mathsf{R}_{\mathsf{BatchRange}(s\mathsf{t})}$, respectively,

$$(\mathbb{x}'_a; \mathbb{o}'_a) = (P; \{[[\hat{a}_j^{(i)}]]\}_{i=1,j=1}^{s,\mathsf{t}}), \quad (\mathbb{x}'_b; \mathbb{o}'_b) = (\left\lfloor \frac{q}{P} \right\rfloor; \{[[\hat{b}_j^{(i)}]]\}_{i=1,j=1}^{s,\mathsf{t}})$$

Theorem 5.5. *Assume the number of variables $m \leq \lambda$, the polynomial norm is upper-bounded by $||g|| < \mathsf{T} \leq 2^\lambda$, the field characteristic q satisfies $(q-P)/2 \geq 2^k \cdot P^k \cdot \max(\mathsf{T}, P)$, $||\boldsymbol{r}||$ is fewer than λ bits, and $\log P \geq 5$. Then PIOR 5.4 for $\mathsf{R}_{\mathsf{IntEval}}(p)$ has perfect completeness and a soundness error of*

$$\epsilon_{\mathsf{IntEval}} = \left(\frac{32\lambda m}{P}\right)^s + \frac{sm^2}{k|\mathbb{F}|}$$

and the following properties:

# Rounds	1
# Oracles, Length, Size (in total bits)	1, $\frac{2sM}{2^k-1}$, $\frac{sM \log q}{2^k-1}$
Prover Work	$sM \cdot \tilde{O}(\log P + \log \mathsf{T})$ bit operations
Proof Size	0
Verifier Work	$sm\lambda$ bit operations
Verifier Oracle Queries	$s+1$

Proof.

Completeness. Given an honest prover, we have that $y = g(r)$. Now, for each $p_i, i \in [s]$ that the verifier samples, we show that the verifier checks hold. In particular, we show the following lemma:

Lemma 5.6. *For every iteration $j \in [0, \ldots, \mathsf{t}]$, the remainder polynomial $a_j^{(i)}(\boldsymbol{r}^{(i)}) \equiv_{p_i} y$ and $||\hat{a}_j^{(i)}|| < \max(\mathsf{T}, P)$.*

Proof. The base case $j = 0$ is true because

$$a_0^{(i)}(\boldsymbol{r}^{(i)}) = g(\boldsymbol{r}^{(i)}) \equiv_{p_i} g(\boldsymbol{r}) = y$$

and by definition $||\hat{a}_0^{(i)}|| = ||\hat{g}|| < \mathsf{T} \leq \max(\mathsf{T}, P)$.
For $j > 0$, assume by induction that $a_{j-1}^{(i)}(\boldsymbol{r}^{(i)}) \equiv_{p_i} y$ and $||\hat{a}_{j-1}^{(i)}|| < \max(\mathsf{T}, P)$. In Step 2a, we have that

$$\hat{g}_j^{(i)}(\boldsymbol{X}) = \hat{a}_{j-1}^{(i)}(X_1, \ldots, X_{m-jk}, \hat{\boldsymbol{r}}_{m-jk+1}^{(i)}, \ldots, \hat{\boldsymbol{r}}_{m-(j-1)k}^{(i)}).$$

We claim that these polynomials are not just equal over $\mathbb{F}$ but also over $\mathbb{Z}$, which is true as long as there is no q overflow during partial evaluation. To be explicit:

$$\begin{aligned}
&a_{j-1}^{(i)}(X_1, \ldots, X_{m-jk}, \\
&\qquad r_{m-jk+1}^{(i)}, \ldots, r_{m-(j-1)k}^{(i)}) \\
&= \sum_{\boldsymbol{x} \in \{0,1\}^{m-jk}} \mathsf{eq}(\boldsymbol{X}; \boldsymbol{x}) \sum_{\boldsymbol{x}' \in \{0,1\}^k} a_{j-1}^{(i)}(\boldsymbol{x}, \boldsymbol{x}') \\
&\qquad \cdot \mathsf{eq}(r_{m-jk+1}^{(i)}, \ldots, r_{m-(j-1)k}^{(i)}; \boldsymbol{x}')
\end{aligned}$$

Since $||a_{j-1}^{(i)}|| < \max(\mathsf{T}, P)$, we have that hypercube evaluation $|a_{j-1}^{(i)}(\boldsymbol{x}, \boldsymbol{x}')| < \max(\mathsf{T}, P)$ and

$$|\mathsf{eq}(r_{m-jk+1}^{(i)}, \ldots, r_{m-(j-1)k}^{(i)}; \boldsymbol{x}')| < P^k,$$

since it is a product of k terms, each of which has norm at most P, when evaluated with $\boldsymbol{x}'$, a hypercube point. Moreover, there are 2^k terms in the inner sum, so by repeated application of the triangle inequality, we have that $||g_j^{(i)}|| < 2^k P^k \cdot \max(\mathsf{T}, P)$, which we can bound to be at most $(q - P)/2$ with appropriate settings of P, k, T. Given this, we have that $\hat{g}_j^{(i)}(\boldsymbol{X})$ does not overflow in $\mathbb{F}$ (specifically q), and therefore the equality holds over $\mathbb{Z}$: $g_j^{(i)}(\boldsymbol{r}^{(i)}) = a_{j-1}^{(i)}(\boldsymbol{r}^{(i)}) \equiv_{p_i} y$. Next, $\hat{g}_j^{(i)}(\boldsymbol{X})$ is broken up into $\hat{a}_j^{(i)}(\boldsymbol{X}), \hat{b}_j^{(i)}(\boldsymbol{X})$, where $\hat{a}_j^{(i)}(\boldsymbol{X}) = \hat{g}_j^{(i)}(\boldsymbol{X}) \bmod p_i$ and $\hat{b}_j^{(i)}(\boldsymbol{X}) = \lfloor \hat{g}_j^{(i)}(\boldsymbol{X})/p_i \rfloor$ over the $m - jk$ hypercube. If we expand $a_j^{(i)}(\boldsymbol{X})$ evaluated at $\boldsymbol{r}^{(i)}$:

$$\begin{aligned}
a_j^{(i)}(\boldsymbol{r}^{(i)}) &= \sum_{\boldsymbol{x} \in \{0,1\}^{m-jk}} \mathsf{eq}(\boldsymbol{r}^{(i)}; \boldsymbol{x}) a_j^{(i)}(\boldsymbol{x}) \\
&= \sum_{\boldsymbol{x} \in \{0,1\}^{m-jk}} \mathsf{eq}(\boldsymbol{r}^{(i)}; \boldsymbol{x}) \big(g_j^{(i)}(\boldsymbol{x}) \bmod p_i\big) \\
&\equiv_{p_i} \sum_{\boldsymbol{x} \in \{0,1\}^{m-jk}} \mathsf{eq}(\boldsymbol{r}^{(i)}; \boldsymbol{x}) g_j^{(i)}(\boldsymbol{x}) \\
&= g_j^{(i)}(\boldsymbol{r}^{(i)}) \equiv_{p_i} y
\end{aligned}$$

We also note that by definition, $||\hat{a}_j^{(i)}|| < p_i \leq \max(\mathsf{T}, P)$. Thus, by induction, we have that $a_j^{(i)}(\boldsymbol{r}^{(i)}) \equiv_{p_i} y$ and $||\hat{a}_j^{(i)}|| < \max(\mathsf{T}, P)$ for every $j \in [\mathsf{t}]$.

□

Returning to the completeness proof:
Step 1: The bound check passes, since the honest evaluation satisfies $y = g(\boldsymbol{r}) = \sum_{\boldsymbol{x} \in \{0,1\}^m} \mathsf{eq}(\boldsymbol{r}; \boldsymbol{x}) g(\boldsymbol{x})$, where each $\mathsf{eq}(\boldsymbol{r}; \boldsymbol{x})$ term is bounded by $||\boldsymbol{r}||^m$ and hypercube evaluation $g(\boldsymbol{x})$ is bounded by T by definition.

Therefore, by the triangle inequality over the $M = 2^m$ terms, we get that $|y|$ is at most $M \cdot ||\boldsymbol{r}||^m \cdot \mathsf{T}$.
Step 3: Since $2^k P^{k+1} \leq q$, the evaluation $\hat{a}_i^{(\mathsf{t})}(\hat{r}_1^{(i)}, \ldots, \hat{r}_{m-\mathsf{t}k}^{(i)})$ does not overflow q and is therefore the same as $a_i^{(\mathsf{t})}(r_1^{(i)}, \ldots, r_{m-\mathsf{t}k}^{(i)})$ over $\mathbb{Z}$. Based on Lemma 5.6, since $a_i^{(\mathsf{t})}(r_1^{(i)}, \ldots, r_{m-\mathsf{t}k}^{(i)}) \equiv_{p_i} y$, we have that $\hat{a}_i^{(\mathsf{t})}(\hat{r}_1^{(i)}, \ldots, \hat{r}_{m-\mathsf{t}k}^{(i)}) \equiv_{p_i} y$ as desired.
Step 4: By definition, $\forall \boldsymbol{x} \in \{0,1\}^{m-jk}, \hat{a}_j^{(i)}(\boldsymbol{x}) + p_i \cdot \hat{b}_j^{(i)}(\boldsymbol{x}) = \hat{g}_j^{(i)}(\boldsymbol{x})$. Since these are all multilinear polynomials, it must hold over $\mathbb{F}$, so the verifier's check will always pass.
Step 5: The batch range checks will always pass since $i \in [s], j \in [\mathsf{t}]$, $||\hat{a}_j^{(i)}|| < p_i < P$ and $||\hat{b}_j^{(i)}|| \leq \frac{(q-P)/2}{p_i} \leq \frac{(q-P)/2}{P/2} \leq \lfloor \frac{q}{P} \rfloor$ for the honest prover by definition.
Overall, the protocol has perfect completeness.

Soundness. Let $y = g(\boldsymbol{r})$. Assume the prover $\tilde{\mathsf{P}}$ is malicious and claims $g(\boldsymbol{r})$ evaluates to y' for some $y' \neq y = g(\boldsymbol{r})$. Since $\tilde{\mathsf{P}}$ passes the bound check in Step 1, we have $|y'| < M \cdot ||\boldsymbol{r}||^m \cdot \mathsf{T}$ (otherwise the verifier rejects), which we invoke below. Aside from this deterministic check, all remaining verifications of the PIOR happen in Step 3 and 4 of the protocol.

Define V_1 to be the event that the verification check in Step 3 passes, and V_2 the event that the verification check in Step 4 passes, given any malicious prover $\tilde{\mathsf{P}}$. Define E to be the event that Equation (6) holds for every $i \in [s], j \in [\mathsf{t}]$. The soundness error of PIOR 5.4 can be expressed as the following,

$$\begin{aligned}
\Pr[\mathsf{V}_1 \wedge \mathsf{V}_2] &= \Pr[\mathsf{V}_1 \wedge \mathsf{V}_2 \wedge \mathsf{E}] + \Pr[\mathsf{V}_1 \wedge \mathsf{V}_2 \wedge \overline{\mathsf{E}}] \\
&\leq \Pr[\mathsf{V}_1 \wedge \mathsf{E}] + \Pr[\mathsf{V}_2 \wedge \overline{\mathsf{E}}] \\
&= \Pr[\mathsf{V}_1 | \mathsf{E}] \cdot \Pr[\mathsf{E}] + \Pr[\mathsf{V}_2 | \overline{\mathsf{E}}] \cdot \Pr[\overline{\mathsf{E}}] \\
&\leq \Pr[\mathsf{V}_1 | \mathsf{E}] + \Pr[\mathsf{V}_2 | \overline{\mathsf{E}}]
\end{aligned}$$

$\Pr[\mathsf{V}_1|\mathsf{E}]$: This is the probability that the verification check in Step 3 passes, assuming Equation (6) holds. We first unroll the definition of $a_j^{(i)}$ and have that, for every $i \in [s]$,

$$\begin{aligned}
&a_{\mathsf{t}}^{(i)}(\boldsymbol{r}_1^{(i)}, \ldots, \boldsymbol{r}_{m-\mathsf{t}k}^{(i)}) \\
&= g(r_1^{(i)}, \ldots, \boldsymbol{r}_m^{(i)}) - p_i \Big(\sum_{j=1}^{\mathsf{t}} b_j^{(i)}(\boldsymbol{r}_1^{(i)}, \ldots, \boldsymbol{r}_{m-jk}^{(i)}) \Big) \\
&\equiv_{p_i} g(\boldsymbol{r}_1^{(i)}, \ldots, \boldsymbol{r}_m^{(i)}) \equiv_{p_i} y
\end{aligned}$$

We also note that the oracle response does not overflow $\mathbb{F}$ given the norm bounds and parameter assumptions:

$$\left| a_i^{(\mathsf{t})}(\boldsymbol{r}_1^{(i)}, \ldots, \boldsymbol{r}_{m-\mathsf{t}k}^{(i)}) \right| \leq 2^k p_i^k ||\hat{a}_i^{(\mathsf{t})}|| < 2^k p_i^k P < 2^k P^{k+1} < q$$

This implies that $\mathtt{toInt}\Big(\hat{a}_i^{(\mathsf{t})}(\hat{\boldsymbol{r}}_1^{(i)}, \ldots, \hat{\boldsymbol{r}}_{m-\mathsf{t}k}^{(i)})\Big)$ is the same as the integer evaluation $a_i^{(\mathsf{t})}(\boldsymbol{r}_1^{(i)}, \ldots, \boldsymbol{r}_{m-\mathsf{t}k}^{(i)})$. $\tilde{\mathsf{P}}$ will only pass all checks in Step 3 if $\mathtt{toInt}(\hat{y}_i) \equiv_{p_i} y'$ for every $i \in [s]$. Thus, we have that

$$\mathtt{toInt}(\hat{y}_i) \equiv_{p_i} y \equiv_{p_i} y' \quad \forall i \in [s] \quad \Rightarrow \quad y' - y \equiv_{p_i} 0 \quad \forall i \in [s] \quad \Rightarrow \quad \prod_{i=1}^{s} p_i \text{ divides } y' - y$$

Let $S = \{p_1', p_2', \ldots\}$ be the set of all primes in $\mathsf{Primes}(P/2, P)$ that divide $y' - y$. We can bound the size

of S as follows,

$$\prod_{i=1}^{|S|} p'_i \;\Big|\; |y'-y| \implies \prod_{i=1}^{|S|} p'_i \le |y'-y| \qquad \text{given } y' \neq y$$

$$(P/2)^{|S|} \le 2 \cdot 2^m \cdot ||\boldsymbol{r}||^m \cdot \mathsf{T} \qquad \text{given } p'_i \ge P/2 \text{ and } |y|, |y'| < 2^m \cdot ||\boldsymbol{r}||^m \cdot \mathsf{T}$$

$$|S| \le \frac{1 + m + m\log||\boldsymbol{r}|| + \log \mathsf{T}}{\log P - 1}$$

$$|S| \le \frac{1 + m + \lambda m + \lambda}{\log P - 1} \qquad \text{given } ||\boldsymbol{r}|| < 2^\lambda, \mathsf{T} < 2^\lambda$$

$$|S| \le \frac{8\lambda m}{\log P} \qquad \text{given } \log P \ge 2$$

Then by Corollary B.1, since $\log P \ge 5$, there are at least $P/4\log P$ primes in $\mathsf{Primes}(P/2, P)$. Thus, this case happens with the following probability, given $y' \neq y$

$$\Pr[\forall i \in [s] \;\; y' \equiv_{p_i} y] = \Big(\frac{|S|}{|\mathsf{Primes}(P/2,P)|}\Big)^s \le \Big(\frac{8\lambda m/\log P}{P/4\log P}\Big)^s = \Big(\frac{32\lambda m}{P}\Big)^s$$

$\underline{\Pr[\mathsf{V}_2|\overline{\mathsf{E}}]}$: This is the probability that the polynomial identity tests in Step 4 pass while Equation (6), $a_j^{(i)}(\boldsymbol{X}) + p_i \cdot b_j^{(i)}(\boldsymbol{X}) = g_j^{(i)}(\boldsymbol{X})$, does not hold for some $i \in [s]$ and $j \in [\mathsf{t}]$. Because of the reduction to $\mathsf{R}_{\mathsf{BatchRange}(st)}$ in Step 5, the LHS $||a_j^{(i)}|| + p_i||b_j^{(i)}||$ is bounded by $P - 1 + p_i \cdot (\lfloor q/P \rfloor - 1) \le P - 1 + q - P = q - 1 < q$, and the right hand side norm is less than $(q-P)/2 < q$. As both sides are less than q, we must have that $\hat{a}_j^{(i)}(\boldsymbol{X}) + p_i \cdot \hat{b}_j^{(i)}(\boldsymbol{X}) \neq \hat{g}_j^{(i)}(\boldsymbol{X})$.

Since both sides are multilinear polynomials, by the Schwartz-Zippel Lemma (Lemma 4.2), each polynomial identity test for an incorrect identity holds over $\mathbb{F}$ with probability at most $\frac{m-(j-1)k}{|\mathbb{F}|} \le \frac{m}{|\mathbb{F}|}$. By union bound over all i, j, the overall error is upper-bounded by $\frac{\mathsf{t}sm}{|\mathbb{F}|} \le \frac{sm^2}{k|\mathbb{F}|}$.

Putting things together: Thus, the soundness error of the entire PIOR is upper-bounded by $\left(\frac{32\lambda m}{P}\right)^s + \frac{sm^2}{k|\mathbb{F}|}$.

Oracles. Since all the oracles $[[\hat{a}_j^{(i)}]], [[\hat{b}_j^{(i)}]]$ are sent before the verifier selects the random challenges $\hat{\boldsymbol{\gamma}}$, they can be batched into one single oracle using the Jagged PCS construction [HJRRR25].
The total oracle length per random small prime p_i is $\frac{2M}{2^k-1}$. This is because in iteration $j \in [\mathsf{t}]$, the prover sends $\{[[\hat{a}_j^{(i)}]]\}_{i=1}^s$ and $\{[[\hat{b}_j^{(i)}]]\}_{i=1}^s$, which each are 2^{m-jk} length. If we sum over all j, the geometric series sum is $\sum_{j=1}^{\mathsf{t}} 2^{m-jk}$ which is upper-bounded by $\frac{2^m}{2^k-1}$. Overall it is $\frac{2sM}{2^k-1}$.
The oracle size in bits is similarly computed, except we can note that $\hat{a}_j^{(i)}$ and $\hat{b}_j^{(i)}$ together have $\log q$ bits because they are decomposed from a q-bounded polynomial. The total size is therefore $\frac{sM\log q}{2^k-1}$.
Rounds. The protocol can be transformed so that there is only *one round* of interaction, where the verifier sends $[p_i]_{i=1}^s$, and the prover responds with all of its oracles.
Prover Work. For each $i \in [s]$, in the first iteration ($j = 1$), the prover partially evaluates a polynomial with m variables and norm bound T at k points, each of the points bounded by p_i. Partial evaluation at the first point takes $M/2$ multiplications between an element at most $\log P$ bits and an element at most $1 + \log \mathsf{T}$ bits; at the second point takes $M/4$ multiplications between an element at most $\log P$ bits and an element at most $2 + \log P + \log \mathsf{T}$ bits; at the third point takes $M/8$ multiplications between an element at most $\log P$ and an element at most $3 + 2\log P + \log \mathsf{T}$ bits. Let $\mathsf{mult}(a, b)$ denote the number of multiplications

in bits for multiplying a bits and b bits together. The total number of operations in bits in the first iteration is therefore at most

$$s \cdot \sum_{i=1}^{k} \frac{M \cdot \mathsf{mult}(\log P, i + (i-1)\log P + \log \mathsf{T})}{2^i} < s \cdot \sum_{i=1}^{k} \frac{M \cdot i \cdot \mathsf{mult}(\log P, \log P + \log \mathsf{T})}{2^i}$$
$$< 2sM \cdot \mathsf{mult}(\log P, \log P + \log \mathsf{T})$$

In the rest of the iterations $j \in \{2, \ldots, \mathsf{t}\}$, the prover partially evaluates a polynomial with $m - jk$ variables and norm bound P at k points.

$$s \cdot \sum_{j=2}^{\mathsf{t}} \sum_{i=1}^{k} \frac{M \cdot 2^i \cdot \mathsf{mult}(\log P, 2\log P)}{2^{jk} \cdot 2^i} < 2sM \cdot \mathsf{mult}(\log P, \log P) \cdot \sum_{j=2}^{\mathsf{t}} \frac{1}{2^{jk}}$$
$$< \frac{sM \cdot \mathsf{mult}(\log P, \log P)}{2^k}$$

The decomposition steps are only done on partially evaluated polynomials, so their cost is dominated by the evaluations.

Using FFT for multiplications, $\mathsf{mult}(\log P, \log P + \log \mathsf{T})$ can be optimized to $O((\log P + \log \mathsf{T}) \log(\log P + \log \mathsf{T})) = \tilde{O}(\log P + \log \mathsf{T})$ bits. The number of additions needed is fewer than the number of multiplications. Therefore, the total cost in bits is

$$O(sM \cdot \mathsf{mult}(\log P, \log P + \log \mathsf{T})) + \frac{sM}{2^k - 1} \log q < sM\Big(\tilde{O}(\log P + \log \mathsf{T}) + \frac{\log q}{2^k - 1}\Big)$$

Verifier Work. The verifier needs to do $s\mathsf{t} \leq \frac{sm}{k}$ $\mathbb{F}$ operations to check the polynomial identity tests. The verifier also needs to do s operations with integers of size $m\lambda + m + \log \mathsf{T}$. Overall the verifier does $O(sm\lambda)$ bit work.

Oracle Queries. In Step 3, the verifier queries the batched long oracle once for every $i \in [s]$; in Step 4, the verifier queries it a single time. □

Lemma 5.7. *Under the assumptions of Theorem 5.5, let* PCS *be a standard polynomial commitment scheme whose* Eval *is an argument of knowledge with knowledge error* ϵ_{PCS}*, and let the batch range check argument have soundness error* ϵ_{rc}*. Combining PIOR 5.4 for* $\mathsf{R}_{\mathsf{IntEval}}(p)$ *with a batch range check and compiling the resulting PIOP with* PCS *(Theorem 4.20) yields a public-coin interactive argument of knowledge for* $\mathsf{R}_{\mathsf{IntEval}}(p)$ *with knowledge error*

$$\epsilon_{\mathsf{IntEvalArg}} = \epsilon_{\mathsf{IntEval}} + 2\epsilon_{\mathsf{rc}} + \mathsf{negl}(\lambda).$$

Proof. By Theorem 5.5, PIOR 5.4 has soundness error $\epsilon_{\mathsf{IntEval}}$, and two batch range checks it reduces to have soundness error $2\epsilon_{\mathsf{rc}}$. By Lemma 4.21, the PIOP has knowledge soundness error $\epsilon_{\mathsf{IntEval}} + 2\epsilon_{\mathsf{rc}}$. Compiling with PCS via Theorem 4.20 adds $\mathsf{negl}(\lambda)$ to the knowledge error. Summing the three contributions yields $\epsilon_{\mathsf{IntEvalArg}}$. □

5.4 Setting the Parameters

In the above theorem, we made several assumptions about the parameters for correctness, soundness, and efficiency purposes. Here, we summarize those assumptions and show reasonable fine-grained asymptotic and concrete settings of these parameters that satisfy all the assumptions and bounds.

Bounds to consider. We start by listing the objectives when setting k, P, s:

- The prime characteristic q of $\mathbb{F}$ should be large enough so that no oracle query response for the final evaluation step overflows $\mathbb{F}$ and therefore q.

$$2^k P^{k+1} < q \qquad \text{(Final Evaluation Bound)}$$

- The evaluations of each partially evaluated polynomial $||g_j^{(i)}||$ should be small enough to avoid overflow, while also maintaining perfect completeness.

$$2^k \cdot P^k \cdot \max(\mathsf{T}, P) \leq (q - P)/2 \qquad \text{(Partial Evaluation Norm Bound)}$$

- The soundness error of the PIOR should be negligible in λ. Particularly, we need

$$\Big(\frac{|S|}{|\mathsf{Primes}(P/2, P)|}\Big)^s \leq \Big(\frac{32\lambda m}{P}\Big)^s \leq \mathsf{negl}(\lambda) \qquad \text{(Soundness Bound 1)}$$

$$\frac{sm^2}{k|\mathbb{F}|} \leq \mathsf{negl}(\lambda) \qquad \text{(Soundness Bound 2)}$$

- The total size of the oracles (in bits) should be small and ideally sub-linear, compared to the size of $[[\hat{g}]]$.

$$2^{m-k} s \log q = o(2^m \log \mathsf{T}) \qquad \text{(Oracle Size Bound)}$$

How to satisfy the bounds intuitively. We start by setting $k := \log \lambda$ because of the Oracle Size Bound, where k needs to be large enough to ensure that we only commit to a sublinear amount of bits compared to the original polynomial. Then, assuming $q \approx 2^{2\lambda}$, in order to satisfy Final Evaluation Bound, we need

$$2^k P^{k+1} < q \implies \log P < \frac{2\lambda - \log \lambda}{\log \lambda + 1}$$

so we can set $\log P = \frac{\lambda}{\log \lambda}$. Partial Evaluation Norm Bound is of a similar form and is similarly satisfied. To set s, we substitute $\log P = \frac{\lambda}{\log \lambda}$ into Soundness Bound 1,

$$\Big(\frac{32\lambda m}{2^{\lambda/\log \lambda}}\Big)^s$$

Setting $s := \log \lambda$ cleanly cancels out the denominator in the exponent and makes the expression negligible in λ.

Now, we more formally show that these settings satisfy the bounds in the following corollary and proof.

Corollary 5.8. *If the PIOR 5.4 is instantiated with the following:*
- *a field $\mathbb{F}$ with characteristic q that is 2λ bits*
- *$\boldsymbol{r} \in \mathbb{Z}_p$ for some λ-bit p*
- *$(\lambda/2)$-bit norm bound T*
- *the number of partially evaluated variables $k = \log \lambda$*
- *the log upper bound of the random primes $\log P = \frac{\lambda}{\log \lambda}$*
- *and the total number of small, random primes $s = \log \lambda$,*

then the PIOR has perfect completeness, asymptotically $\mathsf{negl}(\lambda)$ soundness error and the following properties.

# Rounds	1
# Oracles, Length, Size (in total bits)	1, $\frac{2M\log\lambda}{\lambda-1}$, $\frac{M\log\lambda\log q}{\lambda-1}$
Prover Work	$O(M\lambda\log^2\lambda)$ bit operations
Proof Size	0
Verifier Work	$O(m\lambda\log\lambda)$ bit operations
Oracle Queries	$\log\lambda+1$

Proof. For sufficiently large λ, all of the following hold: For the Final Evaluation Bound, taking $\log$ of both sides and assuming $\log\lambda > 2$, we have that

$$\log(2^k P^{k+1}) = k + (k+1)\log P = \log\lambda + (\log\lambda+1)\frac{\lambda}{\log\lambda} < \log\lambda + 1.5\lambda < 2\lambda = \log q$$

For the Partial Evaluation Norm Bound,

$$2^k \cdot P^k \cdot \max(\mathsf{T}, P) = 2^{k+k\log P+\log\mathsf{T}} \le 2^{\log\lambda+1.5\lambda} \le (2^{2\lambda} - 2^{\lambda/\log\lambda})/2 = (q-P)/2$$

Moreover, this bound completely supersedes the Final Evaluation Bound.
For the Soundness Bound 1,

$$\Big(\frac{32\lambda m}{P}\Big)^s = \Big(\frac{32\lambda m}{2^{\lambda/\log\lambda}}\Big)^{\log\lambda} < \frac{\lambda^5(\lambda m)^{\log\lambda}}{2^\lambda} < \frac{\lambda^{5+2\log\lambda}}{2^\lambda} = \mathsf{negl}(\lambda)$$

For the Soundness Bound 2,

$$\frac{sm^2}{k|\mathbb{F}|} = \frac{m^2}{2^{2\lambda}} \le \frac{\lambda^2}{2^{2\lambda}} = \mathsf{negl}(\lambda)$$

Lastly, for the Oracle Size Bound, if we take the left side divided by the right side, we have,

$$\frac{2^{m-k}s\log q}{2^m\log\mathsf{T}} = \frac{s\log q}{2^k\log\mathsf{T}} < \frac{\log\lambda\cdot 2\lambda}{\lambda\cdot\lambda/2} = \frac{4\log\lambda}{\lambda} = o(1)$$

The properties in the table result from directly substituting the parameter settings in the properties of Theorem 5.5. □

Concrete Parameter Settings. We provide some concrete parameter values given reasonable selections of q, T, p, M in Table 3. In general, we assume q is 256 bits, the limb bound is $\log\mathsf{T} = 32$ bits, p and λ are 128 bits, and m is 25. We give parameters for modifications of these settings as well and also provide examples of settings that minimize extra commitment cost (larger k) or use small fields (e.g. $\log q = 64$).

Remark 5.9. As shown in Table 3, our protocol supports field characteristic q that is much smaller than 2λ bits, at the cost of increased prover commitment cost. The lower bound for q comes from the Partial Evaluation Norm Bound. Setting $k = 1$, $P = 64\lambda m$ (This assignment of P is because we need $P > 32\lambda m$ for Soundness Bound 1), and using $\max(\mathsf{T}, P) \ge P$, the lower bound becomes

$$q \ge 2^{k+1}P^{k+1} = 4P^2 = \Omega(\lambda^2 m^2),$$

which is significantly smaller than $2^{2\lambda}$. This minimal characteristic is achieved with a small limb bound $\mathsf{T} = O(\lambda m)$; a larger limb bound raises the requirement to $q = \Theta(\lambda m\max(\mathsf{T}, \lambda m))$. However, we note that due to the Soundness Bound 2, the order of the extension field must be $|\mathbb{F}| \ge 2^\lambda$ for the PIOR to have negligible soundness error.

Inputs				Choice	Derived		
λ	$\log q$	$\log \mathsf{T}$	m	k	$\log P$	s	Comm. Overhead
128	256	32	25	6	35	6	0.762
128	256	32	25	8	27	10	0.314
128	256	32	25	13	16	34	0.033
128	256	32	30	8	26	10	0.314
128	256	64	25	7	26	10	0.315
128	64	32	25	1	30	8	16

Table 3: The script used to compute these can be found in Appendix A. λ is the security parameter, q is the characteristic of the field $\mathbb{F}$ in the PCS we use to compile the PIOR, T is the bound on the evaluations of the committed polynomial g, m is the number of variables in the polynomial, k is the number of partially evaluated variables in each round, the random primes are selected from $\mathsf{Primes}(P/2, P)$, and s is the number of random primes needed for the soundness of the protocol to be below $2^{-\lambda}$. The commitment overhead is the amount of extra bits committed to relative to the original witness commitment size.

5.5 Overall Integer Mod-PCS Evaluation Protocol

Theorem 5.1 (restated). *Given any correct standard polynomial commitment scheme* $\mathsf{PCS} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Eval})$ *where* $\mathsf{PCS.Eval}$ *has knowledge error* ϵ_{PCS}*, and any perfectly complete batch range check argument with soundness error* ϵ_{rc}*, there exists an integer mod-PCS* modPCS *that is correct and whose* $\mathsf{modPCS.Eval}$ *(Protocol 5.3) is a public-coin interactive argument of knowledge for the integer mod-PCS evaluation relation* $\mathcal{R}_{\mathsf{modeval}}$ *(Definition 4.12) with knowledge error* $\epsilon_{\mathsf{PCS}} + \epsilon_{\mathsf{rc}} + \frac{2l}{p} + \epsilon_{\mathsf{IntEvalArg}}$*.With parameters set according to Corollary 5.8,* $\mathsf{modPCS.Eval}$ *has the following performance properties (excluding the cost of the batch range check arguments)*:

# Rounds	$O(\log\log \mathsf{T}_f)$
# Commitments, Length, Size (in total bits)	1, $\frac{N \log \mathsf{T}_f \log \lambda}{\lambda(\lambda-1)} = o(N)$, $\frac{N \log \mathsf{T}_f \log \lambda \log q}{\lambda(\lambda-1)} = o(N \log q)$
Prover Work	$O(N \log \mathsf{T}_f \log^2 \lambda)$
Proof Size excluding evaluation proofs	$O((n + \log\log \mathsf{T}_f)\lambda)$
Verifier Work	$O((n + \log\log \mathsf{T}_f)\lambda \log \lambda)$
# Evaluation Proofs	$\log \lambda + 1$

Proof of Theorem 5.1. **Correctness.** For an honest committer holding a polynomial f with $||f|| < \mathsf{T}_f$ (i.e. a valid witness for $\mathcal{R}_{\mathsf{modeval}}$), limb-splitting f into L base-T limbs is lossless, since $\mathsf{T}_f = \mathsf{T}^L$, and yields a T-bounded $\hat{f}_\ell$. Hence $\mathsf{modPCS.Commit}$ never rejects (the range check is perfectly complete) and outputs com_f; and $\mathsf{modPCS.Eval}$ always accepts honest evaluations, since both the sumcheck argument and the IntEval argument have perfect completeness.

Knowledge Soundness: Since PCS is knowledge-sound, the extractor Ext can extract the committed limb-split polynomial $\hat{f}_\ell$ from the commitment com_f and the evaluation points with error ϵ_{PCS}. By the soundness of the batch range check in $\mathsf{modPCS.Commit}$, we know that $||\hat{f}_\ell|| < \mathsf{T}$, except with error ϵ_{rc}. The extractor can then compute the integer polynomial f by casting the limbs to the integers and recombining them; the result satisfies $||f|| < \mathsf{T}_f$ since $\mathsf{T}_f = \mathsf{T}^L$, so f is a valid witness for the bounded relation $\mathcal{R}_{\mathsf{modeval}}$.

Now, suppose malicious $\tilde{\mathsf{P}}$ attempts to cheat with a false evaluation claim $f(\boldsymbol{r}_f) \not\equiv_p v$. Since $f(\boldsymbol{r}_f) = \sum_{\boldsymbol{k}\in\{0,1\}^l} \ell(\boldsymbol{k}) \cdot f_\ell(\boldsymbol{r}_f, \boldsymbol{k}) \not\equiv_p v$, the claim fed into the sumcheck is false. This leaves $\tilde{\mathsf{P}}$ two avenues to cheat. First, the prover can reduce this false input claim to a true evaluation claim through sumcheck. The soundness error of the sumcheck argument with a degree-2 polynomial with l variables is $\frac{2l}{p}$. Second, if the reduced claim from sumcheck is false, i.e. $f_\ell(\boldsymbol{r}_f, \boldsymbol{r}_\ell) \not\equiv_p y'$, the integer claim $y = f_\ell(\boldsymbol{r}_f, \boldsymbol{r}_\ell)$ is also false. From Lemma 5.7, the soundness from the IntEval argument is $\epsilon_{\mathsf{IntEvalArg}}$.

Therefore, the overall knowledge soundness error is $\epsilon_{\mathsf{PCS}} + \epsilon_{\mathsf{rc}} + \frac{2l}{p} + \epsilon_{\mathsf{IntEvalArg}}$.

Rounds: The sumcheck argument has $l = \log\left(\frac{\log \mathsf{T}_f}{\log \mathsf{T}}\right) = \log\left(\frac{2\log \mathsf{T}_f}{\lambda}\right) = 1 + \log\log \mathsf{T}_f - \log\lambda = O(\log\log \mathsf{T}_f)$ number of rounds, since $\log \mathsf{T}_f$ is assumed to be negligible in 2^λ, and the IntEval argument has 1 round in addition to the rounds needed for batch range checks.

Commitment cost: Number, size, and length of commitments follow directly from those of the oracles in IntEval PIOR, excluding the commitments needed for the batch range checks. As shown in Corollary 5.8, the cost of committing to these additional polynomials is sublinear in the cost of committing to the witness.

Prover Work: The prover performs $6l$ multiplications in $\mathbb{Z}_p$ for the sumcheck argument, which is $O(\lambda\log\lambda)$ bit operations; and the prover performs $N\left(\frac{\log \mathsf{T}_f}{\log \mathsf{T}}\right) \cdot O(\log \mathsf{T}\log\log \mathsf{T}) = O(N\log \mathsf{T}_f \log^2\lambda)$ bit operations for the IntEval argument. Thus, the total prover work is $O(N\lambda\log^2\lambda)$ bits, and the cost of the batch range checks.

Proof Size: In each round of the sumcheck, the prover sends a degree 2 univariate integer polynomial, which is represented by 3 evaluation points in $\mathbb{Z}_p$. The total bit-length of these prover messages is $O(\lambda\log\log \mathsf{T}_f)$. The prover also sends one integer element, the claimed integer evaluation, at the end of the sumcheck, which is at most

$$2^{n+l} \cdot \mathsf{T} \cdot p^{n+l} \leq 2^{(n+l)(\lambda+1)+\lambda/2}$$

Therefore, the total proof size in bits is $O((n + \log\log \mathsf{T}_f)\lambda)$, dominated by this integer element.

Verifier Work: The verifier performs a constant number of $\mathbb{Z}_p$ operations per round of sumcheck, so the total verifier work for the sumcheck argument is $O(l\lambda) = O(\lambda\log\log \mathsf{T}_f)$ in bits. After the sumcheck, the verifier does 1 operation with an integer of size $(n+l)(\lambda+1)+\lambda/2 = O((n+\log\log \mathsf{T}_f)\lambda)$ in bits. Moreover, the verifier performs $O((n+l)\lambda\log\lambda) = O((n+\log\log \mathsf{T}_f)\lambda\log\lambda)$ bit operations for the IntEval argument.

Evaluation Proofs: Again, since all commitments are done in the IntEval argument, the number of evaluation proofs follows directly from the oracle queries of IntEval PIOR. □

Instantiations with different PCSs. Given any correct standard PCS whose Eval has knowledge error ϵ_{PCS} and any perfectly complete batch range check argument with soundness error ϵ_{rc}, the compiled modPCS.Eval with parameters set according to Corollary 5.8 is a public-coin interactive argument of knowledge with knowledge error $\epsilon_{\mathsf{PCS}} + \epsilon_{\mathsf{rc}} + \frac{2l}{p} + \epsilon_{\mathsf{IntEvalArg}}$ (Theorem 5.1). In particular, if the underlying PCS has a linear prover time and a logarithmic or constant verifier, the protocol's asymptotics are unchanged.

Remark 5.10. The prover and the verifier run a batch range check PIOP to prove that $||a_j^{(i)}|| < P, ||b_j^{(i)}|| < \lfloor\frac{q}{P}\rfloor$ for every $i \in [s], j \in [\mathsf{t}]$. This batch range check consists of two ranges, one of $\frac{\lambda}{\log\lambda}$ bits and the other of $2\lambda - \frac{\lambda}{\log\lambda}$ bits, and one batch oracle of $2s\mathsf{t}$ multilinear polynomials each with $k := \log\lambda$ variables. These ranges can be larger than range check protocols typically handle (i.e. larger than 2^{16}), and may require re-committing to a decomposed representation of these values, but again, the additional commitment cost is minimized because of IntEval's low commitment overhead property with the right parameters. We note that

with a LogUp-GKR batch range check [PH23], the only additional committed data outside the witness is a table-sized multiplicity polynomial, which is amortized across the batch and relatively small compared to the witness.

Remark 5.11. Negative numbers come up in various computations, so we need to be able to handle these in our integer mod-PCS. Since we have a bound on the computation, T_f, we can handle negative numbers by adding T_f to the polynomial f to get $g(\boldsymbol{X}) = f(\boldsymbol{X}) + \mathsf{T}_f$. g is a positive polynomial that is bounded by $2\mathsf{T}_f$. We can run the protocol on g without changes except for the protocol evaluation, where we can subtract T_f from the claimed evaluation $g(\boldsymbol{r})$ to get the actual evaluation of $f(\boldsymbol{r})$.

6 Spartan-Style PIOP for Integer Mod-R1CS

In this section, we present a sumcheck-based PIOP based on Spartan [Set20] for integer arithmetic that directly enables efficient non-native arithmetic. We first present an arithmetization that allows for a natural encoding of a non-native arithmetic circuit. Then, to deal with the norm bound limitation, we describe a method to efficiently handle witness limb decomposition by embedding the recomposition logic directly into the sumcheck formulation. Our construction is based on Spartan [Set20] but can generally be adapted for any sumcheck-based PIOP for circuit satisfiability including HyperPlonk [CBBZ23]. We also note that we can extend our techniques to different arithmetizations beyond R1CS, such as CCS [STW23] and Plonk-ish [GWC19] arithmetizations.

Encoding Sparse Polynomials. We adopt the SPARK-style encoding for sparse polynomials [Set20] in the following definitions for moduli-R1CS. That is, the sparse polynomial $H \in \{A, B, C\}$ is represented by $S = 2^s$ tuples $(\boldsymbol{i}, \boldsymbol{j}, H(\boldsymbol{i}, \boldsymbol{j}))$, and we encode it using $\mathsf{row}_H, \mathsf{col}_H, \mathsf{val}_H \in \mathbb{F}^{\leq 1}[s]$ that encode the S tuples in some canonical order, such that for $\boldsymbol{k} \in \{0,1\}^s$, $\mathsf{row}_H(\boldsymbol{k}) = \boldsymbol{i}, \mathsf{col}_H(\boldsymbol{k}) = \boldsymbol{j}, \mathsf{val}_H(\boldsymbol{k}) = H(\boldsymbol{i}, \boldsymbol{j})$. For simplicity, we do not explicitly write these encodings until Definition 6.6.

Notation. In the following discussions, let $N = N_x + N_w, n_x = \log N_x, n_w = \log N_w$ and $n = \lceil \log N \rceil$.

6.1 Fingerprinting PIOR

First, we propose the following arithmetization, Integer mod-R1CS. By adding mods and quos to the standard R1CS equation and using integers rather than field elements, we can directly represent arbitrary modular arithmetic.

Definition 6.1. *The indexed relation* $\mathsf{R}_{\mathsf{imodR1CS}}$ *is the set of all tuples*

$$(\mathbb{i}; \mathbb{x}; \mathbb{w}) = ((A, B, C, \mathsf{mods}); \boldsymbol{x}; (\boldsymbol{w}, \mathsf{quos}))$$

where $A, B, C \in \mathbb{Z}^{N \times N}, \mathsf{mods} \in \mathbb{Z}^N, \boldsymbol{x} \in \mathbb{Z}^{N_x}, \boldsymbol{w} \in \mathbb{Z}^{N_w}, \mathsf{quos} \in \mathbb{Z}^N$*, such that* $\boldsymbol{z} \in \mathbb{Z}^N := (\boldsymbol{x}, \boldsymbol{w})$ *and the following equality holds,*

$$A\boldsymbol{z} \circ B\boldsymbol{z} = C\boldsymbol{z} + \mathsf{mods} \circ \mathsf{quos} \tag{1}$$

If we view the matrices and vectors as polynomials,applying the fingerprinting technique reduces the indexed relation above, with matrices and vectors over $\mathbb{Z}$, to the indexed oracle relation below, with multilinear polynomials over $\mathbb{Z}_p$.

Definition 6.2. *The indexed oracle relation* $\mathsf{R}_{\mathsf{imodR1CS}}(p)$ *is the set of all tuples*

$$(\mathbb{i}; \mathbb{x}; \mathbb{w}) = ((A, B, C, \mathsf{mods}); (\boldsymbol{x}, p); ([[\boldsymbol{w}]], [[\mathsf{quos}]]); (\boldsymbol{w}, \mathsf{quos}))$$

where $A, B, C \in \mathbb{Z}^{\leq 1}[2n]$, $\mathsf{mods} \in \mathbb{Z}^{\leq 1}[n]$, $x \in \mathbb{Z}^{\leq 1}[n_x]$, $w \in \mathbb{Z}^{\leq 1}[n_w]$, $\mathsf{quos} \in \mathbb{Z}^{\leq 1}[n]$, *all the oracles are mod-oracles; such that* $z \in \mathbb{Z}^{\leq 1}[n] := (x, w)$, *and the following equality holds for every* $\boldsymbol{i} \in \{0,1\}^n$,

$$\Big(\sum_{\boldsymbol{j}\in\{0,1\}^n} A(\boldsymbol{i},\boldsymbol{j})z(\boldsymbol{j})\Big)\Big(\sum_{\boldsymbol{j}\in\{0,1\}^n} B(\boldsymbol{i},\boldsymbol{j})z(\boldsymbol{j})\Big) \equiv_p \sum_{\boldsymbol{j}\in\{0,1\}^n} C(\boldsymbol{i},\boldsymbol{j})z(\boldsymbol{j}) + \mathsf{mods}(\boldsymbol{i})\mathsf{quos}(\boldsymbol{i}) \tag{3}$$

In Protocol 6.1, we present the fingerprinting protocol that reduces proving $\mathsf{R}_{\mathsf{imodR1CS}}$ to $\mathsf{R}_{\mathsf{imodR1CS}}(p)$.

Protocol 6.1: $\mathsf{R}_{\mathsf{imodR1CS}} \to \mathsf{R}_{\mathsf{imodR1CS}}(p)$

Parse: $\mathbb{i} = (A, B, C, \mathsf{mods}); \mathbb{x} = \boldsymbol{x}; \mathbb{w} = (\boldsymbol{w}, \mathsf{quos})$

0. Indexer defines $A, B, C \in \mathbb{Z}^{\leq 1}[2n]$ and $\mathsf{mods} \in \mathbb{Z}^{\leq 1}[n]$ to be the multilinear polynomials with each corresponding item in the index being the evaluations, respectively.
1. Prover defines $w \in \mathbb{Z}^{\leq 1}[n_w]$, $\mathsf{quos} \in \mathbb{Z}^{\leq 1}[n]$ to be the multilinear polynomials with $\boldsymbol{w}$, quos being the evaluations, respectively. Prover sends mod-oracles of the witnesses $[[w]], [[\mathsf{quos}]]$ to the verifier.
2. Verifier selects a random λ-bit prime p and sends it to the prover. Verifier defines x to be the multilinear polynomial with $\boldsymbol{x}$ being the evaluations.
3. Output the following for $\mathsf{R}_{\mathsf{imodR1CS}}(p)$.
$$\mathbb{i}' := (A, B, C, \mathsf{mods}), \quad \mathbb{x}' := (x, p), \mathbb{o}' := ([[w]], [[\mathsf{quos}]]), \quad \mathbb{w}' := (w, \mathsf{quos})$$

Lemma 6.3. *Let* T *be the norm bound of* A, B, C, T_z *that of* $\boldsymbol{z}$, $\mathsf{T}_{\mathsf{mods}}$ *that of* mods, *and* $\mathsf{T}_{\mathsf{quos}}$ *that of* quos. *Assume all the bounds are* $O(\mathsf{T})$. *For any* $\lambda \geq 9$, *the reduction in Protocol 6.1 has perfect completeness, soundness error* $\frac{8n+O(\log \mathsf{T})}{2^\lambda}$, *and the following properties.*

# Rounds	1
Mod-Oracle #, Length, Size	2, $2^{n_w} + 2^n$, $2^{n_w} \log \mathsf{T}_z + 2^n \log \mathsf{T}_{\mathsf{quos}}$
Prover Work	0
Proof Size	0
Verifier Work	0
Oracle Queries	0

Proof.

Completeness: If the instance $\mathbb{x}$ and witness $\mathbb{w}$ satisfy $\mathsf{R}_{\mathsf{imodR1CS}}$, then Equation (1) implies Equation (3) for any modulus p.

Soundness: Soundness error of Protocol 6.1 is the following probability,

$$\Pr_{p \xleftarrow{\$} \mathsf{Primes}(2^{\lambda-1}, 2^\lambda)} [(\mathbb{i}'; \mathbb{x}'; \mathbb{o}'; \mathbb{w}') \in \mathsf{R}_{\mathsf{imodR1CS}}(p) | (\mathbb{i}; \mathbb{x}; \mathbb{w}) \notin \mathsf{R}_{\mathsf{imodR1CS}}]$$

Observe that each relation requires an equality (Equation (1) for $\mathsf{R}_{\mathsf{imodR1CS}}$ and Equation (3) for $\mathsf{R}_{\mathsf{imodR1CS}}(p)$) to hold, which implies the above probability is equivalent to,

$$\Pr_{p \xleftarrow{\$} \mathsf{Primes}(2^{\lambda-1}, 2^\lambda)} [\text{ Equation (3) holds } \forall \boldsymbol{i} \in \{0,1\}^n | \text{ Equation (1) does not hold}]$$

Define E_i for $i \in [N]$ as the event that the ith integer mod constraint does not hold, namely the following

$$\mathsf{E}_i := [A_i \boldsymbol{z} \circ B_i \boldsymbol{z} - C_i \boldsymbol{z} - \mathsf{mods}_i \circ \mathsf{quos}_i \neq 0]$$

where the subscript i denotes the ith row of the matrix/vector. If Equation (1) does not hold, then there must exist $i \in [N]$ such that E_i occurs. The left-hand side is bounded by $(N \cdot \mathsf{T} \cdot \mathsf{T}_z)^2 + (N \cdot \mathsf{T} \cdot \mathsf{T}_z) + \mathsf{T}_{\mathsf{mods}} \cdot \mathsf{T}_{\mathsf{quos}}$. By Lemma 4.29, for any claimed witness oracles $[[\hat{w}]], [[\hat{\mathsf{quos}}]]$ sent by the malicious prover, the soundness error is upper-bounded by

$$\begin{aligned}
&\Pr[\text{ Equation (3) holds } \forall \boldsymbol{i} \in \{0,1\}^n \mid \mathsf{E}_i \ \exists i \in [N]] \\
&\leq \frac{\log((N \cdot \mathsf{T} \cdot \mathsf{T}_z)^2 + (N \cdot \mathsf{T} \cdot \mathsf{T}_z) + \mathsf{T}_{\mathsf{mods}} \cdot \mathsf{T}_{\mathsf{quos}})}{2^{\lambda-2}} \\
&\leq \frac{\log(2(N \cdot \mathsf{T} \cdot \mathsf{T}_z)^2 + \mathsf{T}_{\mathsf{mods}} \cdot \mathsf{T}_{\mathsf{quos}})}{2^{\lambda-2}} \\
&\leq \frac{1 + 2\log N + 2\log \mathsf{T} + 2\log \mathsf{T}_z + \log \mathsf{T}_{\mathsf{mods}} + \log \mathsf{T}_{\mathsf{quos}}}{2^{\lambda-2}} \\
&\leq \frac{2\log N + \mathsf{negl}(2^\lambda)}{2^{\lambda-2}} \\
&\leq \frac{8n + \mathsf{negl}(2^\lambda)}{2^\lambda}
\end{aligned}$$

□

6.2 Spartan-Style PIOR for Integer Mod-R1CS Mod p

Overview. PIOR 6.2 is a mod-PIOR, as its oracle instance contains a collection of mod-oracles. The goal of it is to prove the following

$$((A, B, C, \mathsf{mods}), (x, p), ([[w]], [[\mathsf{quos}]]), (w, \mathsf{quos})) \in \mathsf{R}_{\mathsf{imodR1CS}}(p)$$

It follows the style of Spartan [Set20], which runs a series of sumchecks to prove that the multilinear extension of Equation (3) sums up to 0 over the Boolean hypercube.
After the sumchecks, the verifier queries mod-oracles for z, quos and $\mathsf{R}_{\mathsf{SparseEval}}$ for A, B, C to check the $\mathsf{mod}\ p$ evaluations of the polynomials.

PIOR 6.2: $\mathsf{R}_{\mathsf{imodR1CS}} \to \mathsf{R}^3_{\mathsf{SparseEval}}$.

Parse: $\mathbb{i} = (A, B, C, \mathsf{mods})$; $\mathbb{x} = (x, p)$; $\mathbb{o} = ([[w]], [[\mathsf{quos}]])$; $\mathbb{w} = (w, \mathsf{quos})$

0. The indexer gives mods to the verifier as explicit input, and $[[A]], [[B]], [[C]]$ to the verifier as implicit input.

1. Verifier selects a random point $\boldsymbol{\alpha} \in \mathbb{Z}_p^n$ and sends it to prover. Define $z := (w, \mathsf{quos})$.

2. For $h \in \{a, b, c\}$ and corresponding $H \in \{A, B, C\}$, define virtual multilinear polynomials
$$h(\boldsymbol{i}) := \sum_{\boldsymbol{j} \in \{0,1\}^n} H(\boldsymbol{i}, \boldsymbol{j}) z(\boldsymbol{j}) \ \ \mathsf{mod}\ p$$

Prover and verifier engage in a sumcheck PIOP to prove the following

$$\sum_{\boldsymbol{i}\in\{0,1\}^n} \mathsf{eq}(\boldsymbol{i},\boldsymbol{\alpha})\cdot\Big(a(\boldsymbol{i})\cdot b(\boldsymbol{i}) - c(\boldsymbol{i}) - \mathsf{mods}(\boldsymbol{i})\cdot\mathsf{quos}(\boldsymbol{i})\Big) \equiv_p 0 \tag{7}$$

At the end of this sumcheck, prover sends over the claimed evaluations of $a, b, c, \mathsf{mods}, \mathsf{quos}$ at a random point $\boldsymbol{r}_i \in \mathbb{Z}_p^n$; let $v_a, v_b, v_c, v_{\mathsf{mods}}, v_{\mathsf{quos}}$ be these claimed $\mathbb{Z}_p$ evaluations, respectively.
Verifier directly checks whether $\mathsf{mods}(\boldsymbol{r}_i) \equiv_p v_{\mathsf{mods}}$, and queries $[[\mathsf{quos}]]$ to check whether $\mathsf{quos}(\boldsymbol{r}_i) \equiv_p v_{\mathsf{quos}}$.

3. For $h \in \{a, b, c\}$ and corresponding $H \in \{A, B, C\}$, define virtual multilinear polynomials
$$h_{\boldsymbol{r}_i}(\boldsymbol{j}) := H(\boldsymbol{r}_i, \boldsymbol{j})z(\boldsymbol{j}) \mod p$$
To prove the evaluations v_a, v_b, v_c, verifier sends a random point $\rho \in \mathbb{Z}_p$ and sends it to prover. Define $v := v_a + \rho \cdot v_b + \rho^2 \cdot v_c$. They engage in a sumcheck PIOP to prove the following.
$$\sum_{\boldsymbol{j}\in\{0,1\}^n} a_{\boldsymbol{r}_i}(\boldsymbol{j}) + \rho \cdot b_{\boldsymbol{r}_i}(\boldsymbol{j}) + \rho^2 \cdot c_{\boldsymbol{r}_i}(\boldsymbol{j}) \equiv_p v$$
At the end of these sumchecks, verifier needs to check the following claimed evaluations
$$A(\boldsymbol{r}_i, \boldsymbol{r}_j) \equiv_p v_A, \quad B(\boldsymbol{r}_i, \boldsymbol{r}_j) \equiv_p v_B, \quad C(\boldsymbol{r}_i, \boldsymbol{r}_j) \equiv_p v_C, \quad z(\boldsymbol{r}_j) \equiv_p v_z$$
where $\boldsymbol{r}_i \in \mathbb{Z}_p^n, \boldsymbol{r}_j \in \mathbb{Z}_p^n$ are random challenges. Verifier queries $[[w]]$ to check whether $z(\boldsymbol{r}_j) \equiv_p v_z$.

4. For $H \in \{A, B, C\}$ and the corresponding evaluation claim v_H, output the following for $\mathsf{R}_{\mathsf{SparseEval}}$,
$$\mathbb{x}_{\mathsf{SparseEval}} := (p, v_H, \boldsymbol{r}_i, \boldsymbol{r}_j); \quad \mathbb{o}_{\mathsf{SparseEval}} := ([[\mathsf{col}_H]], [[\mathsf{row}_H]], [[\mathsf{val}_H]]);$$
$$\mathbb{w}_{\mathsf{SparseEval}} := (\mathsf{col}_H, \mathsf{row}_H, \mathsf{val}_H)$$

Lemma 6.4. *PIOR 6.2 has perfect completeness, $\frac{16n}{2^\lambda}$ soundness error, and the following properties.*

# Rounds	$2n$
Oracle #, Length, Size	0
Prover Work	$O(N\lambda)$
Proof Size	$(7n+8)\lambda$
Verifier Work	$O(n\lambda)$
Mod-Oracle Queries	2

Proof.
Completeness. Based on the definitions in Step 1 and Step 2, the honest prover is equivalently proving $\forall \boldsymbol{i} \in \{0,1\}^n$,

$$a(\boldsymbol{i})\cdot b(\boldsymbol{i}) - c(\boldsymbol{i}) - \mathsf{mods}(\boldsymbol{i})\cdot\mathsf{quos}(\boldsymbol{i}) \equiv_p 0 \tag{8}$$

In Step 2, the sumcheck holds because over each point in the hypercube, the value of the expression is 0 as defined so the total sum will be 0 regardless of the choice of $\boldsymbol{\alpha}$. This step is perfectly complete by perfect completeness of the sumcheck protocol.
In Step 3, by definition, $v_h \equiv_p h(\boldsymbol{r}_i) \equiv_p \sum_{\boldsymbol{j}\in\{0,1\}^n} H(\boldsymbol{r}_i, \boldsymbol{j})z(\boldsymbol{j})$. A random linear combination does not affect the validity of the sums, so this check is also perfectly complete by perfect completeness of sumcheck.
Soundness. Assume the prover $\tilde{\mathsf{P}}$ is malicious and $(\mathbb{i}; \mathbb{x}; \mathbb{o}; \mathbb{w}) \notin \mathsf{R}_{\mathsf{imodR1CS}}(p)$, which implies Equation (3) does not hold.
Observe that all verifications happen in Step 2 and 3. Define $\mathsf{V}_2, \mathsf{V}_3$ to be the event that $\tilde{\mathsf{P}}$ passes the verification check in Step 2 and Step 3, respectively. The soundness error of PIOR 6.2 can be expressed as

following,

$$\Pr[\mathsf{V}_2 \wedge \mathsf{V}_3] \leq \Pr[\mathsf{V}_2 \vee \mathsf{V}_3] \leq \Pr[\mathsf{V}_2] + \Pr[\mathsf{V}_3]$$

$\Pr[\mathsf{V}_2]$: This is the probability that the multilinear virtual polynomial with n variables at the left-hand side of Equation (7) is not identically zero, but the zerocheck in Step 2 passes. By Schwartz-Zippel Lemma (Lemma 4.2), Equation (7) holds with probability at most $\frac{n}{p}$. If the equation holds, then the sumcheck is guaranteed to pass; otherwise, the sumcheck, which is over a degree-3 virtual polynomial with n variables, passes with probability at most $\frac{3n}{p}$ by Theorem 4.27. The total probability is at most $\frac{4n}{p}$.
$\Pr[\mathsf{V}_3]$: First, the batching of v_a, v_b, v_c is equivalent to testing the polynomial identity of a degree-2 polynomial at a random point; therefore, the batched equation holds with probability $\frac{2}{p}$ by the Schwartz-Zippel Lemma (Lemma 4.2). If the equation holds, the first sumcheck is guaranteed to pass; otherwise, the first sumcheck passes with probability at most $\frac{2n}{p}$. Note that the two sumchecks are independent since they involve completely different polynomials. Thus, the total probability is $\frac{2n+2}{p}$.
Putting things together: Thus, the soundness error of the entire PIOR is upper-bounded by

$$\frac{4n}{p} + \frac{2n+2}{p} = \frac{6n+2}{p} \leq \frac{8n}{2^{\lambda-1}} = \frac{16n}{2^{\lambda}}$$

Oracle: No oracle is communicated.

Prover Work: The prover does not need to do any work besides the sumchecks in Step 2 and Step 3.
In Step 2, for $h \in \{a, b, c\}$, computing the evaluation table of h takes $SN = O(N)$ operations in $\mathbb{Z}_p$. The sumcheck in Step 2 is degree 3 and has n rounds. In every round $s \in [n]$ of this sumcheck, the prover needs to compute a degree-3 univariate polynomial with a sum over $\boldsymbol{i} \in \{0,1\}^{n-s}$. Computing this polynomial takes $\frac{20N}{2^s}$ operations in $\mathbb{Z}_p$. Overall, computing the prover messages takes at most

$$20N \sum_{s=1}^{n} \frac{1}{2^s} = 20N(1 - \frac{1}{N}) \leq 20N \text{ operations in } \mathbb{Z}_p$$

After each round of the sumcheck, the prover needs to collapse each polynomial evaluation table to bind one more random challenge. The cost of collapsing each table is 2^{n-s} operations in $\mathbb{Z}_p$, and the total cost of collapsing tables is at most $5 \cdot 2^n = 5N$ operations in $\mathbb{Z}_p$.
Overall, prover work in Step 2 is $O(N) + 20N + 5N = O(N)$ operations in $\mathbb{Z}_p$.
In Step 3, for $h \in \{a, b, c\}$, computing the evaluation table of $h_{\boldsymbol{r}_i}$ takes $2N$ field operations.
Each prover message in the sumcheck is a univariate degree-2 polynomial that takes $9N/2^s$ operations to compute in round $s \in [n]$. In total, they cost

$$9N \sum_{s=1}^{n} \frac{1}{2^s} = 9N(1 - \frac{1}{N}) \leq 9N \text{ operations in } \mathbb{Z}_p$$

Collapsing each table of $h_{\boldsymbol{r}_i}$ for $h \in \{a, b, c\}$ takes at most $2^{n/2+1} = 2\sqrt{N}$ operations in total.
Overall, prover work in Step 3 is $9N + 2\sqrt{N}$ operations in $\mathbb{Z}_p$.
Putting everything together, the total prover work of PIOR 6.2 is $O(N)$ operations in $\mathbb{Z}_p$ with small constants.

Proof Size: In Step 2, there are a total of n prover messages in the sumcheck, each of which is a univariate degree-3 polynomial represented by 4 evaluation points. At the end of this sumcheck, the prover sends five claimed evaluations in $\mathbb{Z}_p$.
In Step 3, there are a total of n prover messages, each of which is a univariate degree-2 polynomial represented

by 3 evaluation points. At the end of this sumcheck, the prover sends three claimed evaluations in $\mathbb{Z}_p$.

Verifier work: The verifier performs a constant number of $\mathbb{Z}_p$ operations per round of the sumchecks. Moreover, the verifier needs to directly check mods. By our assumption, mods is highly structured and can be evaluated using a constant number of operations.

Oracle Queries: The verifier queries $[[\mathsf{quos}]]$ once and $[[w]]$ once. □

Remark 6.5. Using this randomly sampled prime p will likely slow down prover performance as p is not a nicely structured prime like a Solinas prime. However, this only affects the prover's field work, while the bottleneck of the prover is often cryptographic work like group operations or hashes which are done in $\mathbb{F}$, not $\mathbb{Z}_p$.

6.3 Evaluating Sparse Matrices

Definition 6.6. *The relation* $\mathsf{R}_{\mathsf{SparseEval}}$ *is the set of all tuples of the form*

$$(\mathbb{x}; \mathbb{o}; \mathbb{w}) := ((p, v, \boldsymbol{r}_i, \boldsymbol{r}_j); ([[\mathsf{col}]], [[\mathsf{row}]], [[\mathsf{val}]]); (\mathsf{col}, \mathsf{row}, \mathsf{val}))$$

such that for any $p \in \mathsf{Primes}(2^{\lambda-1}, 2^{\lambda}), v \in \mathbb{Z}_p, \boldsymbol{r}_i \in \mathbb{Z}_p^n, \boldsymbol{r}_j \in \mathbb{Z}_p^n,$ $\mathsf{col}, \mathsf{row}, \mathsf{val} \in \mathbb{Z}^{\leq 1}[s]$*,the following holds,*

$$\sum_{\boldsymbol{t} \in \{0,1\}^s} \mathsf{val}(\boldsymbol{t}) \cdot \mathbb{1}_{\mathsf{row}}(\boldsymbol{t}, \boldsymbol{r}_i) \cdot \mathbb{1}_{\mathsf{col}}(\boldsymbol{t}, \boldsymbol{r}_j) \equiv_p v \tag{9}$$

where $\mathbb{1}_{\mathsf{row}}(\boldsymbol{t}, \boldsymbol{i})$ *is the multilinear indicator polynomial that outputs* 1 *if* $\mathsf{row}(\boldsymbol{t}) = \boldsymbol{i}$ *and* 0 *otherwise, for* $\boldsymbol{t}, \boldsymbol{i}$ *over the hypercube; similarly for* $\mathbb{1}_{\mathsf{col}}(\boldsymbol{t}, \boldsymbol{j})$.

Protocol 6.3: For proving $(\mathbb{x}; \mathbb{o}; \mathbb{w}) \in \mathsf{R}_{\mathsf{SparseEval}}$, where $\mathbb{o}$ is a collection of mod-oracles.

Parse: $\mathbb{x} = (p, v, \boldsymbol{r}_i, \boldsymbol{r}_j)$; $\mathbb{o} = ([[\mathsf{col}]], [[\mathsf{row}]], [[\mathsf{val}]])$; $\mathbb{w} = (\mathsf{col}, \mathsf{row}, \mathsf{val})$

1. Prover and verifier engage in a PIOP to prove the following,
$$\sum_{\boldsymbol{t} \in \{0,1\}^s} \mathsf{val}(\boldsymbol{t}) \cdot \mathbb{1}_{\mathsf{row}}(\boldsymbol{t}, \boldsymbol{r}_i) \cdot \mathbb{1}_{\mathsf{col}}(\boldsymbol{t}, \boldsymbol{r}_j) \equiv_p v \tag{9}$$
2. At the end of the sumcheck(s), verifier needs to check the following evaluations
$$\mathsf{val}(\boldsymbol{r}_t) \equiv_p v_{\mathsf{val}}; \quad \mathsf{row}(\boldsymbol{r}_t) \equiv_p v_{\mathsf{row}}; \quad \mathsf{col}(\boldsymbol{r}_t) \equiv_p v_{\mathsf{col}}$$
where $\boldsymbol{r}_t \in \mathbb{Z}_p^s$ are random challenges.
Verifier queries the mod-oracles $[[\mathsf{col}]], [[\mathsf{row}]], [[\mathsf{val}]]$ at $\boldsymbol{r}_t$ to check these claims.

Remark 6.7. Given any PIOP Π for the first step of Protocol 6.3. Then Protocol 6.3 has three mod-oracle queries in addition to the properties of Π.

Remark 6.8. The properties of Protocol 6.3 depend on the specific PIOP chosen for the first step. Besides [BCD25], one can also use other lookup arguments but intermediate commitments will result in further invocations of the Integer Evaluation Protocol (PIOR 5.4).

6.4 The Overall Mod-PIOP and Its Compilation into An Argument

Theorem 6.9. *Given a (mod-)PIOP for* $\mathsf{R}_{\mathsf{SparseEval}}$ *that is perfectly complete with soundness error* $\epsilon_{\mathsf{sparse}}$*, there exists a (mod-)PIOP for* $\mathsf{R}_{\mathsf{imodR1CS}}$ *that has completeness, soundness error* $\frac{32n+\mathsf{negl}(2^\lambda)}{2^\lambda} + \epsilon_{\mathsf{sparse}}$*, and the following properties (excluding the cost of the* $\mathsf{R}_{\mathsf{SparseEval}}$ *PIOP).*

# Rounds	$2n+1$
(Mod-)Oracle #, Length, Size	2, $2^{n_w}+2^n$, $2^{n_w}\log \mathsf{T}_z + 2^n \log \mathsf{T}_{\mathsf{quos}}$
Prover Work	$O(N\lambda)$
Proof Size	$(7n+8)\lambda$
Verifier Work	$O(n\lambda)$
Oracle Queries	11

Proof. A mod-PIOP for $\mathsf{R}_{\mathsf{imodR1CS}}$ can be obtained by connecting Protocol 6.1, PIOR 6.2 , and a PIOP for $\mathsf{R}_{\mathsf{SparseEval}}$. Perfect completeness, number of rounds, oracles, and other properties follow trivially from the properties of the three components.

Soundness: The soundness error of the mod-PIOP is the sum of the soundness errors of each of its components. That is,

$$\frac{8n+O(\log \mathsf{T})}{2^\lambda} + \frac{16n}{2^\lambda} + \epsilon_{\mathsf{sparse}} \leq \frac{32n+O(\log \mathsf{T})}{2^\lambda} + \epsilon_{\mathsf{sparse}}$$

□

Succinct interactive argument for $\mathsf{R}_{\mathsf{imodR1CS}}$. First, apply the integer mod-PCS compiler (Section 5) with parameters set as in Corollary 5.8 to an underlying correct PCS whose Eval has knowledge soundness error ϵ_{PCS}, using some range check PIOP that is perfectly complete and has knowledge soundness error, to obtain an integer mod-PCS. Then, use this integer mod-PCS construction to compile the integer mod-PIOP for $\mathsf{R}_{\mathsf{imodR1CS}}$ results in a public-coin interactive argument of knowledge that has perfect completeness, (knowledge) soundness error $\frac{32n}{2^\lambda} + O(\epsilon_{\mathsf{sparse}}) + O(\epsilon_{\mathsf{rc}}) + O(\epsilon_{\mathsf{PCS}})$, and the following properties (excluding the cost of the underlying PCS, the range check PIOP, and the $\mathsf{R}_{\mathsf{SparseEval}}$ PIOP).

# Rounds	$2n + O(\log\log \mathsf{T}_z, \log\log \mathsf{T}_{\mathsf{quos}}) + 1$
# Commitments, Length, Size	2, $2^{n_w}+2^n$, $2^{n_w}\log \mathsf{T}_z + 2^n \log \mathsf{T}_{\mathsf{quos}}$
Prover Work	$O(N(\log \mathsf{T}_z + \log \mathsf{T}_{\mathsf{quos}})\log^2 \lambda)$
Proof Size	$O(\log \mathsf{T}_z + \log \mathsf{T}_{\mathsf{quos}} + n\lambda)$
Verifier Work	$O((n + \log\log \mathsf{T}_z + \log\log \mathsf{T}_{\mathsf{quos}})\lambda \log \lambda)$
# Eval Proofs	$11 \log \lambda$

Non-interactivity by the Fiat-Shamir transform. Public-coin interactive arguments of knowledge with round-by-round soundness can be made non-interactive by applying the Fiat-Shamir transform [AFK21]. For readability, we have only proved (knowledge) soundness of each component in our construction. We note that in the compiled mod-PCS construction, $\mathsf{modPCS.Commit}$ is non-interactive if we choose a non-interactive range check PIOP; the interactions in $\mathsf{modPCS.Eval}$ are purely sumcheck-based, assuming the underlying field PCS has a non-interactive Eval, which implies it is round-by-round sound [CCHLRR18].
Protocol 6.1 is non-interactive. The interactive Protocol 6.3 is purely sumcheck-based, which implies it is round-by-round sound [CCHLRR18]. Finally, by picking a $\mathsf{R}_{\mathsf{SparseEval}}$ protocol that is non-interactive, the

overall mod-PIOP for $\mathsf{R}_{\mathsf{imodR1CS}}$ is round-by-round sound. Furthermore, since round-by-round soundness implies round-by-round knowledge soundness for mod-PIOPs (Lemma 4.22) where the witness consists only of integer polynomials such that the integer mod-oracles to these polynomials are contained in either $\mathfrak{D}$ or the transcript of the protocol, which is consistent with our mod-PIOP, our mod-PIOP for $\mathsf{R}_{\mathsf{imodR1CS}}$ is also round-by-round knowledge sound. Therefore, a SNARK for $\mathsf{R}_{\mathsf{imodR1CS}}$ can be obtained by applying the Fiat-Shamir transform to both our integer mod-PIOP and Limber, respectively, and then compiling them together. We note that both our mod-PIOP and Limber are round-by-round sound, since all interactions are purely sumcheck-based [CCHLRR18]. By Lemma 4.22, round-by-round soundness implies round-by-round knowledge soundness for our protocols. Therefore, our integer mod-PIOP and Limber together with the Fiat-Shamir Transform yield a SNARK.

6.5 Coprocessor Model

Non-native arithmetic inherently has overhead, even with Limber. For computation with both non-native and native arithmetic, or different kinds of non-native arithmetics (e.g. different number of limbs, different extension fields for extension field arithmetic) in the same computation, we would like to avoid paying the maximal overhead for *all* of the constraints. The goal is to have the prover only pay the higher commitment and evaluation cost for each specific part of computation. Roughly speaking, in the non-native vs native example, we can split the witness and circuit into non-native and native parts, and have the prover only use Limber for the non-native portion, while the native PCS can be used for the native portion. Protocol 6.4 is an informal sketch of a PIOP where the computation includes both non-native and native arithmetic. We note that Step 2 is supported by our specific integer mod-PCS (Section 5).

Protocol 6.4: (Informal) PIOP for Coprocessor Model

Given the limb-split non-native polynomials, the native polynomials, and the relevant standard oracles, do the following:

1. Prover and Verifier combine the limb-split non-native polynomials with the native polynomials, and run a PIOP for standard R1CS on these.
2. Prover derives mod-oracles on the non-native polynomials from standard oracles of the limb-split non-native polynomials. Prover and Verifier run PIOR 6.2 using these mod-oracles.

7 Implementation

We implement Limber (Section 5) and the Spartan-style integer mod-PIOP for $\mathsf{R}_{\mathsf{imodR1CS}}$ (deferred to Section 6.4) in 9000 lines of Rust [Imp]. Limber is agnostic to the underlying PCS; for the benchmarks, we instantiate it with the Hyrax PCS [WTSTW18] over the Tom256 elliptic curve [FLM22], which has a base and scalar field of 256-bit prime characteristic, and with the code-based Brakedown PCS [GLSTW23] over the same scalar field. We instantiate the batch range check protocol with LogUp GKR [PH23].

Optimizations and parameters. We highlight the main engineering choices behind the compiled prover. First, we reuse the committed limb polynomial $\hat{f}_\ell$ across both the commit-phase range check and the Mod-PCS evaluation, so the witness is committed only once. For the IntEval parameters, we use the settings in Table 4 across all of the following experiments.

We evaluate our techniques by comparing our fully compiled SNARK for integers to circuit-based approaches on a well-studied non-native arithmetic benchmark and on a mixed-field Poseidon2 hashing

Parameter	Value	Role
q	$\sim 2^{256}$	Field characteristic/size
T	2^{64}	Size of a limb
k	9	Number of partially evaluated variables per IntEval iteration
s	15	Number of IntEval CRT primes; corresponds to field work
Commitment overhead	~ 0.13	Extra committed data vs. witness

Table 4: IntEval parameters used in all experiments. These place us in the low-overhead regime of Table 3; the commitment overhead can be driven lower by increasing k.

benchmark, and by comparing the full SNARK to plain Spartan at the same constraint count to isolate the overhead of our approach. All benchmarks are run single-threaded on a MacBook Pro (Apple M4 Pro chip, 24 GB RAM, 14 cores); reported numbers are medians over 10 runs, and prover times include witness generation and commitment.

7.1 RSA Accumulator Application

RSA accumulators are a common real-world application for non-native arithmetic: each accumulator update is a modular exponentiation over a 2048-bit RSA modulus. We use the previously explored benchmark of two RSA accumulator updates and a hash-to-prime evaluation from [OWWB20]: four Wesolowski exponentiations with 322-bit exponents modulo an RSA-2048 modulus, plus the Poseidon-based hash-to-prime with its Pocklington certificate. We compare against two circuit-based approaches—Arkworks [ark22], a widely used SNARK library, and MultiSwap [OWWB20], a circuit hand-optimized for this benchmark—as well as the concurrent integer SNARK Zinc+ [Abd+26]. The results are in Table 1; all baselines were run on the same machine rather than quoted from prior work. With Hyrax, our prover is $15\times$ faster than the MultiSwap circuit and $30\times$ faster than the Arkworks circuit; with Brakedown, the prover and verifier are slightly faster than with Hyrax, at the cost of an 8.2MB proof. Against Zinc+, our Hyrax prover is $4.8\times$ faster, our verifier is $5.8\times$ faster, and our proof is $3.4\times$ smaller (273KB vs 0.94MB).

7.2 Mixed-Field Poseidon2 Hashing

We also evaluate a hashing workload with three different non-native prime fields: thirty Poseidon2 compressions as three independent ten-compression chains over the scalar fields of BN254, BLS12-381, and secp256k1. The results are in Table 5. Our Hyrax prover is $24\times$ faster than the circuit-based baseline with a $62\times$ faster verifier and a $2.5\times$ smaller proof, and $1.2\times$ ($1.6\times$ with Brakedown) faster than Zinc+ with a $1.2\times$ faster verifier and a $25\times$ smaller proof.

7.3 Overhead over Native Constraints

To isolate the cost of our non-native arithmetic itself, we compare our full SNARK against plain Spartan on the same number of constraints with the same PCS (Hyrax) and the same prover implementation; the main difference is that Spartan operates over native 256-bit $\mathbb{Z}_q$ constraints while our system proves non-native modulo constraints of the same size also with 256-bit moduli. We note that because $\mathsf{T} = 2^{64}$, each 256-bit

[g] Zinc+ with a 256-bit sampled prime; with a 320-bit prime the prover takes 546ms, the verifier 32.3ms, and the proof is 3.1MB.

	# Constraints	Prove	Verify	Proof size
Bellpepper circuit (with Spartan)	9.45 million	10.6s	1.53s	340KB
Zinc+ [Abd+26] (concurrent)	$2^{10} \times 72$ trace[g]	528ms	29.8ms	3.4MB
Limber-Spartan (with Hyrax)	12,990	438ms	24.7ms	136KB
Limber-Spartan (with Brakedown)	12,990	334ms	33.0ms	3.8MB

Table 5: Thirty Poseidon2 compressions over three non-native prime fields (BN254-Fr, BLS12-381-Fr, secp256k1-Fr) in one proof. The constraint counts for Limber are integer constraints padded to 2^{14}; the Zinc+ count is the size of its execution trace; the circuit-based baseline is R1CS over the Tom256 scalar field, padded to 2^{24}. Proof sizes are zstd-compressed. All numbers are single-threaded medians over 10 runs on a MacBook (Apple M4 Pro chip, 24 GB RAM, 14 cores).

witness value splits into 4 limbs, and each limb is committed as four 16-bit chunks in order to share the commitment with the 16-bit range check, so the committed polynomial has $16\times$ as many (small) values as plain Spartan's. We also require additional range checks that plain Spartan does not. These two factors are the main contributors to the overhead, while our integer mod-PCS machinery only adds a small sublinear overhead: the additional commitment overhead is only 13% of the original committed witness and quotient polynomials. Despite these seemingly inherent overheads, our prover is only 5 to $9\times$ slower across 2^{10} to 2^{18} constraints, as shown in Figure 3. Above 2^{18}, the prover ratio stays roughly the same, but we note that reasonable applications likely fall below the 2^{18} circuit in size; the MultiSwap circuit is approximately the workload for the 2^{17} circuit and the Poseidon2 circuit is similar to 2^{14}. Our verifier stays under 30 ms for up to 2^{14} constraints compared to 14–16 ms for plain Spartan, and is $3.2\times$ slower at 2^{18} constraints (158 ms vs 49 ms); the verifier gap comes from the Hyrax openings of the larger committed data, the additional IntEval verification, and the LogUp-GKR range check that run on top of Spartan's checks.

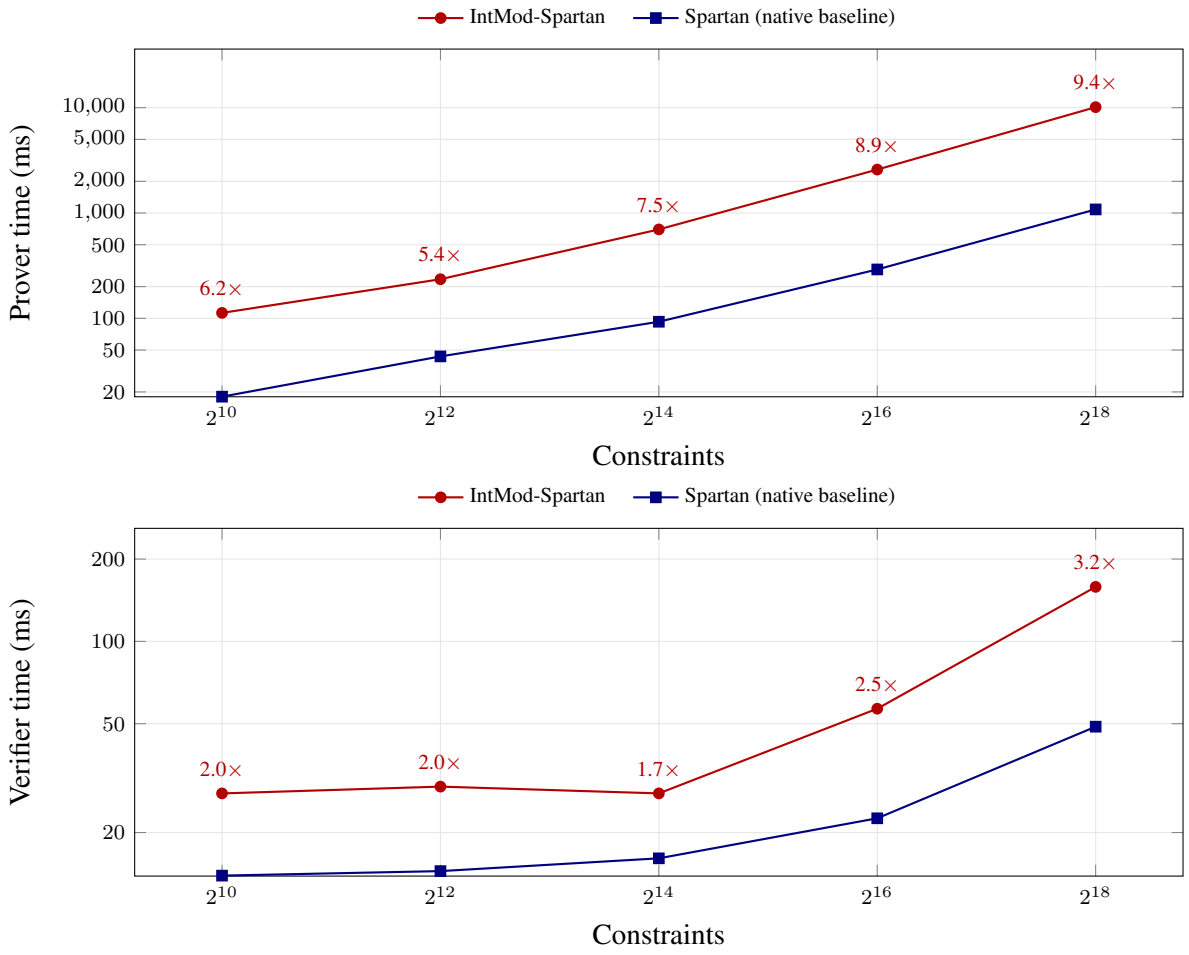


Figure 3: Prover time and verifier time comparison between plain Spartan and Limber-Spartan, our compiled SNARK for integers, at equal constraint counts. Single-threaded medians over 10 runs; error bars show the spread across runs; prover time includes witness generation and commitment.

8 Discussion and Future Work

We have shown that we can construct SNARKs for integer computation with minimal commitment overhead from almost any underlying polynomial commitment scheme. This directly enables efficient and robust SNARKs for non-native arithmetic, which is a crucial bottleneck in many real-world applications. Our implementation demonstrates that our approach is practical and achieves significant speedups and flexibility over prior work.

We leave two open directions for future work. First, we would like to remove any requirement on the characteristic of the PCS field—without significantly growing the commitment overhead or prover work—which would in particular allow compiling binary-field PCSs to integer mod-PCSs.

Second, range proofs—a key component of many integer SNARK constructions—often account for a significant portion of the prover time in our experiments; faster range proofs would directly improve our construction and several of the other schemes.

Acknowledgments

This work was generously supported by Google, DTCC, and Chaincode.

References

[AABDS20] Abdelrahaman Aly, Tomer Ashur, Eli Ben-Sasson, Siemen Dhooghe, and Alan Szepieniec. "Design of Symmetric-Key Primitives for Advanced Cryptographic Protocols". In: 2020.3 (2020), pp. 1–45. DOI: 10.13154/tosc.v2020.i3.1-45.

[Abd+26] Alexander Abdugafarov, Albert Garreta, Amit Kumar, Michał Osadnik, Psi Vesely, Ilia Vlasov, and Kai Zhe Zheng. *Zinc+: SNARKs for Polynomial Rings*. Cryptology ePrint Archive, Paper 2026/855. 2026. URL: https://eprint.iacr.org/2026/855.

[AFK21] Thomas Attema, Serge Fehr, and Michael Klooß. *Fiat-Shamir Transformation of Multi-Round Interactive Proofs*. Cryptology ePrint Archive, Report 2021/1377. 2021. URL: https://eprint.iacr.org/2021/1377.

[AFQ25] Miguel Ambrona, Denis Firsov, and Iñigo Querejeta-Azurmendi. *Efficient Foreign-Field Arithmetic in PLONK*. Cryptology ePrint Archive, Paper 2025/695. 2025. URL: https://eprint.iacr.org/2025/695.

[AGRRT16] Martin Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. *MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity*. Cryptology ePrint Archive, Report 2016/492. 2016. URL: https://eprint.iacr.org/2016/492.

[AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. "Ligero: Lightweight Sublinear Arguments Without a Trusted Setup". In: *Proceedings of the 24th ACM Conference on Computer and Communications Security*. CCS '17. 2017, pp. 2087–2104. DOI: 10.1145/3133956.3134104.

[ark22] arkworks contributors. *arkworks zkSNARK ecosystem*. 2022. URL: https://arkworks.rs.

[AST24] Arasu Arun, Srinath T. V. Setty, and Justin Thaler. "Jolt: SNARKs for Virtual Machines via Lookups". In: 2024, pp. 3–33. DOI: 10.1007/978-3-031-58751-1_1.

[Azt24] Aztec Labs. *Barretenberg: `bigfield` non-native field arithmetic*. GitHub repository. Accessed 2026-06-05. 2024. URL: https://github.com/AztecProtocol/aztec-packages.

[BCD25] Benedikt Bünz, Jessica Chen, and Zachary DeStefano. *Linear*-Time Permutation Check*. Cryptology ePrint Archive, Paper 2025/1850. 2025. URL: https://eprint.iacr.org/2025/1850.

[BCFRRZ24] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. *Blaze: Fast SNARKs from Interleaved RAA Codes*. Cryptology ePrint Archive, Paper 2024/1609. 2024. URL: https://eprint.iacr.org/2024/1609.

[BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. *Linear-Time Accumulation Schemes*. Cryptology ePrint Archive, Paper 2025/753. 2025. URL: https://eprint.iacr.org/2025/753.

[BCTV14a] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves". In: 2014, pp. 276–294. DOI: 10.1007/978-3-662-44381-1_16.

[BCTV14b] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves". In: *Proceedings of the 34th Annual International Cryptology Conference*. CRYPTO '14. Extended version at http://eprint.iacr.org/2014/595. 2014, pp. 276–294.

[Ben+14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. "Zerocash: Decentralized Anonymous Payments from Bitcoin". In: *Proceedings of the 2014 IEEE Symposium on Security and Privacy*. SP '14. 2014, pp. 459–474. DOI: 10.1109/SP.2014.36.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. "Transparent SNARKs from DARK Compilers". In: 2020, pp. 677–706. DOI: 10.1007/978-3-030-45721-1_24.

[BHRRS21] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. "Time- and Space-Efficient Arguments from Groups of Unknown Order". In: 2021, pp. 123–152. DOI: 10.1007/978-3-030-84259-8_5.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: 2023, pp. 499–530. DOI: 10.1007/978-3-031-30617-4_17.

[CCHLRR18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. *Fiat-Shamir From Simpler Assumptions*. Cryptology ePrint Archive, Paper 2018/1004. 2018. URL: https://eprint.iacr.org/2018/1004.

[CFHKKO21] Matteo Campanelli, Dario Fiore, Semin Han, Jihye Kim, Dimitris Kolonelos, and Hyunok Oh. *Succinct Zero-Knowledge Batch Proofs for Set Accumulators*. Cryptology ePrint Archive, Report 2021/1672. 2021. URL: https://eprint.iacr.org/2021/1672.

[CHA24] Matteo Campanelli and Mathias Hall-Andersen. *Fully Succinct Arguments over the Integers from First Principles*. Cryptology ePrint Archive, Paper 2024/1548. 2024. URL: https://eprint.iacr.org/2024/1548.

[CHMMVW20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: 2020, pp. 738–768. DOI: 10.1007/978-3-030-45721-1_26.

[Edi23] Immunefi Editor. "Aztec Multiple-Spend Error Bugfix Review". In: *Immunefi* (2023). URL: https://medium.com/immunefi/aztec-multiple-spend-error-bugfix-review-20074581d224.

[EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. *cq: Cached quotients for fast lookups*. Cryptology ePrint Archive, Report 2022/1763. 2022. URL: https://eprint.iacr.org/2022/1763.

[FLM22] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. "ZKAttest: Ring and Group Signatures for Existing ECDSA Keys". In: 2022, pp. 68–83. DOI: 10.1007/978-3-030-99277-4_4.

[FS87] Amos Fiat and Adi Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: 1987, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.

[GJJZ22] Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Yinuo Zhang. “Succinct Zero Knowledge for Floating Point Computations”. In: 2022, pp. 1203–1216. DOI: 10.1145/3548606.3560653.

[GKKRRS19] Lorenzo Grassi, Daniel Kales, Dmitry Khovratovich, Arnab Roy, Christian Rechberger, and Markus Schofnegger. *Starkad and Poseidon: New Hash Functions for Zero Knowledge Proof Systems*. IACR Cryptology ePrint Archive, Report 2019/458. 2019. URL: https://eprint.iacr.org/2019/458.

[GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. “Delegating computation: interactive proofs for muggles”. In: 2008, pp. 113–122. DOI: 10.1145/1374376.1374396.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. “Poseidon2: A Faster Version of the Poseidon Hash Function”. In: 2023, pp. 177–203. DOI: 10.1007/978-3-031-37679-5_8.

[GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. “Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS”. In: 2023, pp. 193–226. DOI: 10.1007/978-3-031-38545-2_7.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Report 2019/953. 2019. URL: https://eprint.iacr.org/2019/953.

[GWHD25] Albert Garreta, Hendrik Waldner, Katerina Hristova, and Luca Dall’Ava. Zinc*: Succinct Arguments with Small Arithmetization Overheads from IOPs of Proximity to the Integers*. Cryptology ePrint Archive, Paper 2025/316. 2025. URL: https://eprint.iacr.org/2025/316.

[HJRRR25] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. *Jagged Polynomial Commitments (or: How to Stack Multilinears)*. Cryptology ePrint Archive, Paper 2025/917. 2025. URL: https://eprint.iacr.org/2025/917.

[Imp] URL: https://github.com/lucasxia01/limber-impl/tree/main.

[KKSKKO23] Sungwook Kim, Sungju Kim, Yulim Shin, Sunmi Kim, Jihye Kim, and Hyunok Oh. “Efficient Transparent Polynomial Commitments for zk-SNARKs”. In: 2023, pp. 348–366. DOI: 10.1007/978-3-031-51479-1_18.

[KPS18a] Ahmed E. Kosba, Charalampos Papamanthou, and Elaine Shi. “xJsnark: A Framework for Efficient Verifiable Computation”. In: 2018, pp. 944–961. DOI: 10.1109/SP.2018.00018.

[KPS18b] Ahmed Kosba, Charalampos Papamanthou, and Elaine Shi. “xJsnark: A Framework for Efficient Verifiable Computation”. In: *2018 IEEE Symposium on Security and Privacy (SP)*. 2018, pp. 944–961. DOI: 10.1109/SP.2018.00018.

[KS23] Abhiram Kothapalli and Srinath Setty. *CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves*. Cryptology ePrint Archive, Report 2023/1192. 2023. URL: https://eprint.iacr.org/2023/1192.

[KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes". In: 2022, pp. 359–388. DOI: 10.1007/978-3-031-15985-5_13.

[LFKN90] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. "Algebraic Methods for Interactive Proof Systems". In: 1990, pp. 2–10. DOI: 10.1109/FSCS.1990.89518.

[LM22] Daniel Lubarov and Jordi Baylina Melé. *Casting out Primes: Bignum Arithmetic for Zero-Knowledge Proofs*. Cryptology ePrint Archive, Report 2022/1470. 2022. URL: https://eprint.iacr.org/2022/1470.

[LXZ21] Tianyi Liu, Xiang Xie, and Yupeng Zhang. "zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy". In: 2021, pp. 2968–2985. DOI: 10.1145/3460120.3485379.

[OWWB20] Alex Ozdemir, Riad S. Wahby, Barry Whitehat, and Dan Boneh. "Scaling Verifiable Computation Using Efficient Set Accumulators". In: 2020, pp. 2075–2092. URL: https://eprint.iacr.org/2019/1494.

[PH23] Shahar Papini and Ulrich Haböck. *Improving logarithmic derivative lookups using GKR*. Cryptology ePrint Archive, Paper 2023/1284. 2023. URL: https://eprint.iacr.org/2023/1284.

[Pol22] Polygon Zero. *Plonky2*. GitHub repository. Accessed 2026-06-05. 2022. URL: https://github.com/0xPolygonZero/plonky2.

[RS62] J. Barkley Rosser and Lowell Schoenfeld. "Approximate formulas for some functions of prime numbers". In: *Illinois Journal of Mathematics* 6.1 (1962), pp. 64 –94. DOI: 10.1215/ijm/1255631807. URL: https://doi.org/10.1215/ijm/1255631807.

[Sav+21] Sinem Sav, Apostolos Pyrgelis, Juan Ramón Troncoso-Pastoriza, David Froelicher, Jean-Philippe Bossuat, Joao Sa Sousa, and Jean-Pierre Hubaux. "POSEIDON: Privacy-Preserving Federated Neural Network Learning". In: 2021. DOI: 10.14722/ndss.2021.24119.

[Sch80] Jacob T. Schwartz. "Fast Probabilistic Algorithms for Verification of Polynomial Identities". In: *Journal of the ACM* 27.4 (1980), pp. 701–717. DOI: 10.1145/322217.322225.

[Set20] Srinath Setty. "Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup". In: 2020, pp. 704–737. DOI: 10.1007/978-3-030-56877-1_25.

[Sho09] Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. 2nd ed. Cambridge University Press, 2009. ISBN: 978-0-521-51644-0. URL: https://shoup.net/ntb/.

[SSPP26] Alireza Shirzad, Sriram Sridhar, Dimitrios Papadopoulos, and Charalampos Papamanthou. *Relaxed Modular PCS from Arbitrary PCS and Applications to SNARKs for Integers*. Cryptology ePrint Archive, Paper 2026/347. 2026. URL: https://eprint.iacr.org/2026/347.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. *Customizable constraint systems for succinct arguments*. Cryptology ePrint Archive, Report 2023/552. 2023. URL: https://eprint.iacr.org/2023/552.

[STW24] Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. "Unlocking the Lookup Singularity with Lasso". In: 2024, pp. 180–209. DOI: 10.1007/978-3-031-58751-1_7.

[Tha20] Justin Thaler. *Proofs, arguments, and zero-knowledge*. 2020.

[Wes19] Benjamin Wesolowski. "Efficient verifiable delay functions". In: EUROCRYPT '19 (2019), p. 623.

[Wik21] Douglas Wikström. *Special Soundness in the Random Oracle Model*. Cryptology ePrint Archive, Report 2021/1265. 2021. URL: https://eprint.iacr.org/2021/1265.

[WTSTW18] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. "Doubly-efficient zkSNARKs without trusted setup". In: *Proceedings of the 39th IEEE Symposium on Security and Privacy*. S&P '18. 2018, pp. 926–943.

[ZBKMNS22] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. "Caulk: Lookup Arguments in Sublinear Time". In: 2022, pp. 3121–3134. DOI: 10.1145/3548606.3560646.

[Zip79] Richard Zippel. "Probabilistic algorithms for sparse polynomials". In: *Proceedings of the 1979 International Symposium on Symbolic and Algebraic Computation*. EUROSAM '79. 1979, pp. 216–226. DOI: 10.1007/3-540-09519-5_73.

[Zkb] URL: https://github.com/0xPARC/zk-bug-tracker.

A Script for Computing Parameters of Integer Evaluation Protocol

```
def calc(log_q, log_p, lamb, mu, log_T):
    for k in range(1, 20):
        # max(T, P) * 2^k P^k < (q - P) / 2
        log_P = 0
        for lp in range(1, log_q):
            if max(log_T, lp) + k * lp + k <= log_q - 1:
                log_P = lp
                continue
            else:
                break
        if log_P <= 1:
            continue

        # (P/2)^t < 2^mu p^mu T
        t = (mu + log_p * mu + log_T) // (log_P - 1)

        # (t/(P/2ln_P))^s < 2^-lamb
        ln_P = log_P * math.log(2)
        denom = log_P - (math.log(t, 2) + math.log(2*ln_P, 2))
        if denom <= 0:
            continue
        s = math.ceil(lamb / denom)

        extra_commit_ratio = s * log_q / ((2**k - 1) * log_T)

        print(f"k={k}, log_P={log_P}, t={t}, s={s},
            extra_commit_ratio={extra_commit_ratio}")
```

B Additional Proofs

Corollary B.1. *For $P \geq 2^5$, $|\mathsf{Primes}(P/2, P)| > \frac{P}{4 \log P}$.*

Proof. By Theorem 4.28, $\pi(P) > \frac{P}{\ln P}$ and $\pi(P/2) < \frac{1.25506\,(P/2)}{\ln(P/2)}$. Since $\ln x = \ln 2 \cdot \log x$ (with $\log := \log_2$), writing $L := \log P$,

$$\begin{aligned} |\mathsf{Primes}(P/2, P)| = \pi(P) - \pi(P/2) &> \frac{P}{\ln 2}\Big(\frac{1}{L} - \frac{0.62753}{L-1}\Big) \\ &= \frac{P}{\ln 2 \cdot L} \cdot \frac{0.37247L - 1}{L-1}. \end{aligned}$$

The last factor is increasing in L, so for $P \geq 2^5$ (i.e. $L \geq 5$) it is at least $\frac{0.37247 \cdot 5 - 1}{4} > 0.21 > \frac{\ln 2}{4}$. Therefore $|\mathsf{Primes}(P/2, P)| > \frac{P}{4 \log P}$. □

C Use of Generative AI

We disclose that we used AI assistance to help generate the figures and to format the tables in this paper. All such content was reviewed and edited by the authors, who take full responsibility for the content of this work. In particular, none of the protocol constructions or proofs involved the use of AI.